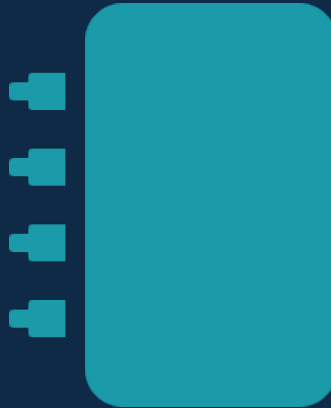


Module 1

VLSI Design Fundamentals

A complete 60-hour foundation — from silicon and the MOS transistor to CMOS logic, layout, interconnect and timing.

Comprehensive Training Deck · 127 slides · worked examples throughout





About this Module & Learning Outcomes

Module 1 builds the conceptual bedrock for the entire chip-design course. Everything in later modules — RTL, timing closure, FPGA — rests on the transistor-level understanding you gain here.

By the end you will be able to:

- Explain the VLSI design cycle and abstraction levels
- Describe how NMOS / PMOS transistors switch and conduct
- Analyse the CMOS inverter, its VTC, noise margins, delay & power
- Design any static gate (NAND, NOR, AOI) from a Boolean function
- Read schematics, stick diagrams and basic layout with design rules
- Model on-chip wires as RC and estimate interconnect delay
- Compute propagation delay and reason about setup/hold timing

At a glance

Duration	60 hours (theory + lab)
Prerequisite	Basic digital electronics
Tools introduced	NGSpice, Magic VLSI
Assessment	Concept + design problems
Leads into	M2 Verilog · M3 STA · M4 FPGA



Four Topics, One Foundation

The official Module 1 syllabus has four topic areas. This deck expands each into theory + multiple worked examples.

1

VLSI Design Cycle & CMOS Transistor Theory

Topics 1–2: VLSI flow & significance, MOS physics, I–V, switch model

~16 hrs

2

CMOS Inverter Characteristics & Logic Design

Topics 3–4: VTC, noise margins, delay, power, NAND/NOR/AOI, adders

~14 hrs

3

Layout, Interconnect & Packaging

Topics 5–7: schematics, layout, design rules, wire modelling, bonding & packaging

~14 hrs

4

Delay, Timing & Optimization

Topics 8–10: gate delays & logical effort, critical-path optimization, sequential timing

~16 hrs

SECTION 1

VLSI Design Cycle & CMOS Transistor Theory

How chips are designed, and the physics of the MOS switch that makes it all possible.

Topic 1.1 • approx. 18 hours

- ✓ VLSI technology, significance & applications
- ✓ Front-end & back-end design flow in detail
- ✓ Design styles: full-custom, standard-cell, FPGA
- ✓ CMOS theory: MOSFET operation, modeling & characteristics
- ✓ Square-law model, capacitances, second-order effects
- ✓ Scaling trends: Moore/Dennard, nodes, FinFET & GAA

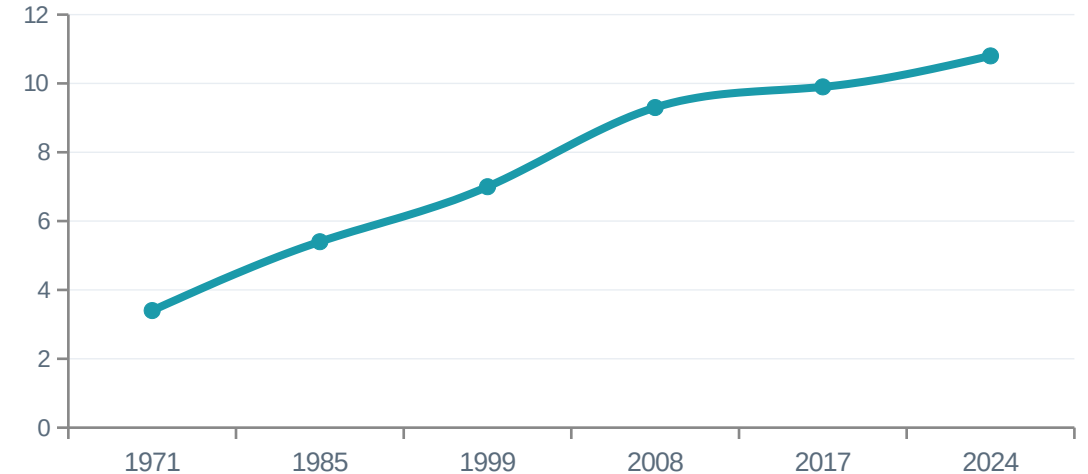


What is VLSI?

VLSI (Very Large Scale Integration) is the process of creating an integrated circuit by combining millions–billions of transistors onto one silicon chip — enough to build a whole digital system (a System-on-Chip).

SSI	1960s	~1–10 gates
MSI	1970s	~10–100 gates
LSI	late 70s	~hundreds–thousands
VLSI	1980s+	tens of thousands+
ULSI / SoC	today	millions–billions

Moore's Law (log scale)



Example — Moore's Law in numbers

Intel 4004 (1971): ~2,300 transistors.

Apple M-series / modern SoC: tens of billions.

That is roughly a doubling every ~2 years for 50 years.



Evolution of IC Integration

Integrated circuits have grown through five named eras, each defined by how many devices fit on one chip. VLSI is the era that made the modern computer possible.

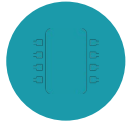
SSI	Small-Scale Integration <i>early 1960s</i>	~1–10 gates	Single logic gates, flip-flops
MSI	Medium-Scale Integration <i>late 1960s</i>	~10–100 gates	Adders, counters, multiplexers
LSI	Large-Scale Integration <i>1970s</i>	~1k–10k devices	First microprocessors, memories
VLSI	Very-Large-Scale Integration <i>1980s onward</i>	tens of thousands+	Full CPUs on one chip
ULSI / SoC	Ultra-Large-Scale / System-on-Chip <i>today</i>	millions–billions	Entire systems on one die

The pace — a doubling of transistor count roughly every two years — is Moore's Law, and it has held for ~50 years.



Where VLSI Powers the World

Almost every electronic product you touch is enabled by VLSI chips. Integration is what makes them small, fast, cheap and power-efficient enough to be everywhere.



Computing

CPUs, GPUs, memory — laptops to data-centre servers



Mobile & Wireless

Smartphone SoCs, 5G modems, Wi-Fi & Bluetooth radios



AI & Accelerators

Neural-network chips, TPUs, edge-AI inference engines



Automotive

Engine/EV control, ADAS, sensors, infotainment



IoT & Wearables

Ultra-low-power microcontrollers & sensor hubs



Healthcare & Comms

Medical imaging, implants, networking & satellites



Why VLSI Matters

Integration is not just convenient — it fundamentally improves every design metric, and it underpins a strategically vital global industry.

Technical advantages of integration

- ✓ **Smaller size:** Billions of devices on a fingernail-sized die
- ✓ **Higher speed:** Short on-chip wires → faster signalling
- ✓ **Lower power:** Tiny capacitances switch with little energy
- ✓ **Lower cost:** Batch fabrication → very low cost per chip at volume
- ✓ **Better reliability:** Fewer external connections → fewer failure points

Economic & strategic importance

Semiconductors are a **trillion-dollar** global industry underpinning all modern electronics.

Chip design (front-end) and fabrication (foundry) are increasingly **separate businesses** — the fabless model.

Nations treat chip capability as **strategic infrastructure**; India's Semiconductor Mission aims to build design & fab talent.

This course trains the **design** side — exactly the skills in highest demand.



Why VLSI? Design Metrics (PPA)

Integrating more on one chip improves almost everything that matters. Chip designers constantly trade off three metrics — Power, Performance, Area (PPA) — plus cost and reliability.



Performance

Speed / clock frequency, throughput, latency



Power

Dynamic + leakage power; battery life & heat



Area

Silicon area → cost per chip; smaller = cheaper



Cost

Design (NRE) + per-unit manufacturing cost



Reliability

Noise immunity, yield, lifetime



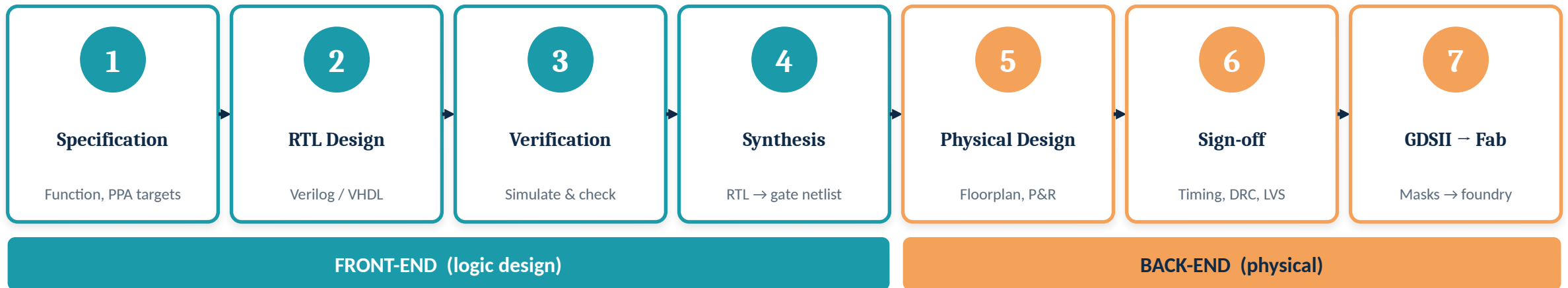
Trade-off example

Widening a transistor **speeds up** a gate (lower R) but **costs area & power** (higher C). PPA is always a balance.



The VLSI Design Flow — Big Picture

A chip is built through well-defined stages, each refining the design and adding detail from idea to fabricated silicon. The flow splits into a logic-focused FRONT-END and a geometry-focused BACK-END.



The hand-off between them is the **gate-level netlist** produced by synthesis.

- Each stage is verified before the next — a bug caught late costs far more to fix.



Where the course tools fit

Yosys → synthesis; **OpenSTA** → timing;
NGSpice → simulation; **Magic** → layout/DRC;
GTKWave → waveforms.



Front-End Design Flow (Logic)

The front-end turns an idea into a verified, technology-mapped gate-level netlist. It is all about defining and proving correct behaviour — no physical geometry yet.

1 Specification

Capture function, interfaces and PPA targets in a spec / micro-architecture.

2 Architectural design

Partition into blocks (datapath, control, memory); define the high-level plan.

3 RTL coding

Describe behaviour in Verilog/VHDL at the register-transfer level.

4 Functional verification

Simulate against a testbench; check coverage. (GTKWave views waveforms.)

5 Logic synthesis

Map RTL to a gate-level netlist in a target cell library. (Yosys.)

6 DFT & formal checks

Insert scan/test logic; run equivalence checking and early timing.

Output of the front-end → a verified gate-level netlist, handed to the back-end.



Back-End Design Flow (Physical)

The back-end turns the netlist into manufacturable geometry (GDSII). It is all about placing cells, wiring them, and proving the layout meets timing and fabrication rules.

- | | | |
|---|------------------------------|--|
| 1 | Floorplanning | Fix the die size, block locations, power grid and I/O placement. |
| 2 | Placement | Position every standard cell on the floorplan, minimising wire length. |
| 3 | Clock-tree synthesis | Build a balanced clock network so all flip-flops switch together. |
| 4 | Routing | Connect all cells with metal layers, honouring spacing rules. |
| 5 | Extraction & STA | Extract real R/C parasitics; sign off timing on every path. (OpenSTA.) |
| 6 | Physical verification | Run DRC and LVS; export GDSII for the foundry. (Magic.) |

Output of the back-end → GDSII layout, the file a foundry uses to make photomasks.



Front-End vs Back-End

Both halves are essential and tightly linked through the netlist hand-off. Knowing which concerns live where helps you debug and optimise effectively.

Aspect	Front-End	Back-End
Concern	What the chip does (logic)	Where everything physically goes
Main inputs	Specification	Gate-level netlist + constraints
Key activities	RTL, verification, synthesis	Floorplan, place, CTS, route
Verified by	Functional sim, equivalence	STA, DRC, LVS
Primary output	Gate-level netlist	GDSII layout
Course tools	Yosys, GTKWave, NGSpice	OpenSTA, Magic

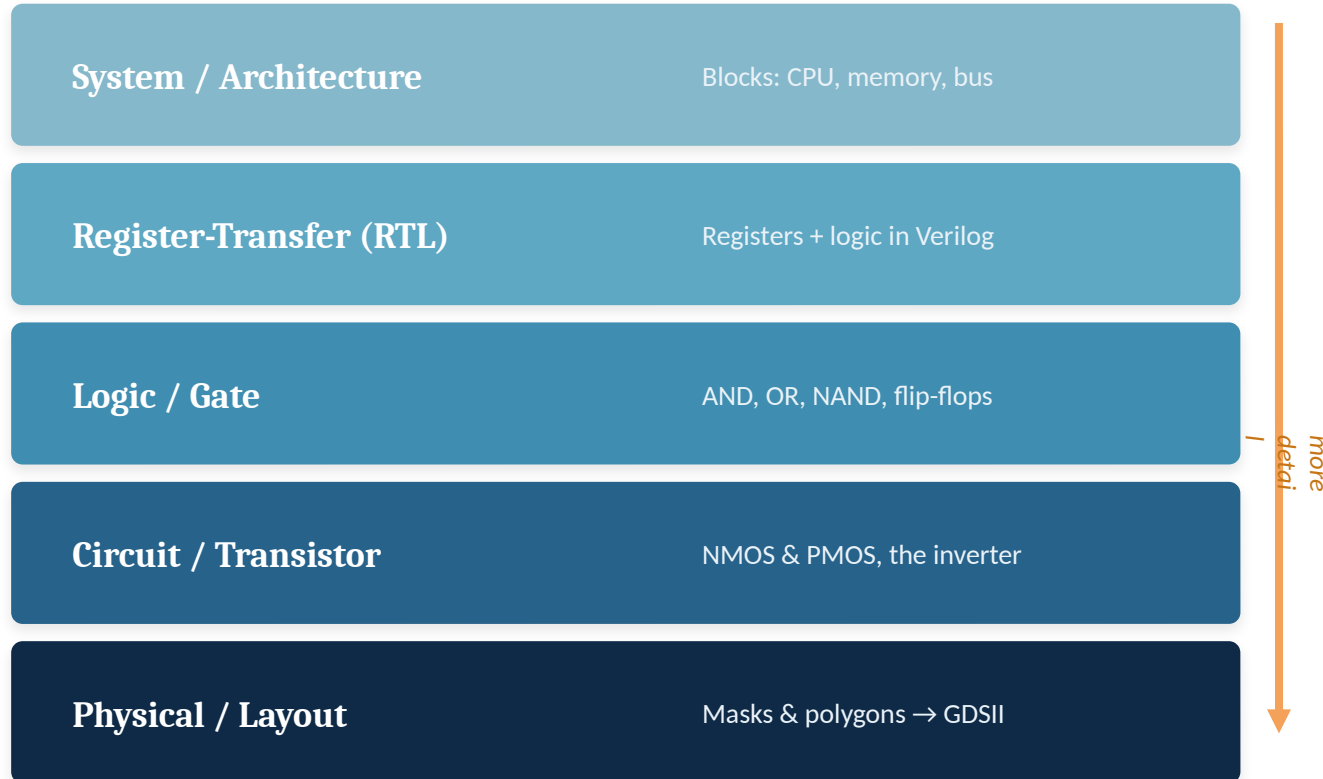


The hand-off

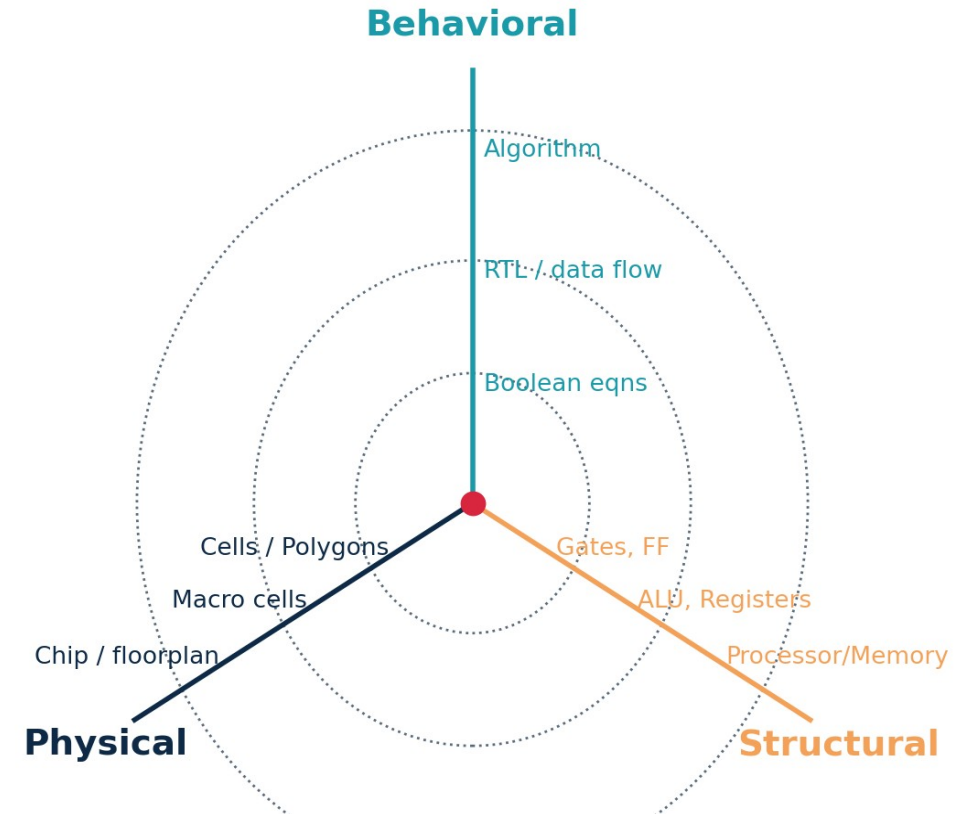
Synthesis ends the front-end by emitting a **gate-level netlist**; the back-end consumes it. A timing problem found in back-end STA may send the design all the way back to RTL — which is why front-end quality matters so much.



Levels of Abstraction & the Y-Chart



Gajski-Kuhn Y-Chart: three design domains





Design Styles & ASIC vs FPGA

The same logic can be realised in different ways, trading design effort, cost, performance and flexibility.



Full-custom

Every transistor hand-drawn. Best PPA, slowest to design (e.g. CPU cores).



Standard-cell

Pre-designed cell library auto placed & routed. The mainstream ASIC flow.



Gate array / SoG

Pre-fabricated transistors, customise only wiring. Faster turnaround.



FPGA

Re-programmable logic — no fabrication. Best for prototyping (Module 4).

ASIC vs FPGA at a glance

Aspect	ASIC	FPGA
Unit cost (high vol.)	Low	High
Upfront/NRE cost	Very high	Low
Time-to-market	Long	Short
Re-programmable	No	Yes



Rule of thumb

Prototyping or low volume → **FPGA**.
Millions of units, tight power → **ASIC**.

CMOS Transistor Theory

A deep dive into the MOSFET — the switch behind every CMOS gate: how it works, how we model it, and how it has scaled for fifty years.

a

Fundamentals of MOSFET operation

Channel formation, threshold, NMOS & PMOS, on/off behaviour

b

MOSFET characteristics & modeling

I-V regions, the square-law model, capacitances, non-idealities

c

Scaling trends & technology advancements

Moore & Dennard scaling, nodes, FinFET and gate-all-around





Semiconductor & MOS Basics

Silicon is a semiconductor — conductivity between metal and insulator.

Doping adds impurities: **n-type** (extra electrons) or **p-type** (extra holes).

A **pn-junction** is the basic building block; a MOSFET uses doped source/drain regions in a body.

- MOS = Metal–Oxide–Semiconductor: a gate over a thin oxide controls a channel.



Threshold voltage V_{th}

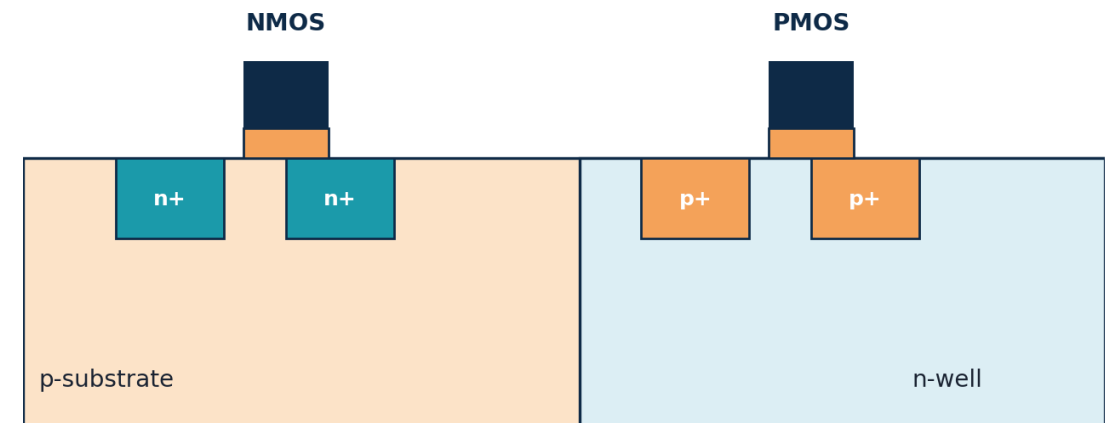
The gate voltage at which a conducting channel just forms.

For NMOS: channel of **electrons** forms when $V_{GS} > V_{th}$ (positive).

For PMOS: channel of **holes** forms when $V_{GS} < V_{tp}$ (negative).

Typical $|V_{th}| \approx 0.3\text{--}0.7\text{ V}$ in modern processes.

CMOS inverter — fabricated cross-section



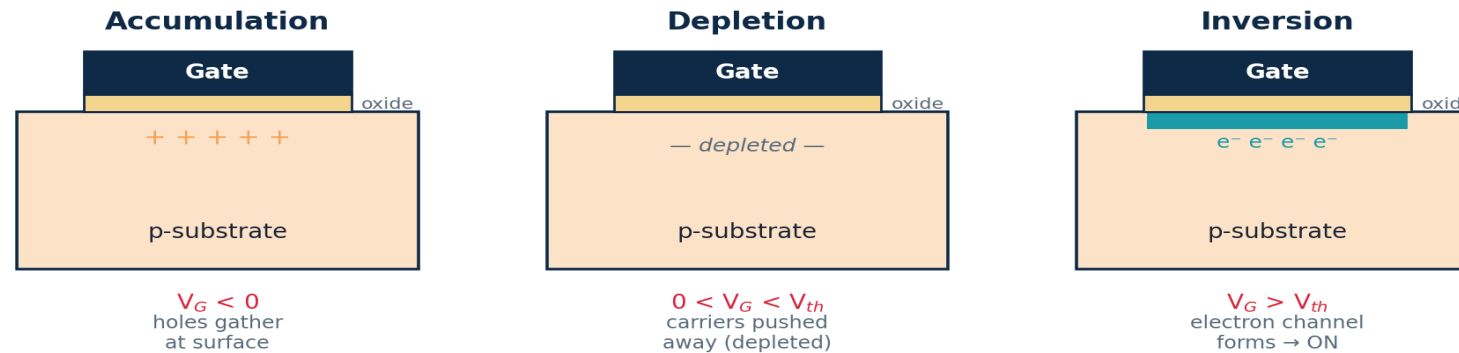
A CMOS process places NMOS in the p-substrate and PMOS in an n-well, side by side.



The MOS Capacitor & Channel Formation

Before current can flow, the gate must create a conducting channel. Watch what happens at the silicon surface as the gate voltage on an NMOS (p-substrate) rises.

MOS capacitor: how the channel forms as gate voltage rises



Threshold voltage V_{th}

V_{th} is the gate voltage at which inversion begins.

Set by oxide thickness, doping, and the gate material.

Typical $|V_{th}| \approx 0.3\text{--}0.7\text{ V}$.

- Accumulation:** $V_G < 0$: majority holes pile up at the surface — no channel.
- Depletion:** $0 < V_G < V_{th}$: holes are pushed away, leaving a depleted region.
- Inversion:** $V_G > V_{th}$: electrons invert the surface → a conducting n-channel forms.



NMOS Structure & Operation

Four terminals: Gate, Source, Drain, Body.

Built in the **p-substrate** with n+ source/drain.

$V_{GS} > V_{th} \rightarrow$ electron channel forms $\rightarrow$ **ON**.

Passes a **strong 0**, a weak 1 $\rightarrow$ used to **pull down**.



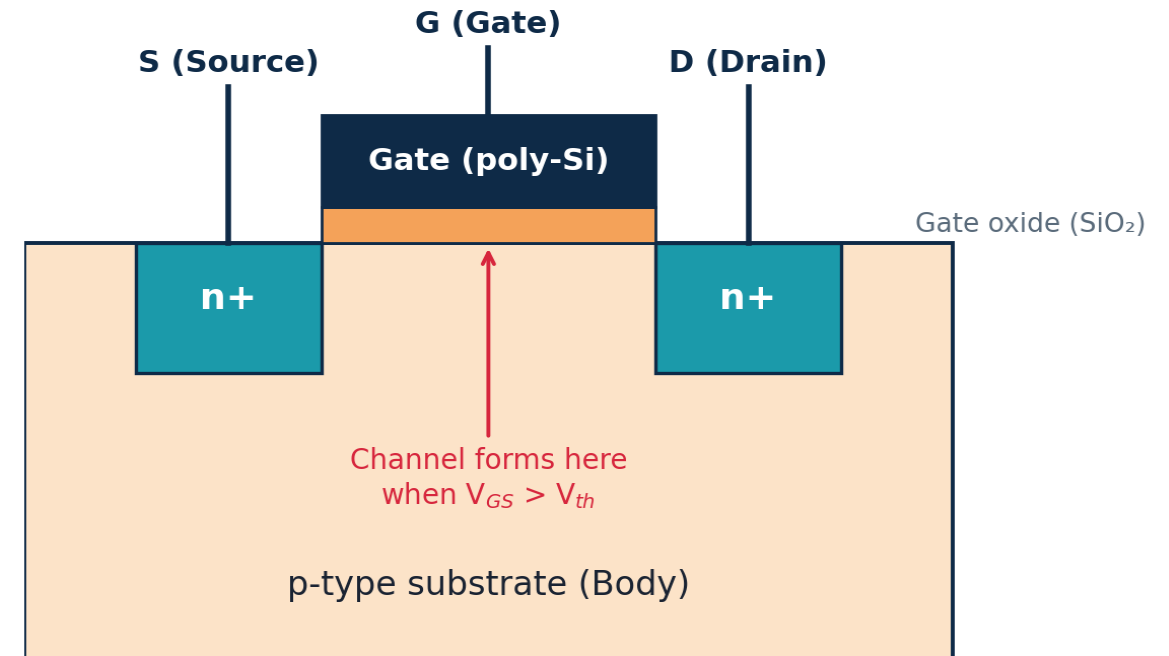
Example — is the NMOS on?

Given $V_{th} = 0.5\text{ V}$.

(a) $V_G = 1.8\text{ V}$, $V_S = 0 \rightarrow V_{GS} = 1.8\text{ V} > 0.5 \rightarrow$ **ON**.

(b) $V_G = 0.3\text{ V}$, $V_S = 0 \rightarrow V_{GS} = 0.3\text{ V} < 0.5 \rightarrow$ **OFF**.

(c) As a pass device $V_S = 1.3\text{ V}$, $V_G = 1.8\text{ V} \rightarrow V_{GS} = 0.5\text{ V} = V_{th} \rightarrow$ barely on (weak 1).



NMOS — a switch closed by a HIGH gate



PMOS Structure & Operation

Built inside an **n-well** with p+ source/drain.

$V_{GS} < V_{tp}$ (negative) $\rightarrow$ hole channel $\rightarrow$ **ON**.

Passes a **strong 1**, a weak 0 $\rightarrow$ used to **pull up**.

- *PMOS is the electrical complement of NMOS.*



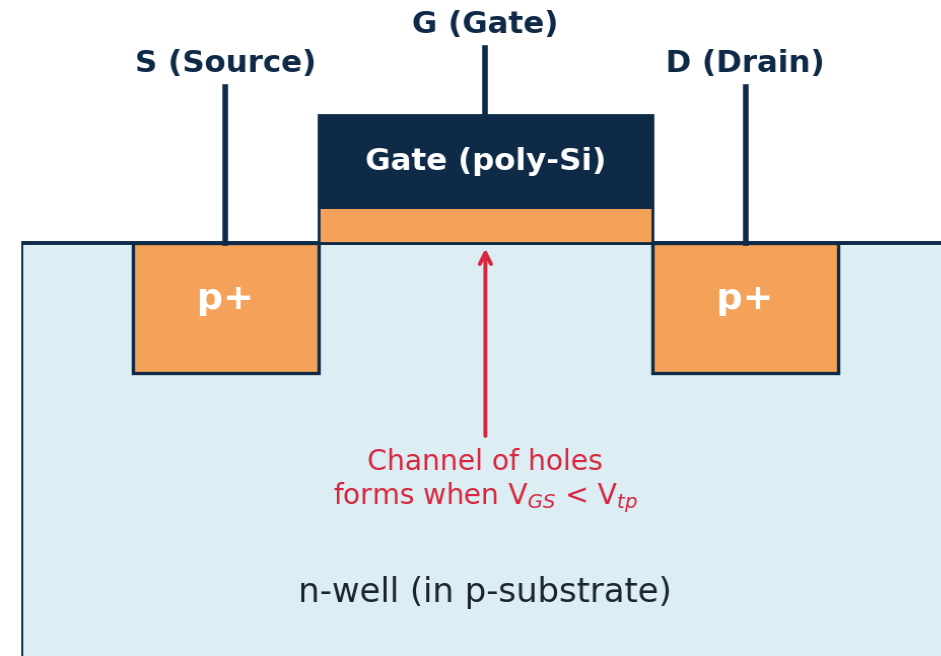
Example — is the PMOS on?

Given $V_{tp} = -0.5$ V, source tied to $V_{DD} = 1.8$ V.

(a) $V_G = 0$ V $\rightarrow V_{GS} = 0 - 1.8 = -1.8$ V < -0.5 $\rightarrow$ **ON**.

(b) $V_G = 1.8$ V $\rightarrow V_{GS} = 0$ V > -0.5 $\rightarrow$ **OFF**.

So a LOW input turns the pull-up PMOS on — exactly what an inverter needs.



PMOS — a switch closed by a LOW gate



MOSFET I-V & Operating Regions

Cutoff

$$V_{GS} < V_{th}$$

$I_D \approx 0$. Switch OFF.

Triode / Linear

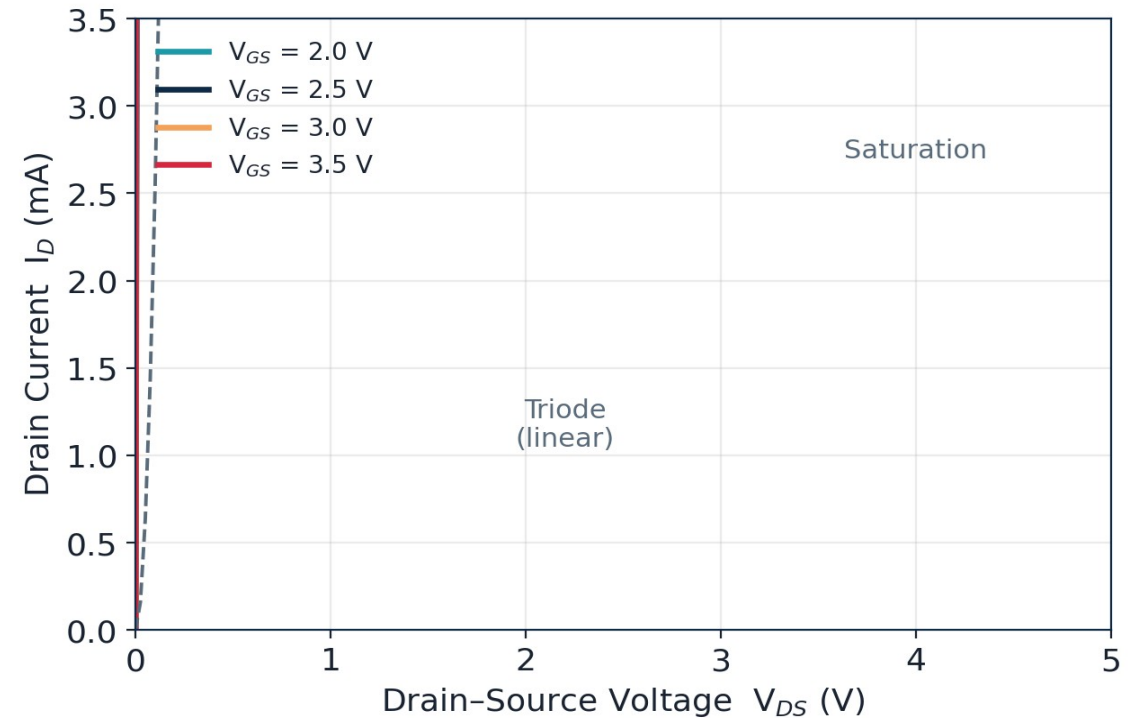
$$V_{DS} < V_{GS} - V_{th}$$

$I_D = k[(V_{GS} - V_{th})V_{DS} - V_{DS}^2/2]$. Resistor-like.

Saturation

$$V_{DS} \geq V_{GS} - V_{th}$$

$I_D = \frac{1}{2}k(V_{GS} - V_{th})^2$. ~Constant current.



Quick check

$V_{th} = 0.4$, $V_{GS} = 1.2$, $V_{DS} = 0.3$: overdrive = 0.8; $V_{DS} (0.3) < 0.8 \rightarrow$ **triode**.



The Square-Law (Shockley) Model

The first-order MOSFET model gives one equation per region. It is the workhorse for hand analysis and for understanding behaviour.

Cutoff ($V_{GS} < V_{th}$)

$$I_D = 0$$

The switch is open — only tiny leakage flows.

Triode / Linear ($V_{DS} < V_{GS} - V_{th}$)

$$I_D = k \cdot [(V_{GS} - V_{th}) \cdot V_{DS} - V_{DS}^2 / 2]$$

Channel acts like a voltage-controlled resistor.

Saturation ($V_{DS} \geq V_{GS} - V_{th}$)

$$I_D = \frac{1}{2} \cdot k \cdot (V_{GS} - V_{th})^2 \cdot (1 + \lambda V_{DS})$$

Channel pinches off; current $\approx$ constant (λ adds slope).

The parameters

$k = \mu \cdot C_{ox} \cdot (W/L)$ — the transconductance parameter.

μ = carrier mobility (electrons > holes).

C_{ox} = oxide capacitance / area.

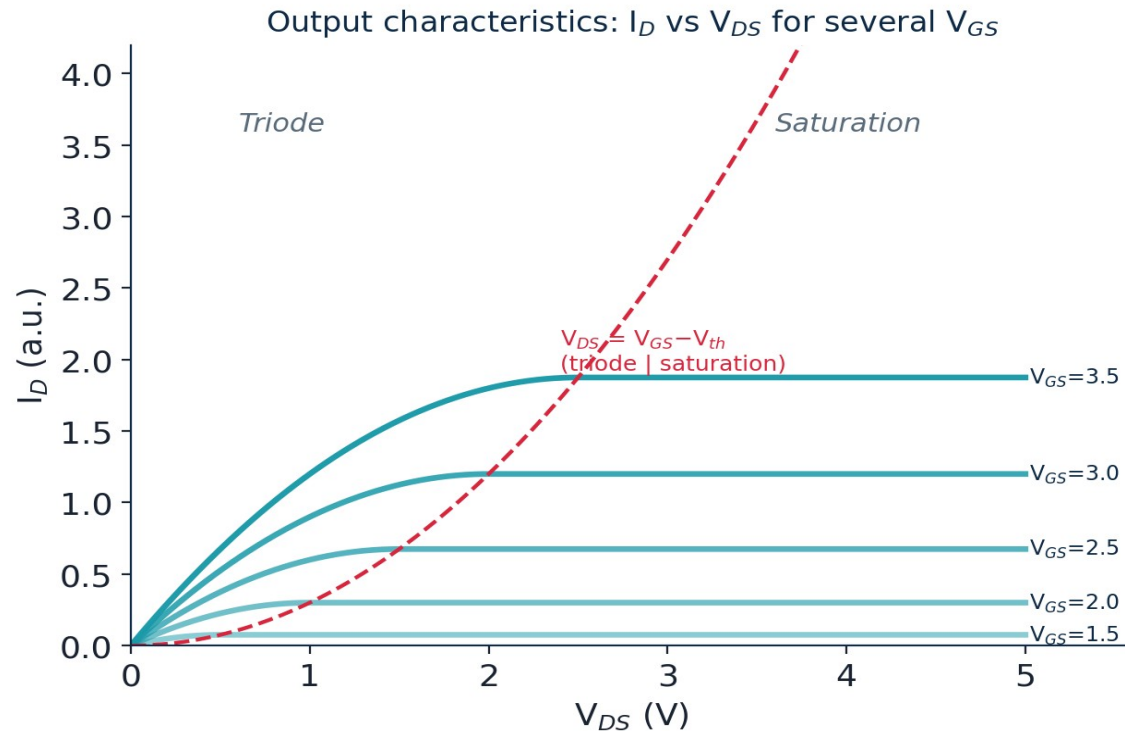
W/L = transistor aspect ratio — the designer's main knob.

λ = channel-length-modulation factor.

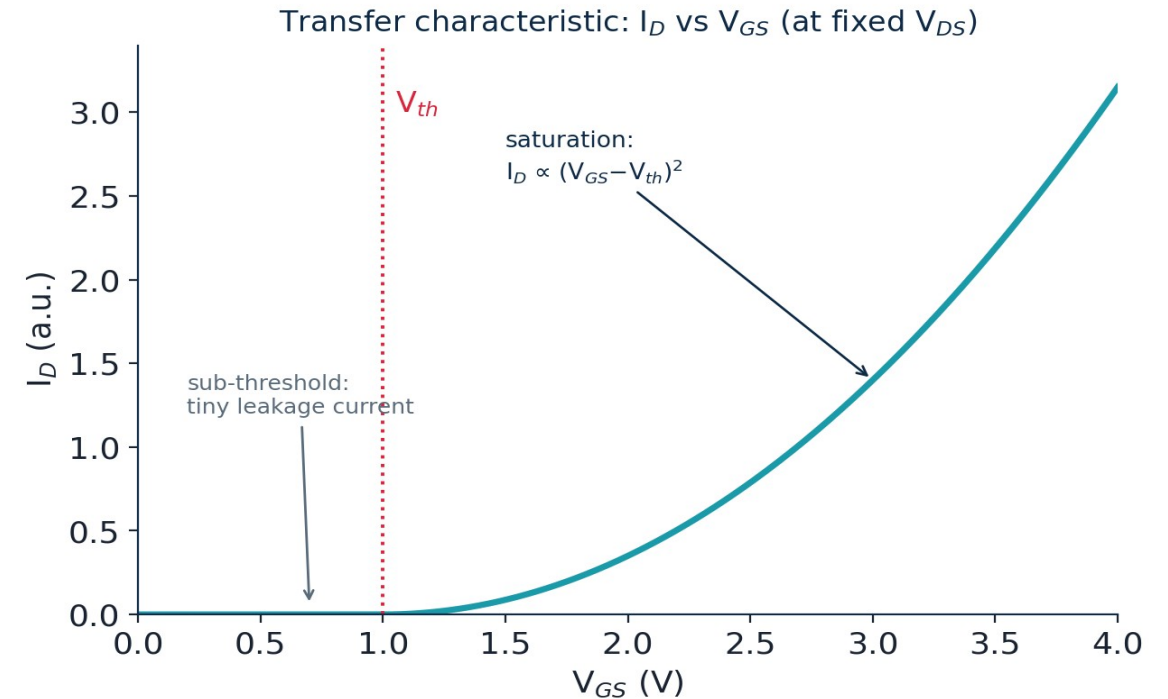


Output & Transfer Characteristics

Two standard plots summarise a transistor's behaviour. Together they let you read off the operating region and the drive current at a glance.



Output: I_D - V_{DS} for stepped V_{GS} — shows triode→saturation.



Transfer: I_D - V_{GS} — shows V_{th} and the sub-threshold tail.



Worked Examples — Drain Current

For all three: $k = 200 \mu\text{A}/\text{V}^2$, $V_{th} = 0.5 \text{ V}$. First find the region, then apply the right equation.



Example 1 — saturation

$V_{GS} = 1.5 \text{ V}$, $V_{DS} = 2.0 \text{ V}$. Overdrive = 1.0 V ; $V_{DS} \geq 1.0 \rightarrow$ saturation.

$$I_D = \frac{1}{2} \cdot k \cdot (V_{ov})^2 = \frac{1}{2} \cdot 200 \mu \cdot (1.0)^2 = \mathbf{100 \mu A}.$$



Example 2 — triode

$V_{GS} = 1.5 \text{ V}$, $V_{DS} = 0.5 \text{ V}$. Overdrive = 1.0 V ; $V_{DS} < 1.0 \rightarrow$ triode.

$$I_D = k \cdot [(V_{ov})V_{DS} - V_{DS}^2/2] = 200 \mu \cdot [0.5 - 0.125] = \mathbf{75 \mu A}.$$



Example 3 — cutoff

$V_{GS} = 0.3 \text{ V} < V_{th} (0.5)$. The device is off.

$$I_D \approx \mathbf{0 \text{ (only leakage)}}.$$



Example 4 — sizing effect

Double W/L (so $k \rightarrow 400 \mu\text{A}/\text{V}^2$) in Example 1.

$$I_D = \frac{1}{2} \cdot 400 \mu \cdot (1.0)^2 = \mathbf{200 \mu A} \text{ — current scales with } W/L.$$



MOSFET Capacitances & Speed

A transistor is not just a switch — it has **parasitic capacitances** that must be charged and discharged.

Gate caps: C_{gs} , C_{gd} , C_{gb} — between gate and the other terminals.

Junction caps: C_{jsb} , C_{jdb} — reverse-biased source/drain to body.

These set how fast a gate can switch and how much **dynamic power** it burns.

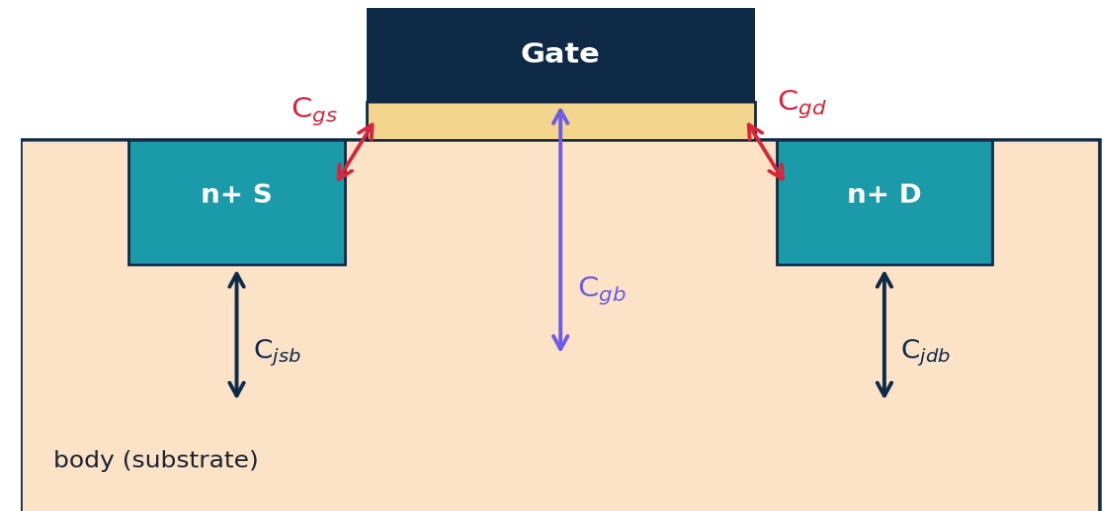


Why modeling capacitance matters

Delay $\approx 0.69 \cdot R \cdot C$ and dynamic power $\propto C \cdot V^2 \cdot f$.

Every fF of gate and wire capacitance directly slows the circuit and raises its energy — so accurate C models drive timing and power sign-off.

MOSFET parasitic capacitances (set switching speed)





Second-Order (Non-ideal) Effects

The simple square-law model is a teaching tool. Real short-channel devices show extra effects you should recognise:



Channel-length modulation

I_D rises slightly with V_{DS} in saturation (the λ term).



Body effect

V_{th} increases when source-to-body voltage is non-zero.



Velocity saturation

Carriers stop speeding up at high fields → current grows ~linearly, not squared.



Sub-threshold leakage

A small current flows even below V_{th} — a key source of static power.



Sub-threshold leakage

A small current flows even below V_{th} — a major source of static power in modern chips, and why leakage now matters as much as switching.

You won't be asked to derive these in Module 1 — just to know they exist and why the ideal equations are approximations.



The Transistor as a Switch (RC Model)

For digital design we hide the I-V curves and treat each MOSFET as an **ideal switch** with a finite ON resistance.

- NMOS: ON when gate = 1 (closes to GND).

PMOS: ON when gate = 0 (closes to V_{DD}).

ON device $\approx$ resistor R ; the load it drives $\approx$ capacitor $C \rightarrow$ an **RC switching model**.

Switch $\rightarrow$ logic mapping

Gate input	NMOS	PMOS
0 (Low)	OPEN	CLOSED
1 (High)	CLOSED	OPEN

Exactly opposite — the basis of complementary CMOS.



Example — ON resistance & delay

A gate's ON resistance $R \approx 8 \text{ k}\Omega$ drives $C \approx 30 \text{ fF}$.

Switching time constant $\tau = RC = 8\text{k} \times 30\text{f} = \mathbf{0.24 \text{ ns}}$.

Delay $\approx 0.69 \cdot RC \approx \mathbf{166 \text{ ps}}$.



Why this model is powerful

It lets us reason about whole gates and paths with simple R and C values — no differential equations.

Series switches $\rightarrow$ resistances add; parallel switches $\rightarrow$ conductances add.

This is the foundation for the delay maths in the performance topics.



Moore's Law & Dennard Scaling

Moore's Law

An **economic / empirical** observation: transistor count per chip doubles ~every 2 years.

Driven by shrinking feature sizes — it predicts **density**, not speed.

Dennard Scaling

A **physics** rule: as dimensions shrink by $1/S$, voltage scales too, so **power density stays constant**.

This is what made each node both smaller *and* faster at the same power.



The key distinction

Moore's Law says we **can** put more transistors on a chip; Dennard scaling explained why doing so used to come **for free** in power. Around 2005 Dennard scaling broke down — the start of the 'power wall'.

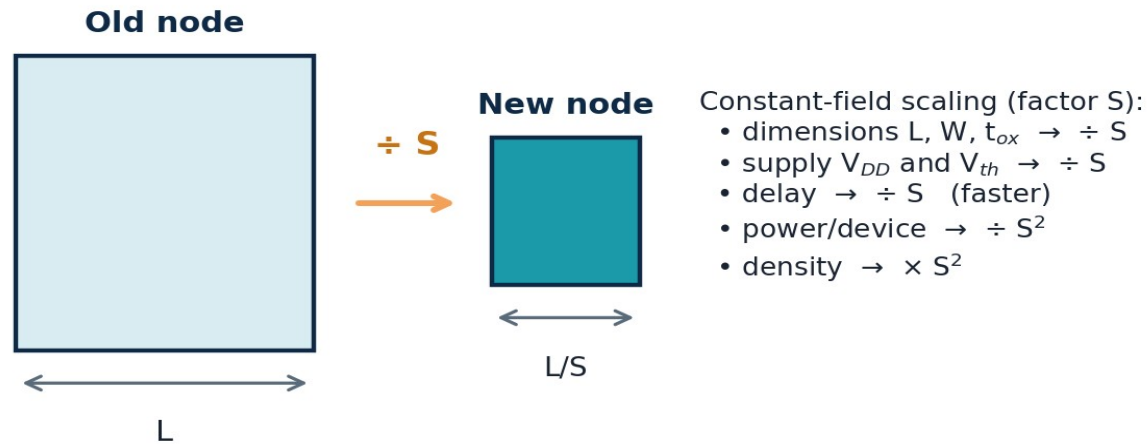
Consequence of the breakdown: clock frequencies stalled near a few GHz, and designers turned to multi-core and specialised accelerators instead of ever-faster single cores.



Constant-Field Scaling Theory

In ideal constant-field scaling, every dimension and the supply voltage shrink by the same factor S , keeping the internal electric field constant.

Dennard (constant-field) scaling



Scaling a device by factor S

How key quantities scale (factor S)

Quantity	Scales by
Length, width, oxide	$\div S$
Supply voltage V_{DD}	$\div S$
Gate delay	$\div S$ (faster)
Power per device	$\div S^2$
Power density	$\approx$ constant
Transistors / area	$\times S^2$



Worked example

$S = 2$: dimensions halve, density $\times 4$, delay halves, power/device $\div 4$ — power density unchanged.



Technology Nodes & Short-Channel Effects

The 'node' (once the gate length, now a marketing label) has shrunk from microns to a few nanometres. Shrinking brings new problems collectively called short-channel effects.



Short-channel effects to know



Velocity saturation

Current grows ~linearly, not as the square, at high fields.



DIBL

Drain voltage lowers the barrier $\rightarrow$ V_{th} drops with V_{DS} .



Increased leakage

Thin oxides & low V_{th} raise sub-threshold and gate-tunnelling current.



V_{th} variation

Random dopant fluctuation makes identical devices differ.

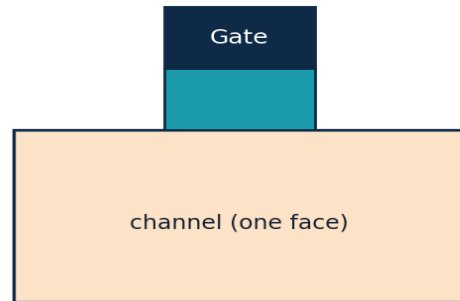


Device Evolution: Planar → FinFET → GAA

To keep scaling working once planar transistors leaked too much, the industry re-shaped the device so the gate controls the channel from more sides.

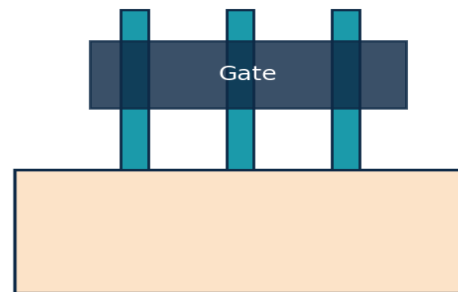
Device evolution: more gate control → better electrostatics at small sizes

Planar MOSFET



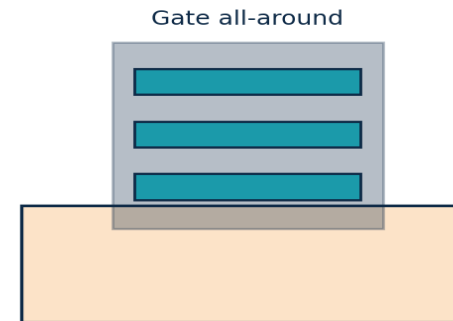
gate controls 1 side

FinFET



gate wraps 3 sides of fins

GAA / Nanosheet



gate surrounds all 4 sides

Better control →

Planar: 1 side.

FinFET: 3 sides (≈2011, 22nm).

GAA / nanosheet: all 4 sides (≈2nm).



Plus material & technique advances

Strained silicon (faster carriers), **high-k / metal gate** (less gate leakage), and **EUV lithography** (printing tiny features) all keep Moore's Law alive even as Dennard scaling has slowed.

SECTION 2

CMOS Inverter Characteristics & Logic Design

The inverter is the 'hello world' of VLSI. Master its behaviour, robustness and sizing, and every other gate follows.

Topic 1.2 • approx. 18 hours

- ✓ Inverter operation and the voltage transfer characteristic (VTC)
- ✓ Switching threshold and voltage gain
- ✓ Noise-margin analysis and power (dynamic, short-circuit, leakage)
- ✓ Inverter sizing: the β ratio and equal rise/fall
- ✓ Driving large loads: buffer chains, FO4 and optimal stages
- ✓ Static CMOS logic and NAND / NOR / AOI design

CMOS Inverter Characteristics

Everything that makes the inverter a good digital gate — its transfer curve, its robustness to noise, its power, and how we size it for speed.

a

Operation & voltage transfer characteristic

The five VTC regions, switching threshold V_M and gain

b

Noise margins & power consumption

NM_H / NM_L analysis; dynamic, short-circuit and leakage power

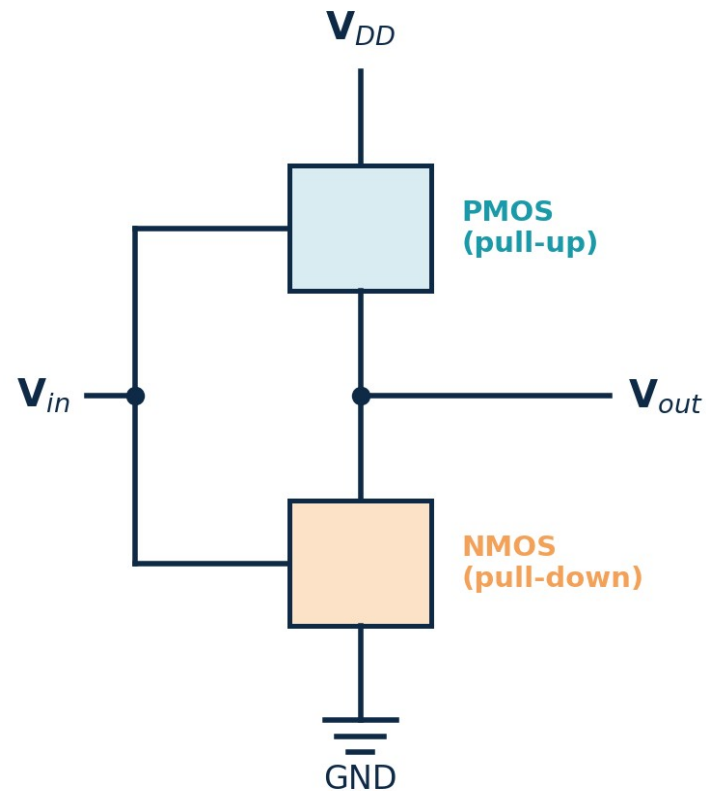
c

Sizing & optimization techniques

β ratio, equal rise/fall, buffer chains, FO4 and optimal stages



The CMOS Inverter



One PMOS (pull-up) + one NMOS (pull-down) share the input at their gates and the output at their drains, between V_{DD} and GND.

V_{in}	PMOS	NMOS	V_{out}
0 (Low)	ON	OFF	1 (High)
1 (High)	OFF	ON	0 (Low)



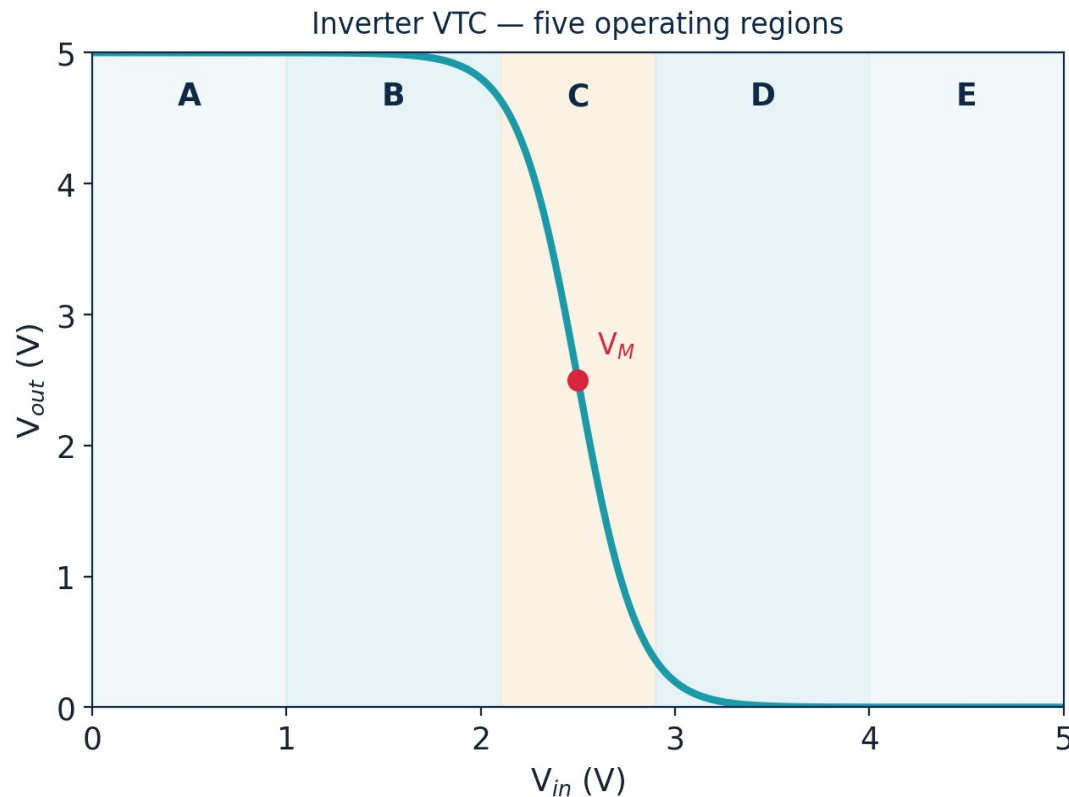
Trace it — $V_{in} = 0$

PMOS gate sees 0 → ON; NMOS gate sees 0 → OFF. Output connects to V_{DD} → logic 1. Complement confirmed.

Key property: exactly one network conducts at a time, so there is no steady DC path from V_{DD} to GND → near-zero static power. This is why CMOS won.



Voltage Transfer Characteristic (VTC)



The VTC plots V_{out} vs V_{in} and passes through five regions (A–E).

Region A: NMOS off, output = V_{DD} .

Region C: both transistors saturated — the steep, high-gain transition.

Region E: PMOS off, output = 0.

- A steeper transition ($|gain| \gg 1$) = a more 'ideal' digital gate.



Switching threshold V_M

V_M is where $V_{in} = V_{out}$. For a balanced inverter $V_M \approx V_{DD}/2$.
Sizing PMOS wider raises V_M ; sizing NMOS wider lowers it.



Switching Threshold & Voltage Gain

The switching threshold V_M is the input at which $V_{out} = V_{in}$ (both devices saturated).

It is set by the **drive strengths**: V_M rises if the PMOS is made relatively stronger (wider).

Voltage gain = the slope dV_{out}/dV_{in} near V_M — a good inverter has $|gain| \gg 1$.

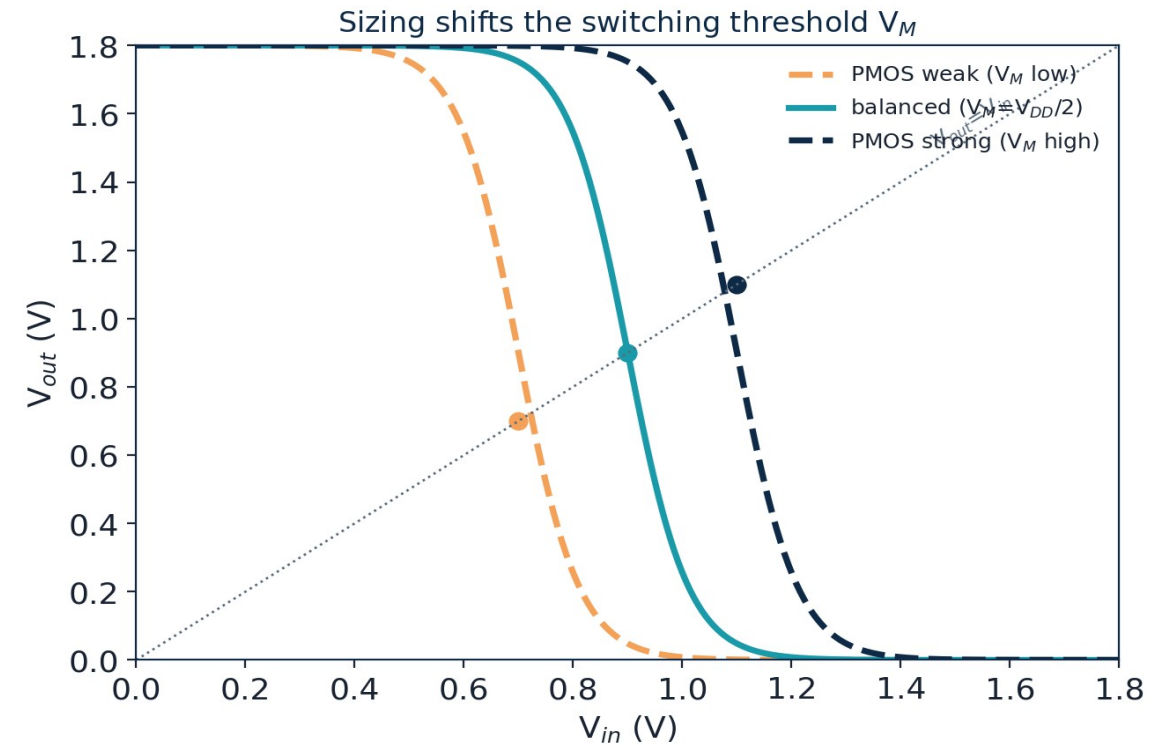
High gain **regenerates** weak logic levels back to full $V_{DD}/0$ — essential for cascading gates.



Why a centred V_M is good

$V_M \approx V_{DD}/2$ makes the high and low noise margins roughly equal → the most robust, balanced gate.

This is the target the β ratio (Topic 3c) is chosen to hit.





Noise Margins

Noise margins measure how much unwanted voltage a signal can absorb and still be read correctly.

$$NM_H = V_{OH} - V_{IH} \text{ (high-side margin)}$$

$$NM_L = V_{IL} - V_{OL} \text{ (low-side margin)}$$

V_{IH} , V_{IL} are the VTC points where the slope = -1.

The four key voltages

Symbol	Meaning
VOH	Output high level
VOL	Output low level
VIH	Min input read as 1
VIL	Max input read as 0



Worked example — compute margins

Given $V_{OH}=1.8$, $V_{OL}=0.0$, $V_{IH}=1.1$, $V_{IL}=0.7$ V.

$$NM_H = 1.8 - 1.1 = \mathbf{0.7 \text{ V.}}$$

$$NM_L = 0.7 - 0.0 = \mathbf{0.7 \text{ V.}}$$

Equal, healthy margins → robust, well-balanced gate.



Why it matters

Crosstalk, supply droop and IR drop all inject noise.

Larger NM → fewer functional failures, better yield.

Margins shrink as V_{DD} scales down — a real design challenge.



Noise-Margin Analysis on the VTC

V_{IL} and V_{IH} are the two points where the VTC slope = -1 (unity gain).

Below V_{IL} the input is safely a 0; above V_{IH} safely a 1.

The margins are the gaps to the output levels: $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.

A steeper VTC pushes V_{IL} and V_{IH} toward $V_M \rightarrow$ larger margins.

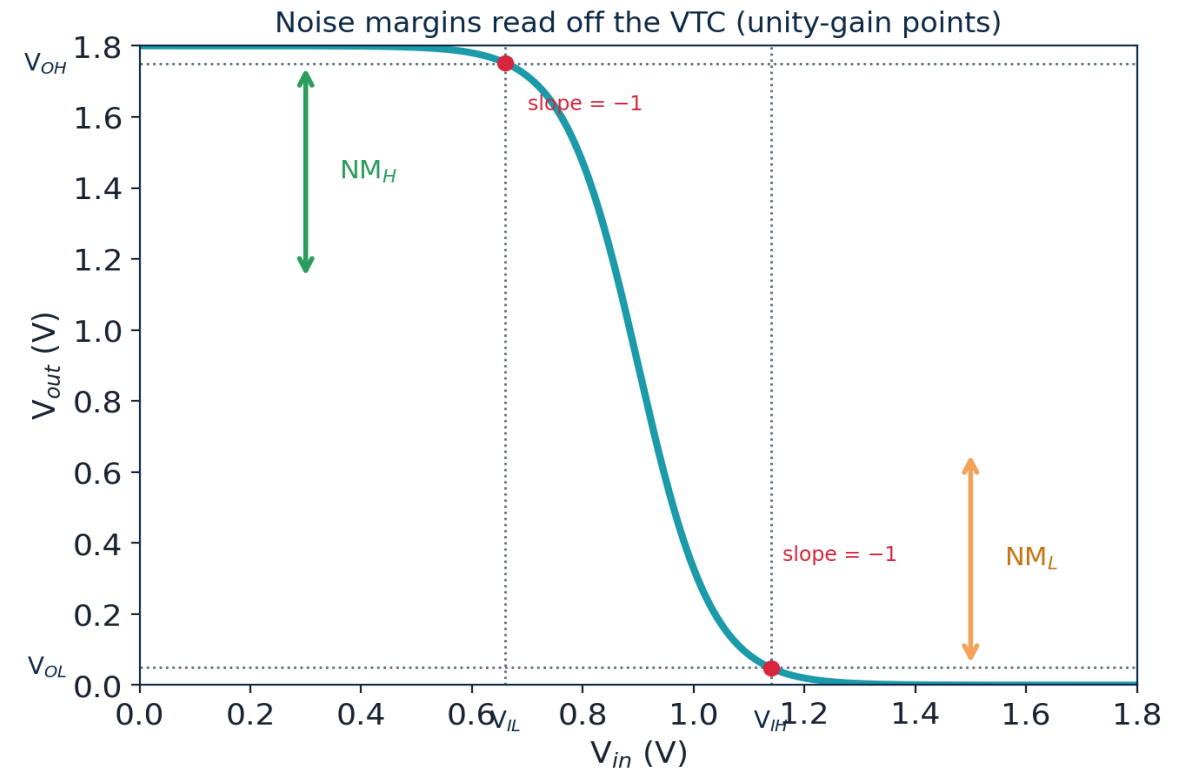


Worked example (3.3 V logic)

$V_{OH} = 3.3$, $V_{OL} = 0.1$, $V_{IH} = 2.0$, $V_{IL} = 1.1$ V.

$NM_H = 3.3 - 2.0 = 1.3$ V; $NM_L = 1.1 - 0.1 = 1.0$ V.

Both positive and sizeable $\rightarrow$ a robust gate.





Switching Delay of the Inverter

Propagation delay t_p : input-50% to output-50%. Average of t_{pHL} and t_{pLH} .

Rise/fall times t_r , t_f : 10%–90% output transition.

RC model: $t_p \approx 0.69 \cdot R_{on} \cdot C_L$.

- Lower R (wider transistors) or lower C (lighter load) → faster.



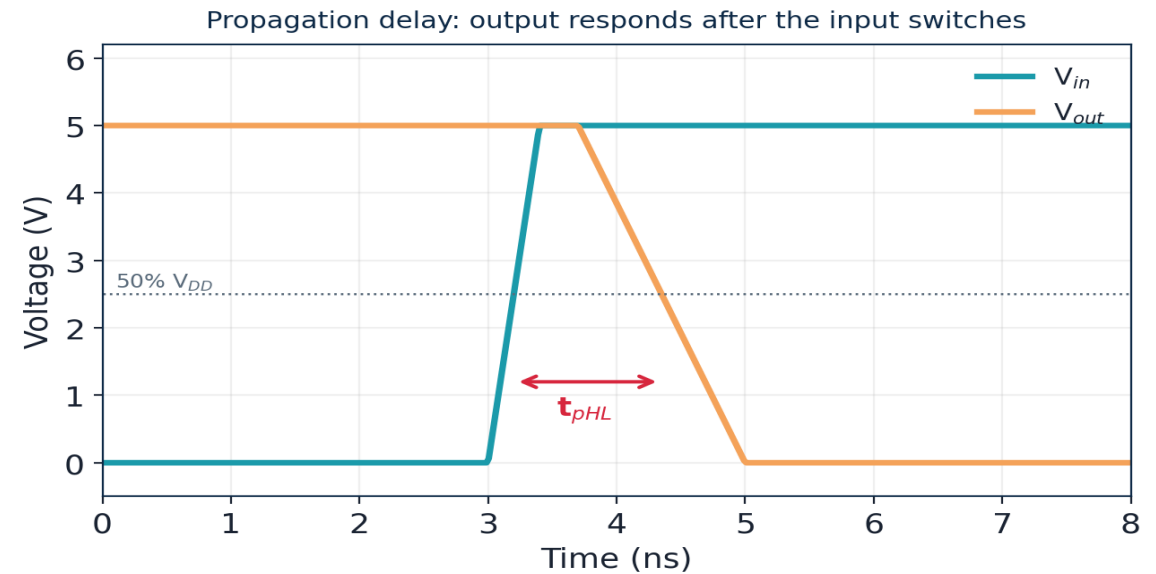
Worked example — inverter delay

$R_{on} = 12 \text{ k}\Omega$, $C_L = 25 \text{ fF}$.

$RC = 12 \times 10^3 \times 25 \times 10^{-15} = 3.0 \times 10^{-10} \text{ s}$.

$t_p = 0.69 \times 3.0 \times 10^{-10} = \mathbf{207 \text{ ps}}$.

Double the load → delay doubles (414 ps): delay is linear in C.



Fan-out, wire load and input slope all add to the effective C and slow the gate down.



Power Dissipation in CMOS

Dynamic (switching): $P = \alpha \cdot C \cdot V_{DD}^2 \cdot f$. Charging/discharging load caps — usually dominant.

Short-circuit: brief V_{DD} -to-GND current while both devices conduct during a transition.

Static (leakage): sub-threshold + gate leakage; flows even when idle. Grows as devices shrink.



Worked example — dynamic power

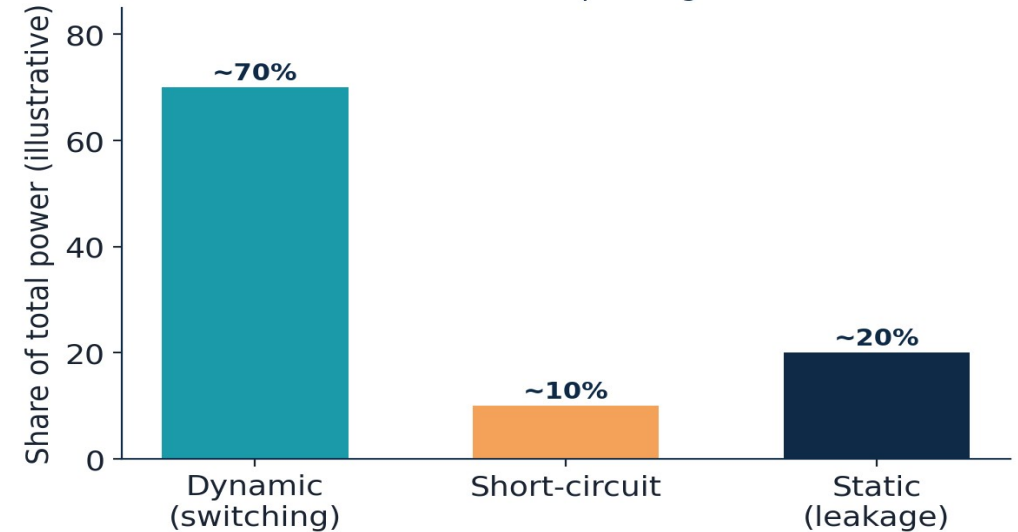
$C = 5 \text{ pF}$ total switching cap, $V_{DD} = 1.2 \text{ V}$, $f = 1 \text{ GHz}$, activity $\alpha = 0.2$.

$$P = 0.2 \times 5 \times 10^{-12} \times (1.2)^2 \times 1 \times 10^9$$

$$= 0.2 \times 5 \times 10^{-12} \times 1.44 \times 10^9 = \mathbf{1.44 \text{ mW}}$$

Halving V_{DD} cuts dynamic power $\sim 4\times$ — why voltage scaling is so valuable.

Where CMOS power goes



Energy per switch = $\frac{1}{2} \cdot C \cdot V_{DD}^2$ — independent of frequency.

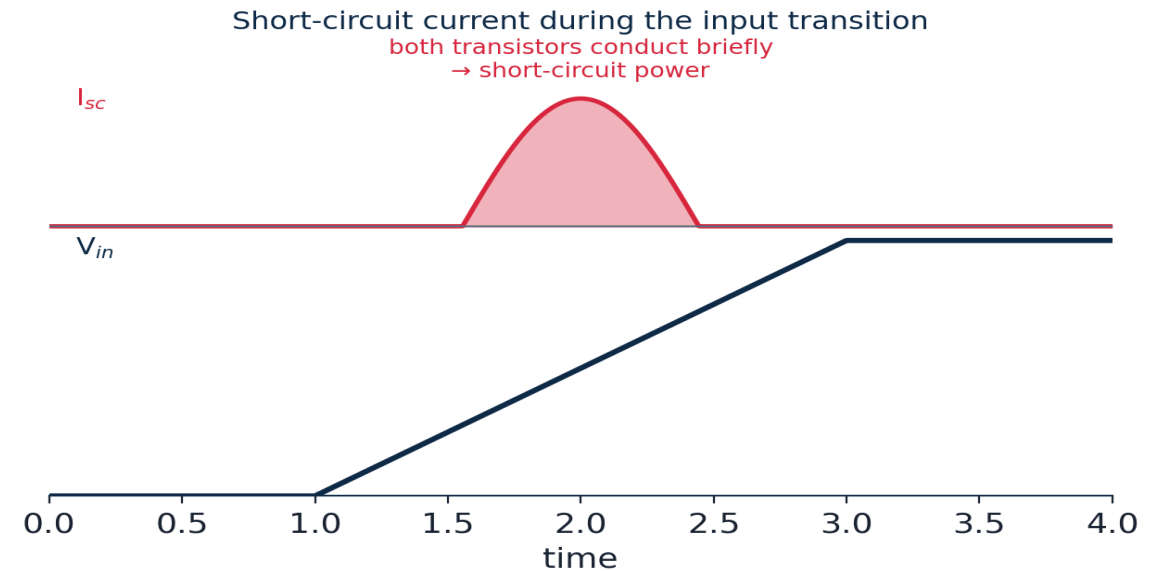


Short-Circuit, Leakage & Low-Power

Short-circuit power: during the input transition there is a brief window where $V_{in} < V_{DD} - |V_{tp}|$ and **both** transistors conduct — a current spike straight from V_{DD} to GND.

Kept small by **fast input edges** (sharp transitions spend less time in the danger window).

Leakage (static) power: sub-threshold + gate-oxide tunnelling; now a major fraction at advanced nodes.



Low-power design techniques

Voltage scaling (biggest lever, V^2 term) • **clock gating** (cut α) • **power gating** (cut leakage when idle) • multi- V_{th} **cells** (fast where needed, low-leak elsewhere) • **smaller sizing** (less C).



Inverter Sizing: the β Ratio

Electrons move $\sim 2\text{--}3\times$ faster than holes, so an equal-width PMOS is weaker than the NMOS. We size the PMOS wider to balance the inverter.

For a symmetric VTC ($V_M = V_{DD}/2$)

Size so the pull-up and pull-down strengths match: $W_p / W_n \approx \mu_n / \mu_p \approx 2\text{--}3$.

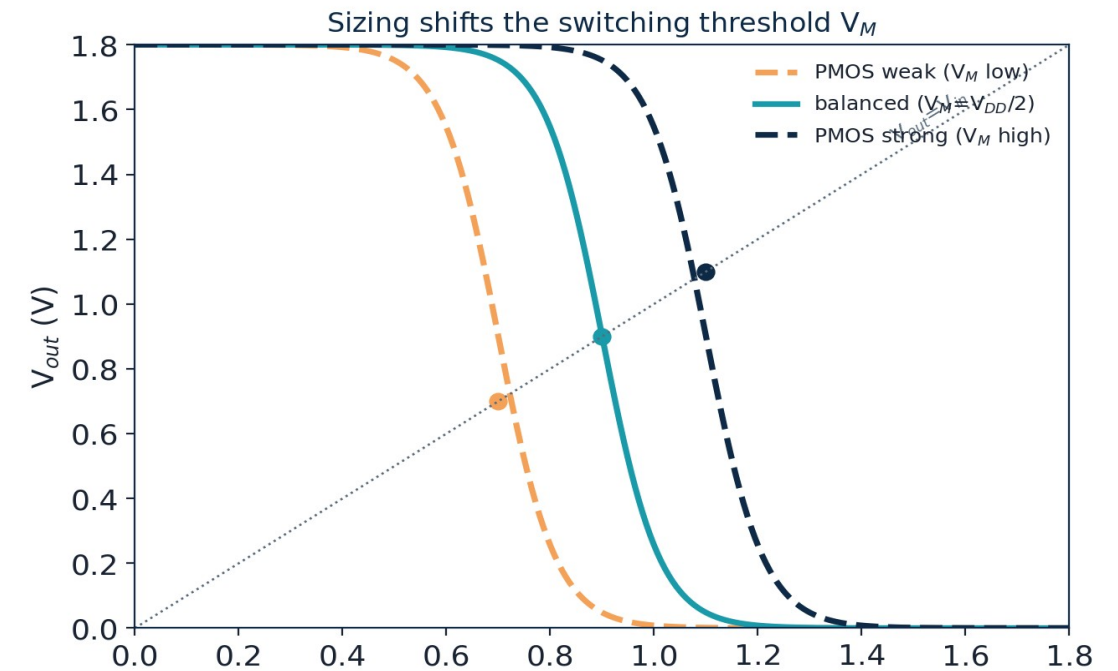
Effect of the ratio ($\beta = \beta_n / \beta_p$)

Wider PMOS $\rightarrow$ higher V_M , stronger pull-up, faster rise; costs area & input capacitance.



Trade-off

A ratio of ~ 2 balances rise and fall and centres the VTC; using 1:1 saves area but skews V_M low and makes the rise slower than the fall.





Sizing for Delay & Equal Rise/Fall

Delay $\approx 0.69 \cdot R \cdot C$. Making a transistor **wider** lowers its R (faster) but raises the C it presents to the previous stage.

Equal rise/fall: choose $W_p \approx 2 \cdot W_n$ so $t_{pLH} \approx t_{pHL}$.

Up-sizing a gate only helps if it drives a **large external load**; for self-load it just moves the problem upstream.

The right size depends on the **fan-out** (load $\div$ input cap) the gate must drive.



Worked example — equal rise/fall

NMOS $R_n = 10 \text{ k}\Omega$; PMOS of equal width $R_p = 20 \text{ k}\Omega$ (holes slower).

Double the PMOS width $\rightarrow R_p$ halves to $10 \text{ k}\Omega \rightarrow$ rise and fall delays now match.

Rule of thumb



Need balanced VTC

$W_p/W_n \approx 2-3$



Need min area

$W_p/W_n = 1$ (skewed)



Driving big load

Up-size whole gate / add buffers



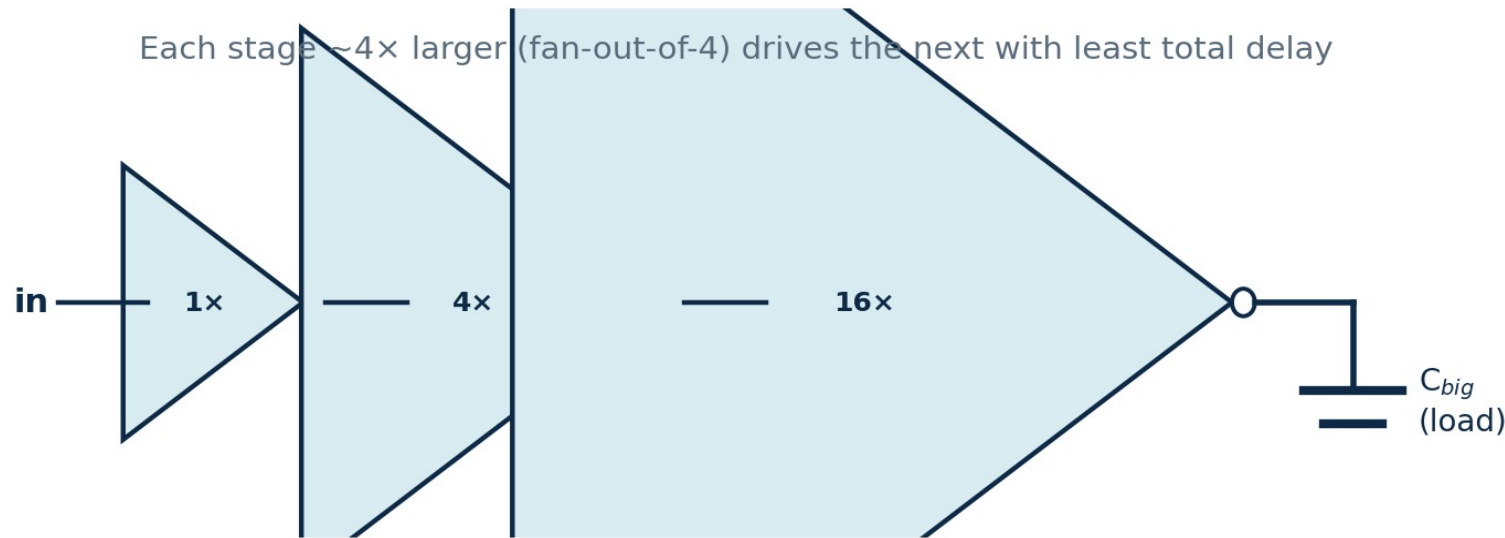
Speed-critical path

Use FO4-sized stages



Driving Large Loads: Buffer Chains & FO4

A tiny gate cannot quickly drive a big load (a long wire or many inputs). The fix is a chain of inverters that grows geometrically in size.



Why taper?

One **big gate** would overload the source.

Too many **tiny stages** add overhead.

A **geometric taper** shares the effort evenly.

Fan-out-of-4 (FO4): each stage about $4\times$ the previous one is close to the minimum-delay taper — a standard rule of thumb and a common delay unit.



Choosing the Optimal Number of Stages

For a fixed total job (path effort F), there is a **sweet spot** in the number of stages N .

- Too few stages → each must do too much work (huge fan-out) → slow.
- Too many stages → the fixed per-stage overhead (parasitic delay) dominates → slow.

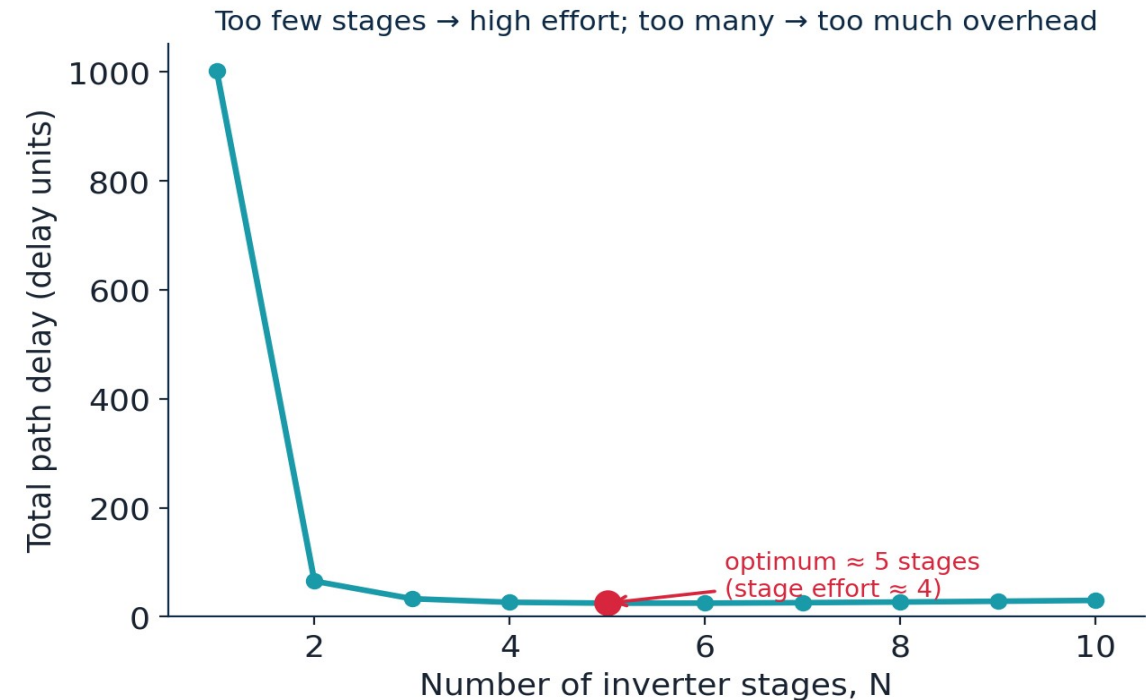
Minimum delay is near a **per-stage effort of ≈ 4** (the mathematical optimum is $e \approx 2.7$, but ~ 4 is used in practice).



Worked example

Drive a load $64\times$ the input cap ($F = 64$).

Use stage effort $\approx 4 \rightarrow N = \log_4(64) = 3$ stages sized $1\times \rightarrow 4\times \rightarrow 16\times$ (driving $64\times$).



CMOS Logic Design

From the four basic gates to any combinational function — built systematically from complementary MOS networks.

a

Basic logic gates: AND, OR, NAND, NOR

Symbols, truth tables, CMOS realisation and universality

b

Combinational logic design in CMOS

The PUN/PDN method and a systematic design procedure

c

Designing complex logic functions

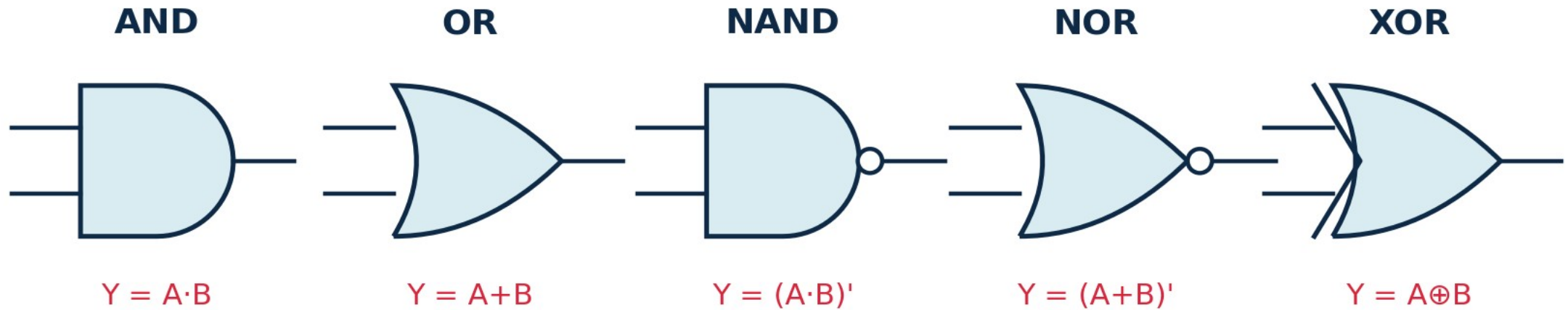
AOI / OAI gates, factoring, and transmission-gate logic





Basic Logic Gates

Every digital function is built from a small set of gates. Here are the workhorses, their Boolean expressions and truth tables.



A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0



In CMOS

NAND & NOR are the **natural** CMOS gates; AND/OR add an inverter.



Designing a 2-input NAND

$$Y = (A \cdot B)'$$

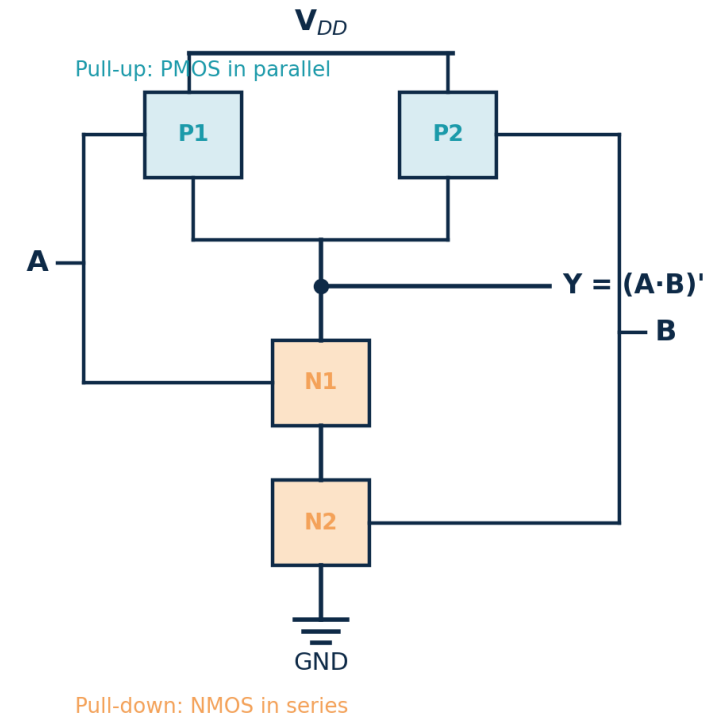
- 1 Output is 0 only when A AND B = 1. (*AND condition.*)
- 2 PDN: two NMOS in SERIES to GND. (*series = AND.*)
- 3 PUN (dual): two PMOS in PARALLEL to VDD. (*parallel.*)
- 4 Total = 4 transistors. Verify all 4 rows.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



Generalise it

An n-input NAND needs **2n transistors** (n NMOS in series + n PMOS in parallel).
3-input NAND → 6 transistors.





Designing a 2-input NOR

$$Y = (A + B)'$$

- 1 Output is 0 when A OR B = 1. (*OR condition.*)
- 2 PDN: two NMOS in PARALLEL to GND. (*parallel = OR.*)
- 3 PUN (dual): two PMOS in SERIES to VDD. (*series.*)
- 4 Total = 4 transistors. Verify all 4 rows.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

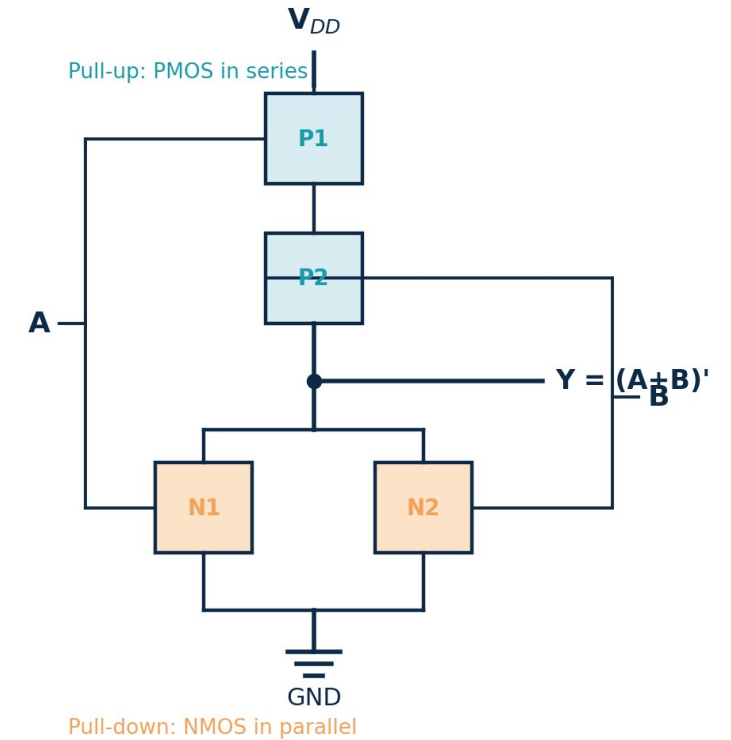


NAND vs NOR pattern

NAND → NMOS series, PMOS parallel.

NOR → NMOS parallel, PMOS series.

In practice NAND is preferred (PMOS in series is slow).



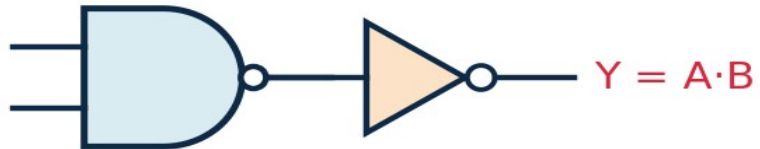


Building AND and OR in CMOS

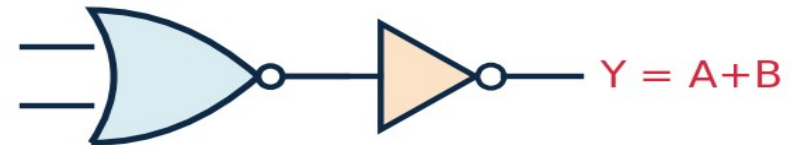
Static CMOS gates are naturally inverting (the output network always inverts). A plain AND or OR therefore needs an extra inverter on the output of a NAND or NOR.

Static CMOS is naturally inverting: AND/OR need an extra inverter

AND = NAND → INVERT



OR = NOR → INVERT



Consequence: AND = 4 (NAND) + 2 (INV) = 6 transistors; a NAND alone is only 4. This is why synthesis tools favour NAND/NOR/AOI over AND/OR.



NAND & NOR are Universal

A single gate type from which ANY logic function can be built is called universal. Both NAND and NOR are universal — a powerful and practical fact.

NAND is a universal gate: NOT, AND and OR all built from NANDs only

NOT



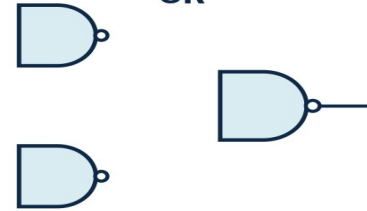
tie inputs together

AND



NAND → NAND (as INV)

OR



3 NANDs

NOT = a NAND with its inputs tied together.

AND = NAND followed by NAND-as-inverter.

OR = invert both inputs (NANDs) then NAND them (De Morgan).



Why it matters

A whole chip can be built from one repeated cell — simpler libraries and fabrication.

De Morgan's laws let you convert any AND/OR network into NAND-only or NOR-only form.

NAND is usually preferred over NOR (faster pull-up).



Static CMOS: the Universal Recipe

Every static CMOS gate is built from two complementary networks between the output and the rails.

Pull-Up Network (PUN)

PMOS only. Connects output to V_{DD} → drives a logic 1. Conducts when the inputs are LOW.

Pull-Down Network (PDN)

NMOS only. Connects output to GND → drives a logic 0. Conducts when the inputs are HIGH.

Duality: PUN and PDN are duals — devices in series in one network are in parallel in the other.

4-step design recipe

1

Write the output as $Y = (\text{expression})'$.

2

Build the PDN: AND → NMOS in series; OR → NMOS in parallel.

3

Build the PUN as the dual (swap series ↔ parallel) with PMOS.

4

Count transistors (2 per input) and verify the truth table.



Worked Example — a 2:1 Multiplexer

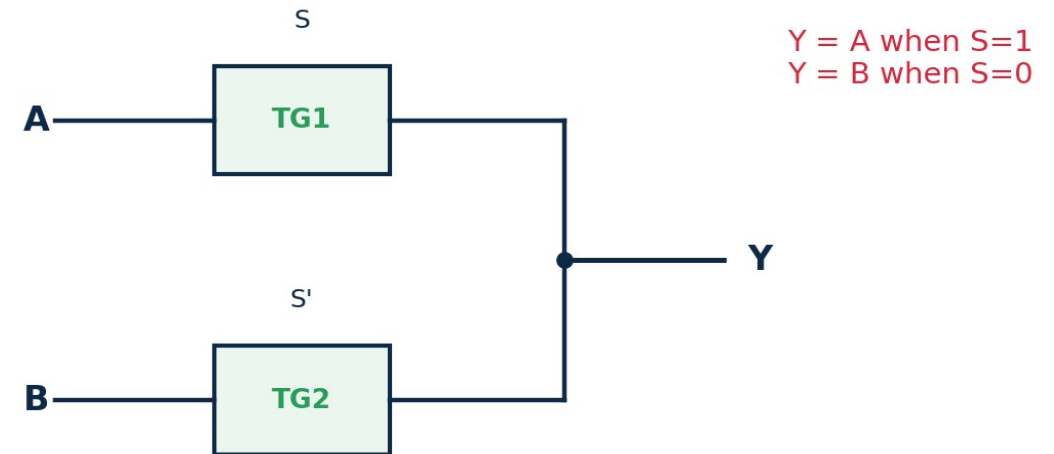
A multiplexer selects one of two inputs. It is a classic small combinational block, and shows two valid CMOS styles.

Function: $Y = S \cdot A + S' \cdot B$

S	Y
0	B
1	A

Static-CMOS style: realise $Y' = (S \cdot A + S' \cdot B)'$ as one AOI-style gate, then invert.

Transmission-gate style: two TGs select A or B — only ~6 transistors (shown).



A 2:1 multiplexer from two transmission gates + an inverter for S'



Takeaway

Most combinational blocks (adders, comparators, decoders) are assembled from these gate-level pieces.



Worked Example — a Half Adder

A half adder adds two bits and produces a sum and a carry. It shows how a small combinational block is assembled from basic gates.

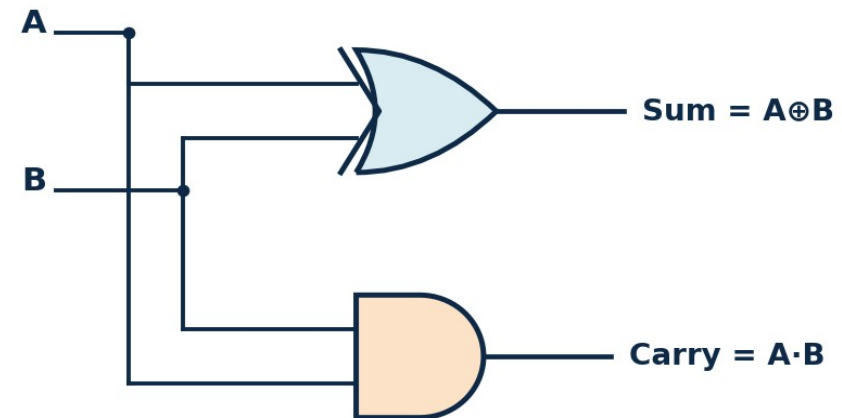
Logic: Sum = $A \oplus B$, Carry = $A \cdot B$

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum is an XOR (true when inputs differ).

Carry is an AND (true only when both are 1).

Half adder = XOR (sum) + AND (carry)



CMOS realisation

XOR $\approx$ 8–12 transistors; AND = NAND+INV. Or build the whole thing in transmission-gate logic for fewer devices.



Complex Gates — AOI

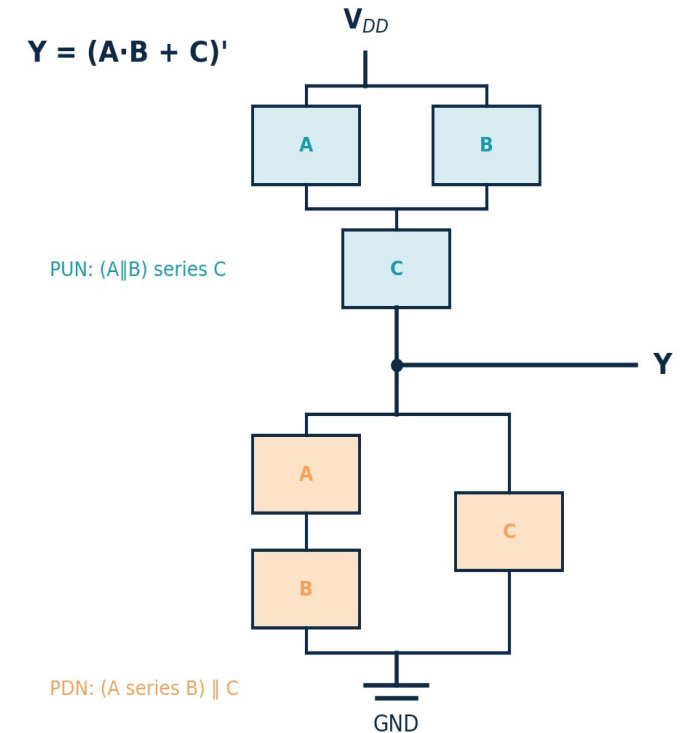
$$Y = (A \cdot B + C)' \text{ — AND-OR-Invert}$$

- 1 Inside the inversion: $(A \cdot B) + C$. (*read the expression.*)
- 2 PDN: (A series B) in PARALLEL with C. (*AND→series, OR→parallel.*)
- 3 PUN dual: $(A \parallel B)$ in SERIES with C. (*swap everything.*)
- 4 3 NMOS + 3 PMOS = 6 transistors.



Why complex gates are great

A single AOI gate implements $A \cdot B + C'$ -style logic with **fewer transistors and less delay** than wiring separate AND + OR + INV gates (which would need ~14 transistors).





Complex Gates — OAI

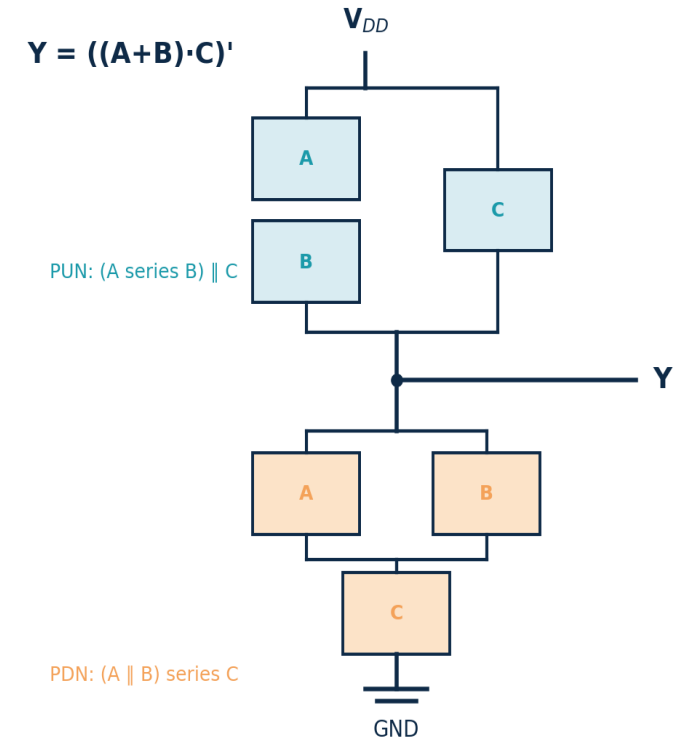
$$Y = ((A + B) \cdot C)' \text{ — OR-AND-Invert}$$

- 1 Inside the inversion: $(A+B) \cdot C$. *(read the expression.)*
- 2 PDN: $(A \parallel B)$ in SERIES with C. *(OR → parallel, AND → series.)*
- 3 PUN dual: $(A \text{ series } B)$ in PARALLEL with C. *(swap everything.)*
- 4 3 NMOS + 3 PMOS = 6 transistors.



AOI vs OAI

OAI is the **dual** of AOI: start from the OR terms in the PDN. Choose whichever matches your Boolean expression with the fewest transistors.





Mapping Any Function to a CMOS Gate

Any inverting function can become one complex static gate. The trick is to factor the Boolean expression to minimise transistors.

- 1 Express the output in inverted form $Y = (f)'$.
- 2 In the PDN map AND→series, OR→parallel of NMOS.
- 3 Build the PUN as the dual with PMOS.
- 4 Factor f to share transistors and shrink the stack.



Factoring example

$Y = (A \cdot B + A \cdot C)'$ has 6 literals.

Factor: $A \cdot B + A \cdot C = A \cdot (B + C) \rightarrow Y = (A \cdot (B + C))'$ — now only **3 NMOS in the stack** (A in series with $B \parallel C$) instead of 4.

Practical limits

Series stacks are slow — keep them to ≈ 4 transistors high (fan-in limit).

Tall PMOS stacks are worst (slow holes) — prefer AOI over OAI when you can.

Big functions are split into several gates (multi-level logic) for speed.

- *Output is always inverting; add an inverter for the true polarity if needed.*



Worked Example — a Full Adder

A full adder adds three bits (A, B and a carry-in). It is the canonical example of multi-level logic — several gates composed to realise a richer function.

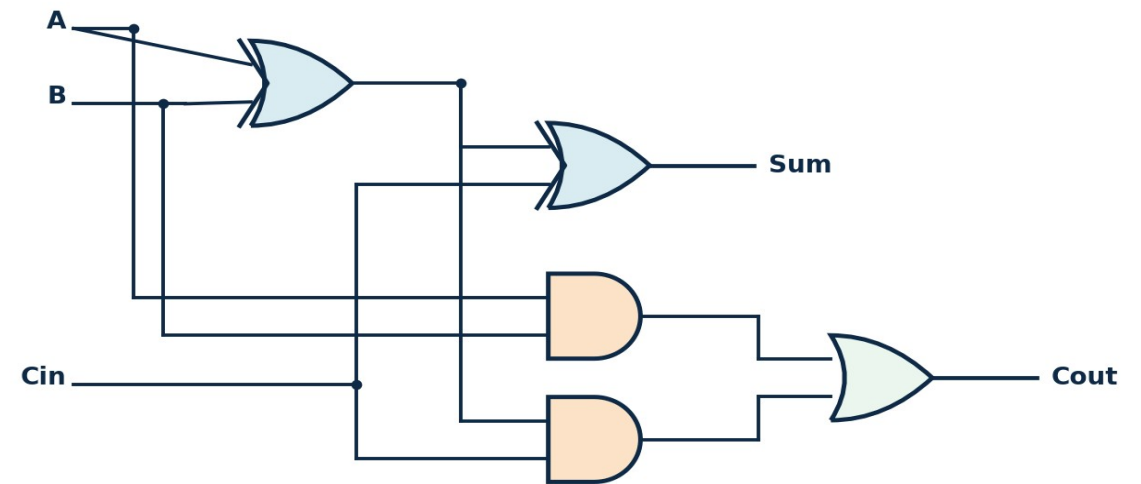
Logic: Sum = $A \oplus B \oplus \text{Cin}$

Cout = $A \cdot B + \text{Cin} \cdot (A \oplus B)$

A	B	Cin	Sum	Cout
0	0	0	0	0
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1

Built from **2 XOR + 2 AND + 1 OR** (or two half adders + an OR).

Full adder = 2 XOR + 2 AND + 1 OR



Why it matters

Chaining n full adders gives an n-bit ripple-carry adder — the heart of every ALU.



Transmission-Gate & Pass Logic

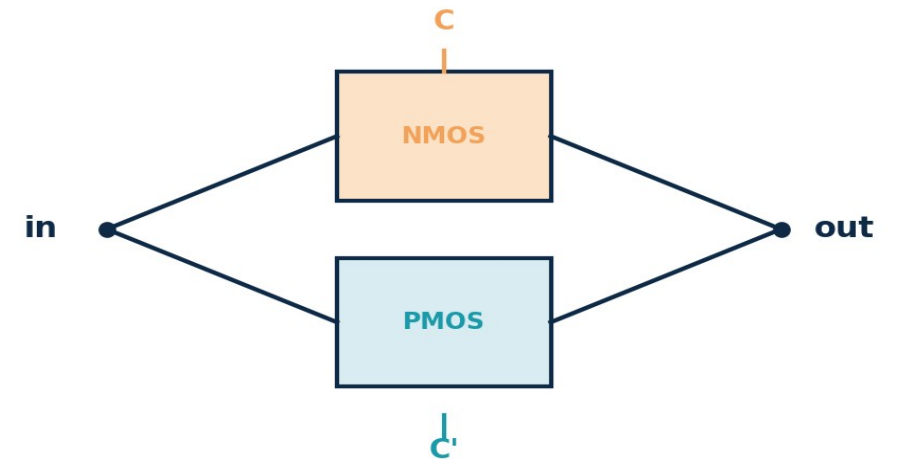
- A single NMOS passes a strong 0 but weak 1; a single PMOS the opposite. Put them in **parallel** (gated by C and C') → a **transmission gate** that passes both 0 and 1 cleanly. Used to build **multiplexers**, **latches** and **XOR** compactly. This is **pass-transistor logic** — efficient, but needs care with signal levels and drive.



Example — a 2:1 multiplexer

Two transmission gates select between inputs A and B using S and S':
Out = A when S=1, B when S=0 — just 4 transistors plus an inverter for S'.

Transmission gate (NMOS + PMOS in parallel)



Static CMOS is the default; pass-transistor logic is a specialised tool.

SECTION 3

Transistor Schematic, Layout & Wire Modelling

Turning a circuit into geometry that a foundry can actually manufacture.

Topic 1.3 • approx. 12 hours

- ✓ Transistor-level schematic design & Euler-path layout
- ✓ Schematic vs layout, stick diagrams & standard cells
- ✓ λ design rules, DRC & LVS verification
- ✓ Drawing the inverter layout; metal layers & routing
- ✓ Standard-cell strategy and full-custom layout
- ✓ Then: a dedicated topic on on-chip wire modeling

Transistor-Level Schematics & Layouts

From a transistor netlist to a manufacturable polygon layout — the physical-design half of the flow.

a

Transistor-level design techniques

Reading schematics, shared diffusion and Euler-path ordering

b

Schematic design & layout considerations

Schematic vs layout, stick diagrams, cell layout & standard cells

c

Design rules, metal layers & routing

λ -rules, DRC/LVS, the metal stack, vias and interconnect routing





Transistor-Level Schematic Design

Before any layout, the circuit is captured as a transistor schematic (and an equivalent text netlist). It fixes exactly which transistors connect to which nets.

Each **transistor** is drawn with its gate, source, drain (and body) and a W/L size.

Nets (wires) join terminals; V_{DD} and GND are the rails.

A **SPICE netlist** lists every device, e.g. M1 Y A VDD VDD pmos W=2u L=0.1u.

Simulated in **NGSpice** to check function, delay and power before layout.

Example netlist — inverter

```
* CMOS inverter
M1 Y A VDD VDD pmos W=2u L=0.1u
M2 Y A GND GND nmos W=1u L=0.1u
C1 Y GND 4f
.end
```



Schematic → layout → silicon

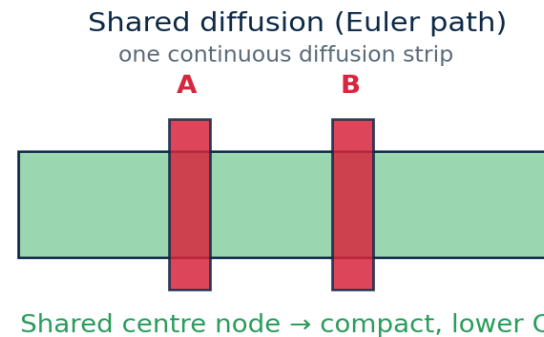
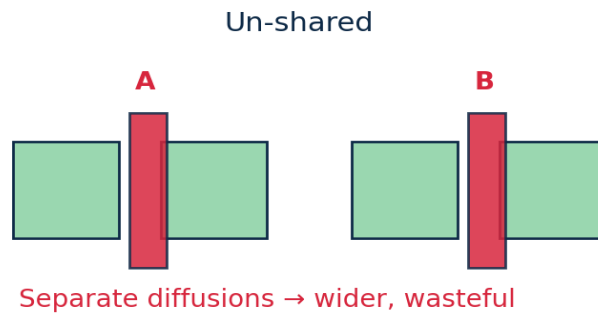
The schematic defines **connectivity & sizes**; the layout then realises it as polygons. LVS later proves the two match before tape-out.



Shared Diffusion & Euler Paths

A key transistor-level layout technique: order transistors so that adjacent ones share a source/drain diffusion. This shrinks area and capacitance.

Order transistors so source/drain regions abut — found via an Euler path



Abutting diffusions removes a contact and a gap → smaller, faster cell.

An **Euler path** through the gate network gives the transistor ordering that keeps the diffusion strip unbroken.



Why it matters

Series transistors (as in a NAND PDN) naturally share their middle node.

A good ordering means **one continuous diffusion** per row → minimum width.

Standard-cell libraries are hand-tuned this way for density.



Schematic vs Layout

Schematic

- Shows WHAT connects to what (connectivity).
- Transistors as symbols, nets as lines.
- Captures the logical & electrical design.
- Verified by simulation (NGSpice).
- Says nothing about physical size or position.

Layout

- Shows WHERE every shape sits on silicon.
- Coloured polygons on stacked mask layers.
- This is literally what gets fabricated.
- Must obey design rules (checked by DRC).
- Determines real area, parasitics and yield.



CMOS Fabrication Layers

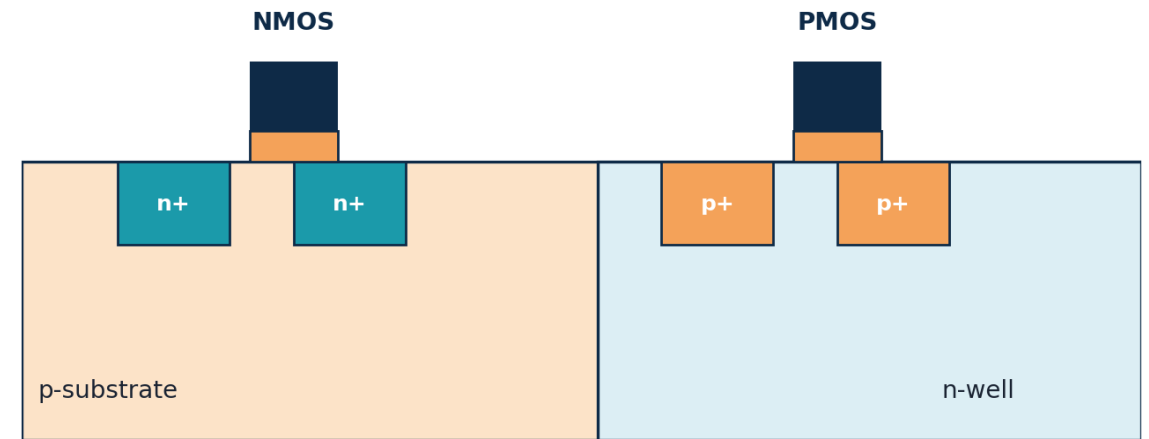
	n-well	Houses the PMOS devices
	Active / diffusion	n+ and p+ source/drain regions
	Polysilicon	Transistor gates + short wires
	Contacts / vias	Vertical connections between layers
	Metal 1...N	Horizontal wiring; many stacked layers



Each layer = one photomask

A modern process can use **40–60+ masks**. Mask cost is a big part of the high ASIC NRE we saw in Section 1.

CMOS inverter — fabricated cross-section



Layers are built up by repeated deposit → pattern (mask) → etch steps.



Stick Diagrams

A **stick diagram** is a colour-coded cartoon of the layout — lines for wires, no exact dimensions.

Colours by convention: **green** diffusion, **red** poly, **blue** metal.

Where poly crosses diffusion, a **transistor** is formed.

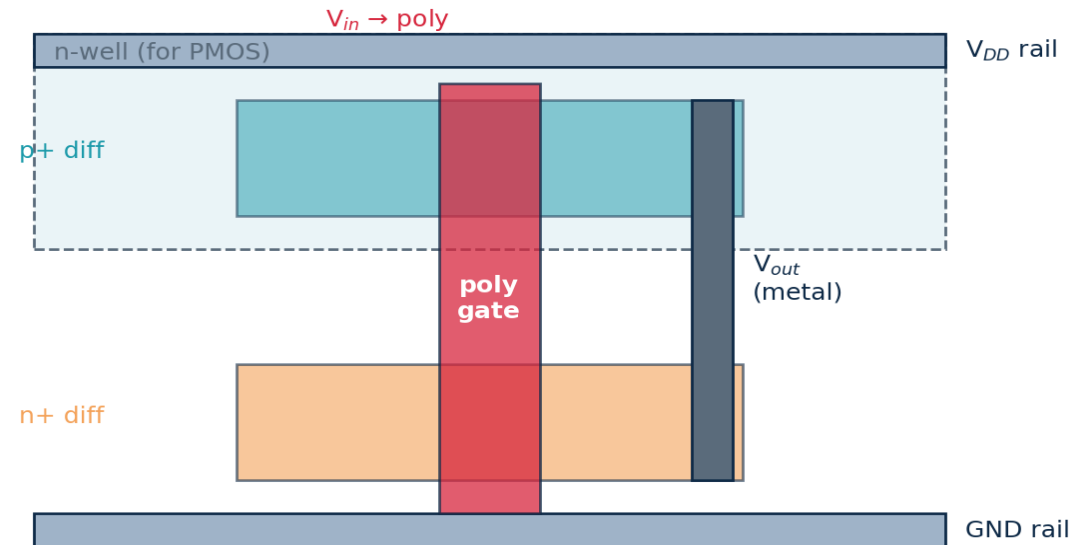
Used to plan topology & ordering *before* committing to full layout.



Example — inverter stick

One vertical poly line = the shared gate (input).
PMOS diffusion on top (in n-well, near V_{DD}), NMOS below (near GND).
A metal line joins the two drains = output. That's the whole inverter.

CMOS Inverter — simplified layout (top view)





Drawing the Inverter Layout

PMOS on top (p+ in the n-well, by V_{DD}); **NMOS below** (n+, by GND).

One vertical **poly** strip forms both gates = the input A.

Metal-1 makes the V_{DD} and GND rails and the shared output Y.

Contacts (black) tie diffusion and poly up to the metal.

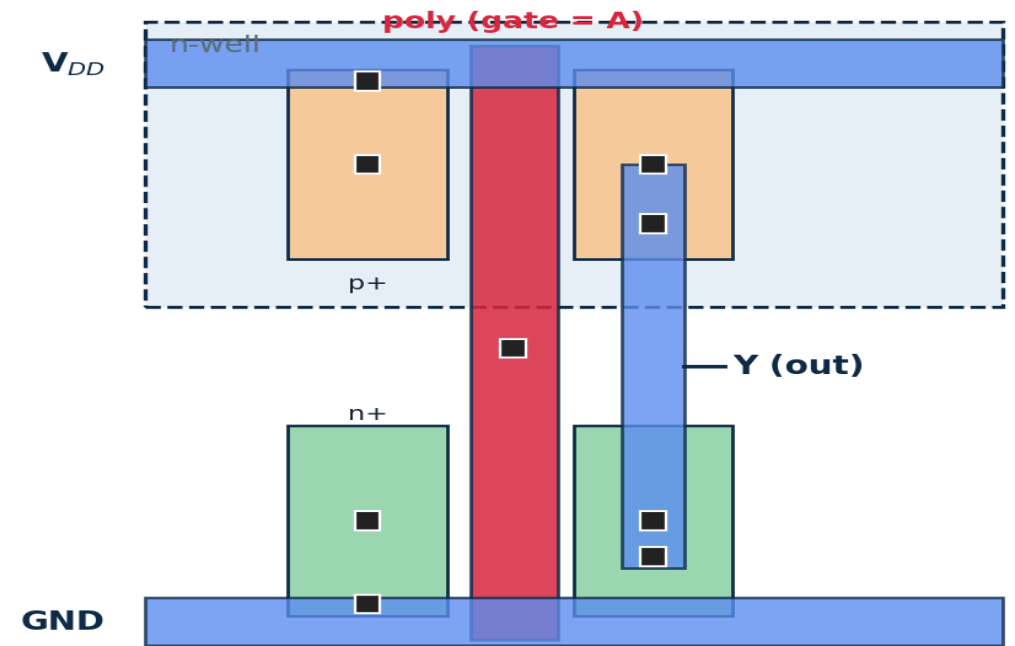
- Both transistors share the input gate and the output node — exactly the schematic.



Layout = the schematic, in geometry

Every wire and device in the schematic maps to specific polygons on specific mask layers.

CMOS inverter layout (top view)

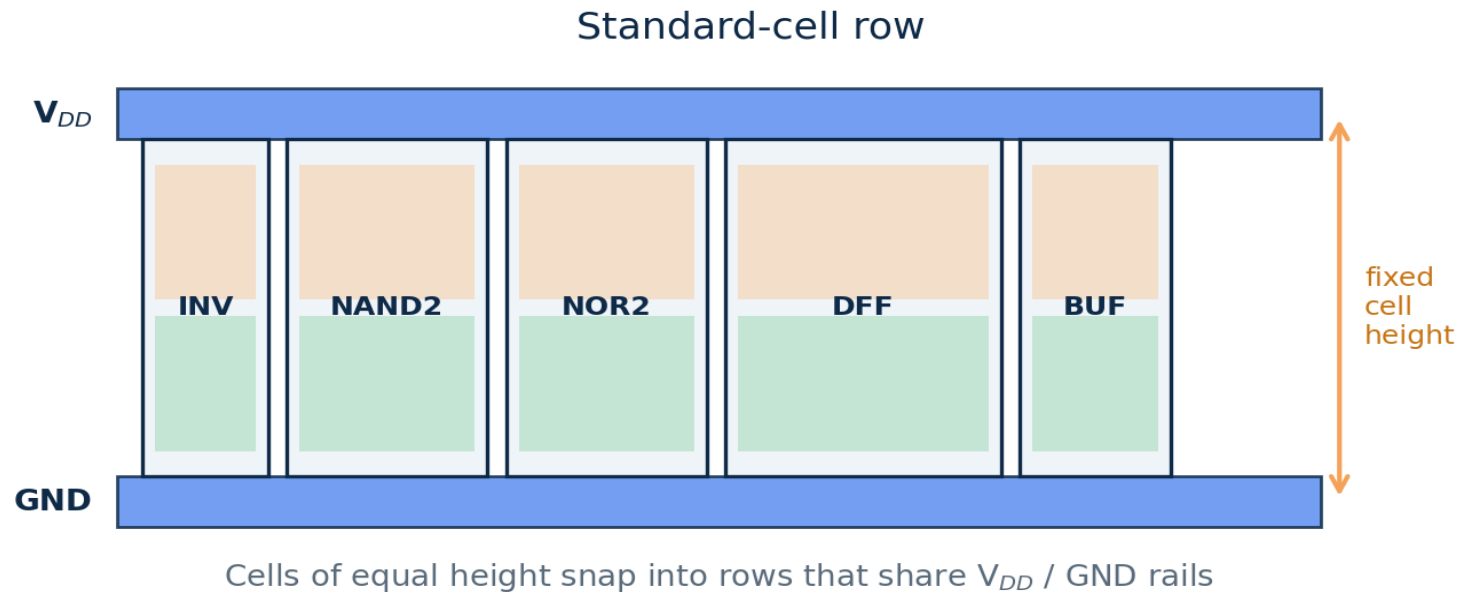


Contacts (black) tie diffusion/poly to metal



Standard-Cell Layout Strategy

Rather than hand-draw every gate, modern chips use a library of fixed-height standard cells that the tools place in rows — the basis of automated physical design.



Why standard cells?

Pre-characterised for timing & power.

Enable automatic place & route.

Reused across designs → fast turnaround.

- *Full-custom is reserved for critical blocks.*

Fixed cell height lets cells abut into rows that share V_{DD} / GND rails; width varies with complexity.



Layout Design Rules (λ -rules)

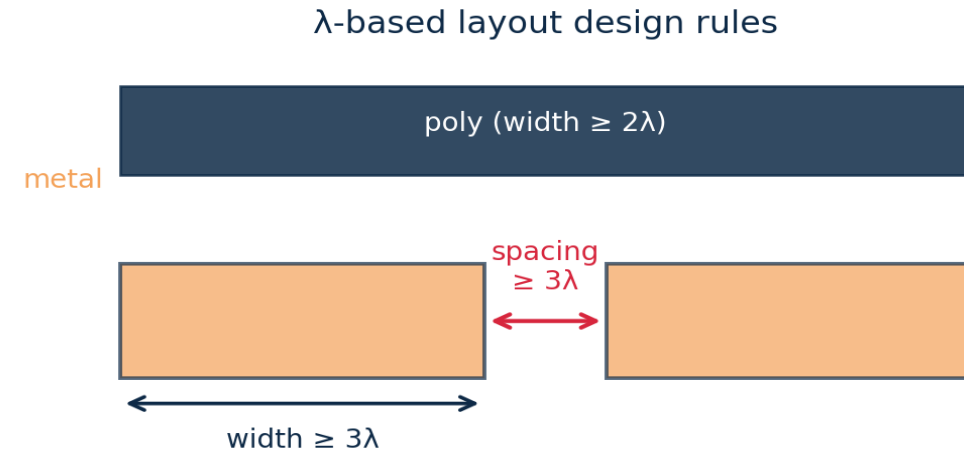
Design rules are the minimum widths and spacings the foundry guarantees it can build reliably.

Expressed in λ (**lambda**) — a scalable unit $\approx$ half the minimum feature size.

- Examples: min metal width 3λ , min spacing 3λ , gate poly 2λ .

DRC (Design-Rule Check) flags every violation before tape-out.

LVS separately checks Layout-vs-Schematic equivalence.



Design rules are expressed in multiples of λ (half the min. feature size)



Example — λ scaling

If $\lambda = 0.1 \mu\text{m}$, a 3λ wire is $0.3 \mu\text{m}$ wide. Move to a newer node ($\lambda = 0.05 \mu\text{m}$) and the *same drawing* shrinks $2\times$ automatically.



Physical Verification: DRC & LVS

Before a layout can be sent to the foundry it must pass two automated checks that guarantee it is both manufacturable and correct.

DRC — Design Rule Check

Checks the layout obeys every **geometric rule** (min width, spacing, enclosure).

Guarantees the shapes can actually be **fabricated** with acceptable yield.

Answers: *"Can this be made?"*

LVS — Layout vs Schematic

Extracts a netlist from the layout and **compares** it to the schematic.

Confirms the geometry implements the **intended circuit** — same devices and connections.

Answers: *"Did we draw the right circuit?"*



In this course, **Magic** performs DRC (and extraction) on your layouts. Both checks must be clean before tape-out.



Metal Layers & the Routing Stack

Wiring runs on several stacked **metal layers** (M1, M2, ... up to 10+), separated by insulator.

Lower metals (M1–M3) are thin and dense → local, short connections.

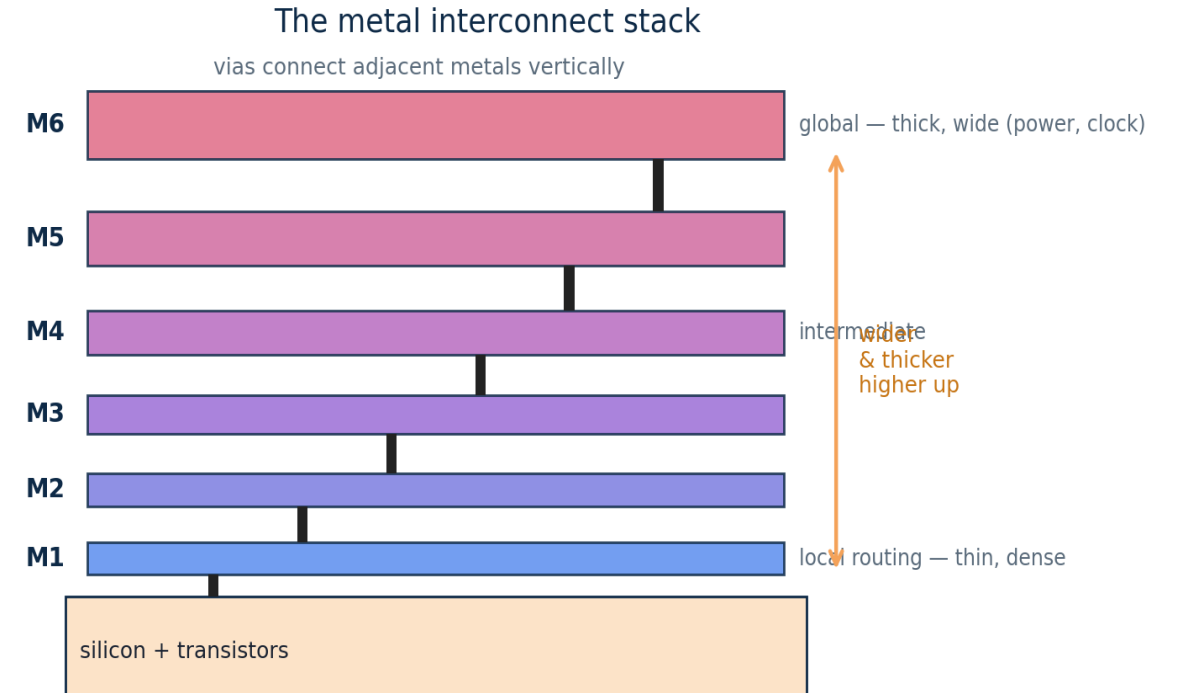
Upper metals are thick and wide → low-resistance global routing, power grids and clocks.

Vias are the vertical connections between adjacent metal layers.



Why so many layers?

Millions of nets cannot share one plane. Many layers give the **routing capacity** to connect everything without short circuits.

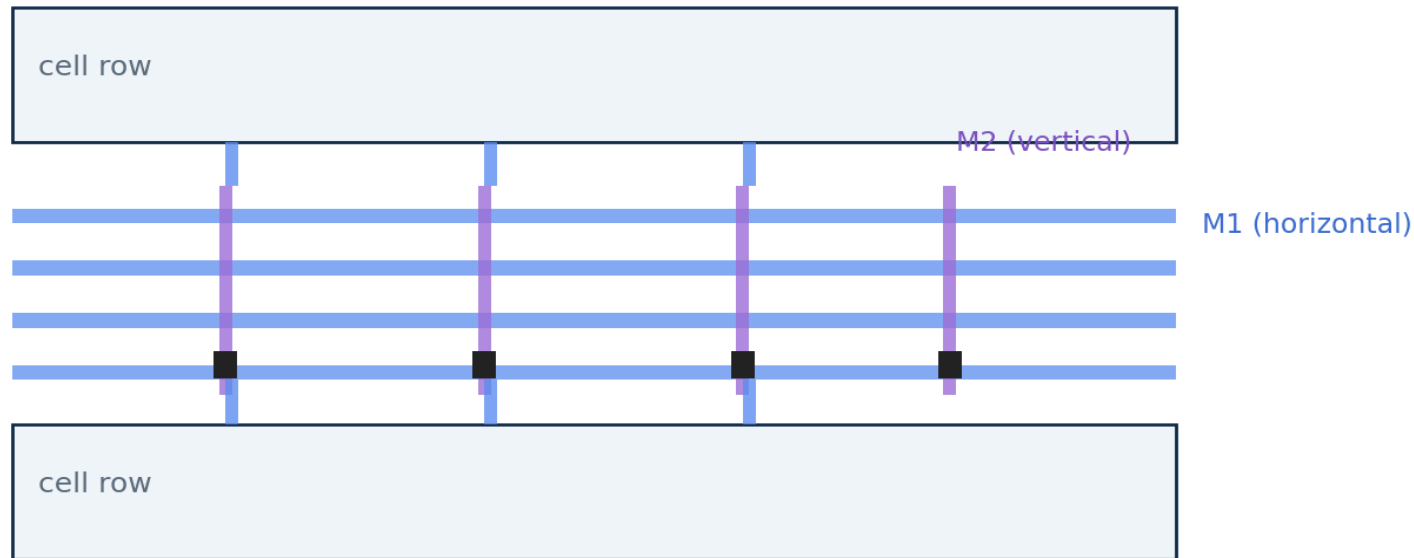




Interconnect Routing

Routing is the step that connects all the placed cells with wires on the metal stack, obeying the design rules and the available tracks.

Interconnect routing between cells



Router uses alternating metal directions (M1 horizontal, M2 vertical) joined by vias

Preferred directions: one layer routes horizontally, the next vertically, to avoid clashes.

Vias jump between layers where a wire must change direction.

Tracks are the legal wire positions; congestion happens when too many nets want the same region.

Goals: 100% connection, meet timing, minimise wire length & vias.

On-Chip Wire Modeling

Wires are no longer free. At modern nodes the interconnect — not the transistor — often sets a chip's speed and power.

a

Introduction to on-chip interconnects

The wire hierarchy and why interconnect now matters

b

Wire parasitics & their impact

R, C and coupling — delay, crosstalk, IR drop, electromigration

c

Modeling & simulation of on-chip wires

Lumped / π / distributed / Elmore models, repeaters and extraction

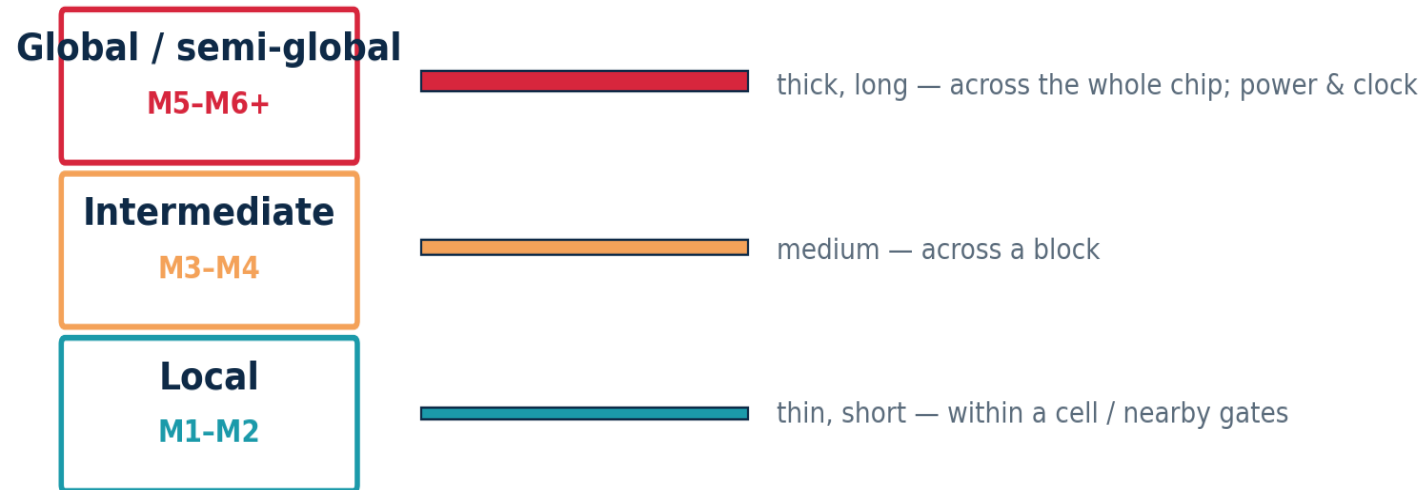




Introduction to On-Chip Interconnects

An interconnect is simply the metal wire that carries a signal between gates. A modern chip holds kilometres of wiring across many metal tiers.

On-chip interconnect hierarchy



Higher tiers = longer, thicker wires (lower R per length) for chip-wide signals

Local wires are short and thin; **global wires** are long and thick (lower resistance) for chip-wide signals, power and clocks.

The interconnect problem

Transistors got faster every node.

Wires got relatively slower (thinner $\rightarrow$ more R).

So interconnect now **dominates** delay, power and area on many paths.



On-Chip Wires Are Not Ideal

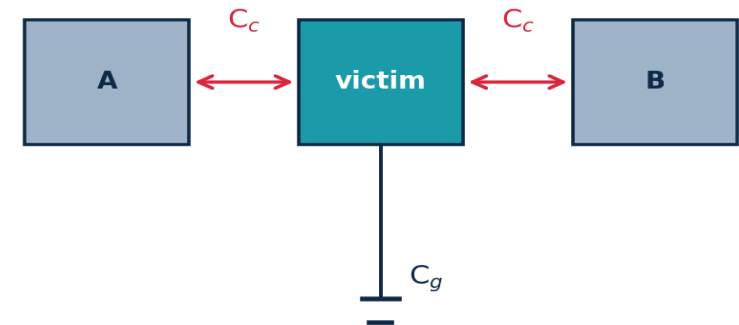
Every wire has **resistance R** (grows with length, shrinks with width/thickness).

and **capacitance C** — to the substrate (C_g) and to neighbours (coupling C_c).

Coupling C_c causes **crosstalk** — a switching neighbour disturbs a quiet wire.

At advanced nodes wire delay can **exceed** gate delay.

Each wire has capacitance to ground (C_g) and to neighbours (C_c)



substrate / ground plane



Scaling reality

Transistors got faster every node, but wires got **relatively slower** — so interconnect now dominates many timing paths.



Wire Resistance & Capacitance

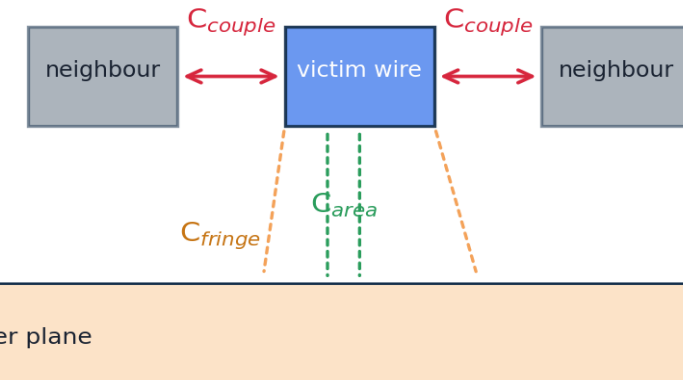
Resistance $R = \rho \cdot L / (W \cdot t) = R_{\text{sheet}} \cdot (L/W)$. Long, thin wires are resistive.

Capacitance has three parts: **area** (to the plane below), **fringe** (edge fields) and **coupling** (to side neighbours).

As wires get closer, **coupling C** dominates — and it is data-dependent.

Where a wire's capacitance comes from

Total wire C = area + fringe + coupling



Worked example — wire R

$R_{\text{sheet}} = 0.1 \, \Omega/\square$, length $200 \, \mu\text{m}$, width $0.2 \, \mu\text{m}$.

Squares = $L/W = 200/0.2 = 1000$.

$R = 0.1 \times 1000 = 100 \, \Omega$ for that single wire.



Impact on Circuit Performance

Wire parasitics degrade a design in four distinct ways. Physical design exists largely to keep these under control.



Propagation delay

RC of the wire adds to path delay; long wires can dominate timing.



Crosstalk noise

Coupling C lets a switching aggressor inject noise onto a victim net.



IR drop

Resistance in the power grid lowers the local V_{DD} , slowing gates.



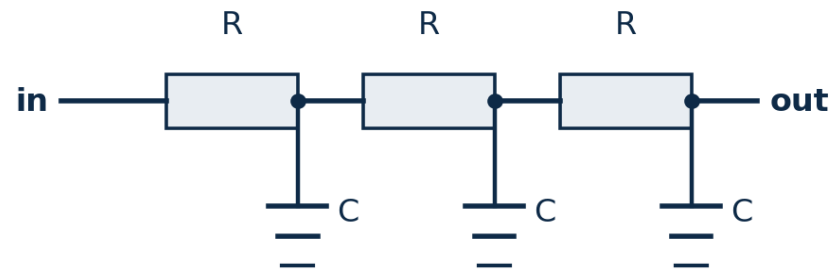
Electromigration

High current density in thin wires moves metal atoms → long-term failure.



RC Wire-Delay Models

A long on-chip wire behaves as a distributed RC network

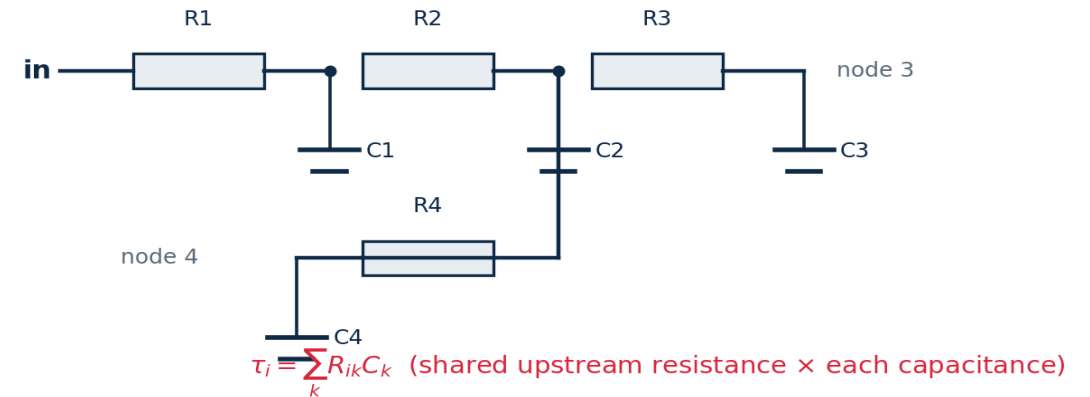


Elmore delay $\tau \approx 0.69 R_{total} C_{total} \rightarrow$ delay grows with wire length²

Lumped RC: treat the whole wire as one R and one C. Simple, pessimistic.

Distributed RC: a ladder of small R-C segments. Delay $\propto$ length².

Elmore delay on an RC tree



Worked example — Elmore delay

Wire as 2 segments: $R_1 = R_2 = 1 \text{ k}\Omega$, $C_1 = C_2 = 10 \text{ fF}$.

$$\tau = R_1(C_1 + C_2) + (R_1 + R_2)C_2 = 1\text{k} \cdot 20\text{f} + 2\text{k} \cdot 10\text{f} = \mathbf{40 \text{ ps.}}$$

Fix long wires by inserting buffers to break length² growth.

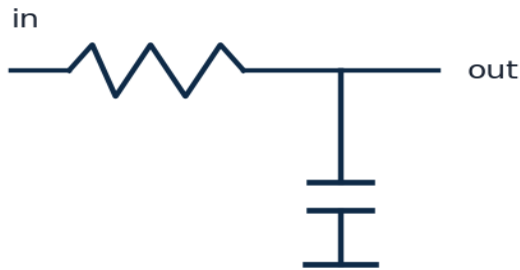


Choosing a Wire Model

How a wire is modelled trades accuracy for effort. Pick the simplest model that is accurate enough for the job.

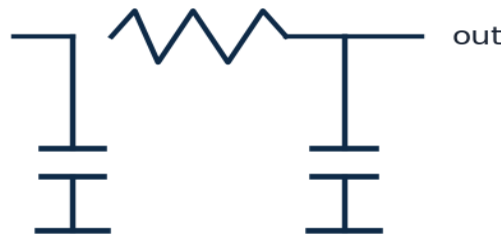
Modeling a wire: from one lump to a distributed ladder

Lumped (R, C)



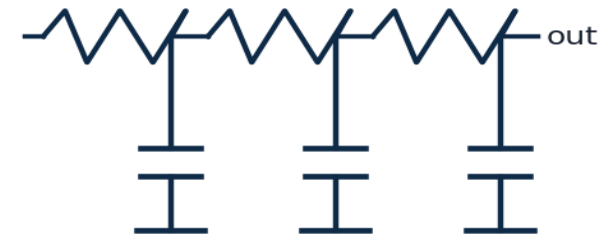
simple, pessimistic

π -model ($C/2 \cdot R \cdot C/2$)



good single-segment fit

Distributed (R-C ladder)



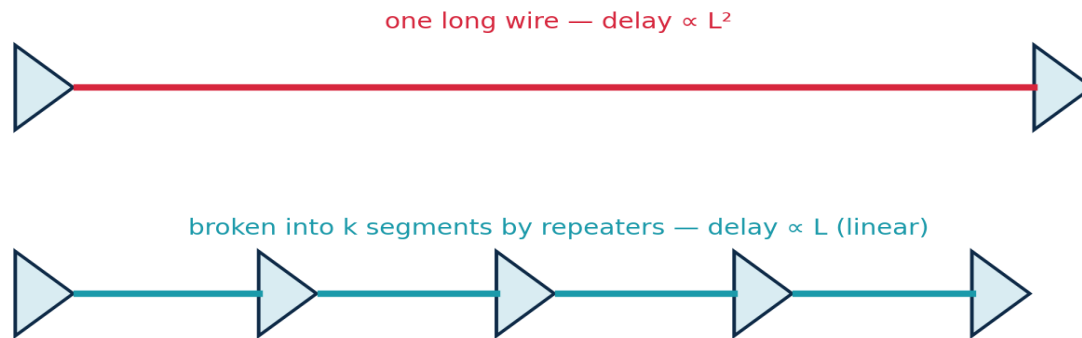
accurate; delay $\propto L^2$

Model	Use it for	Accuracy
Lumped R, C	Quick hand estimates of short nets	Pessimistic
π -model ($C/2 \cdot R \cdot C/2$)	One-segment sign-off of medium nets	Good
Distributed / Elmore	Long nets and RC trees	High



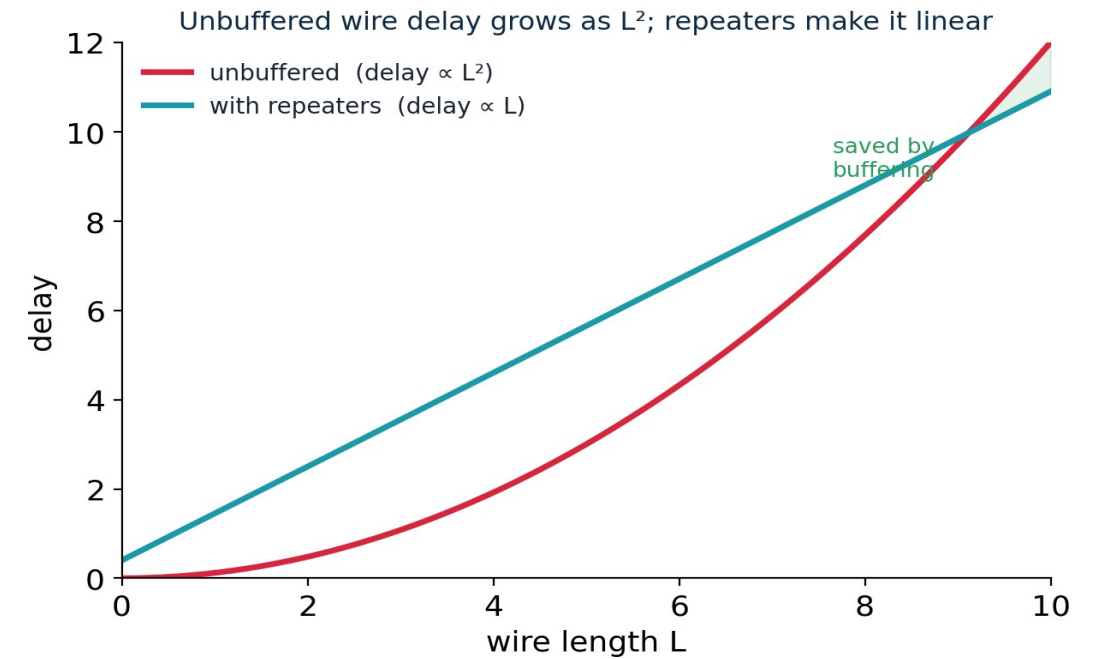
Repeaters: Taming Long-Wire Delay

Repeater (buffer) insertion linearises long-wire delay



An unbuffered wire's delay grows as L^2 (distributed RC).

Splitting it into k segments with **repeaters (buffers)** makes delay grow linearly with L.

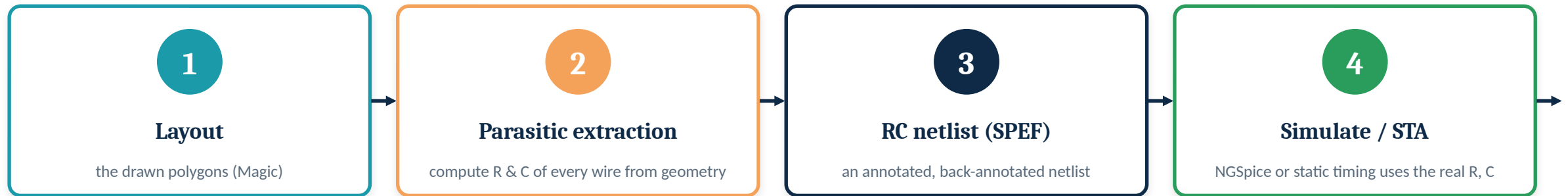


There is an optimal repeater size and spacing that minimises total delay — a standard back-end optimisation.



Extracting & Simulating Wires

Before sign-off, the real wires in a layout are turned into R and C values and simulated — this is how timing tools know the true delay.



Extraction reads the layout geometry and the process data to compute resistance and capacitance for each net.

The result feeds **circuit simulation (NGSpice)** and static timing — closing the loop between layout and real-world performance.



In this course

Magic extracts parasitics from your layout; **NGSpice** then simulates the netlist with those R and C values to confirm the delay you estimated by hand.

Bonding, Packaging & Assembly

A bare die cannot be used on its own. Packaging protects it, connects it to the board, and removes its heat — the bridge from silicon to system.

a

Packaging technologies & assembly

The assembly flow and the main package families

b

Bonding diagram design & considerations

Wire-bond vs flip-chip, and mapping die pads to package pins

c

Packaging trends & challenges

Advanced packaging, thermal, signal integrity, heterogeneous integration

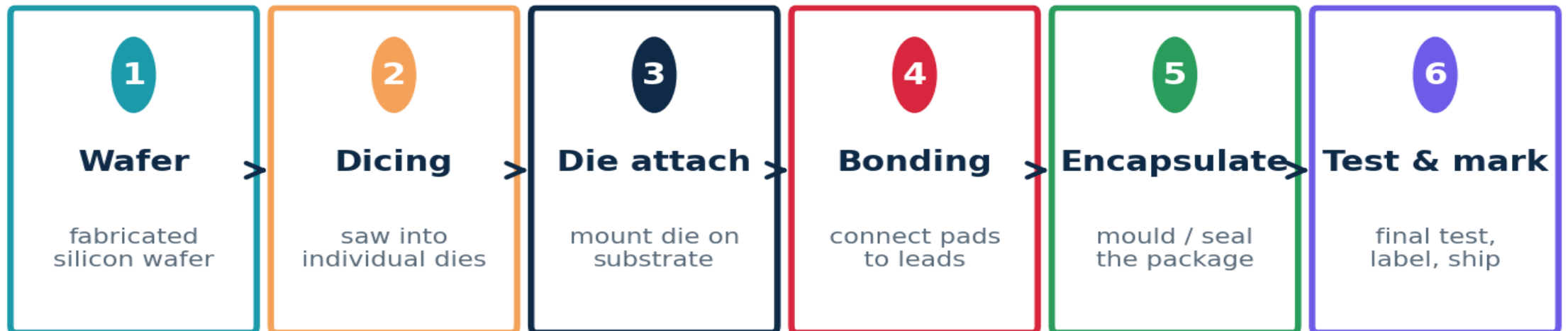




From Die to Package: the Assembly Flow

After fabrication, the wafer is cut into dies and each good die is assembled into a package. Packaging gives the chip mechanical protection, electrical connections and a heat path.

Assembly flow: from wafer to packaged chip

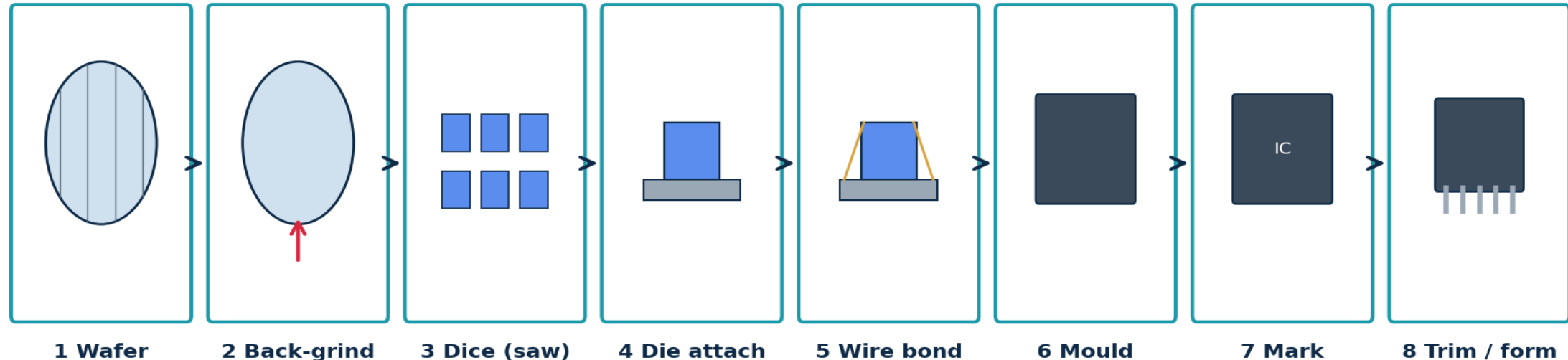


A package must deliver four things: protection, power & signal connections, heat removal, and a footprint the board can use.



The Assembly Process, Step by Step

The assembly process, stage by stage (physical view)



1-2 Wafer & back-grind: the finished wafer is thinned from the back so the die is flatter and dissipates heat better

4 Die attach: each good die is picked and glued onto the leadframe paddle (or substrate) with epoxy

6 Encapsulation (mould): the assembly is sealed in epoxy mould compound (or a ceramic/metal lid) for protection

3 Dicing (sawing): a diamond saw (or laser) cuts the wafer into individual dies along the scribe lines

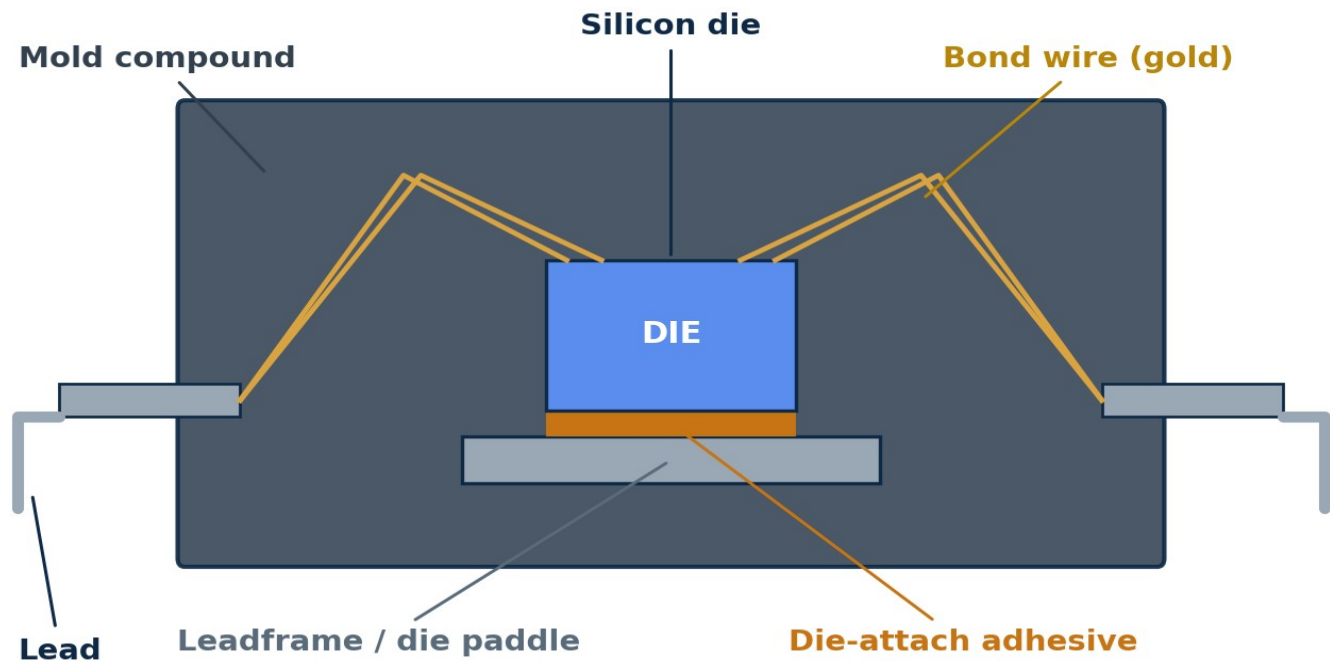
5 Wire bonding: fine gold/copper wires join the die's bond pads to the package leads

7-8 Mark, trim & form: laser-mark the part number, then trim the leadframe and bend the leads to shape; final test follows



Anatomy of a Package

Anatomy of a wire-bonded plastic package (cross-section)



What is actually inside a packaged chip:

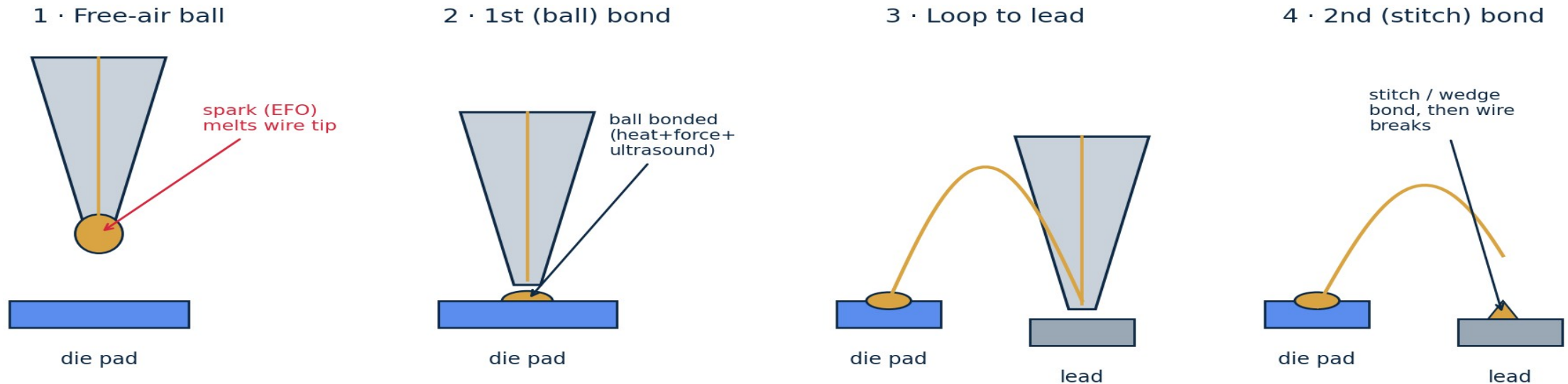
-  **Silicon die:** the chip itself
-  **Die-attach adhesive:** glues the die to the paddle, conducts heat
-  **Leadframe / paddle:** metal frame that supports the die and forms the leads
-  **Bond wires:** carry signals from die pads to the leads
-  **Mould compound:** epoxy that seals and protects everything
-  **Leads / balls:** the external pins the board solders to



How a Wire Bond Is Made

The bonding step (4 in the assembly flow) is itself a precise four-stage process. The most common method is thermosonic ball-stitch bonding, using heat, force and ultrasonic energy.

How a wire bond is made: thermosonic ball-stitch bonding



1 Free-air ball: an electric spark (EFO) melts the wire tip into a tiny ball at the capillary

3 Loop: the capillary rises and moves to the lead, paying out a controlled arc of wire

2 First (ball) bond: the ball is pressed onto the die pad — heat + force + ultrasound weld it

4 Second (stitch) bond: the wire is crushed onto the lead (wedge bond), then the wire is broken to finish

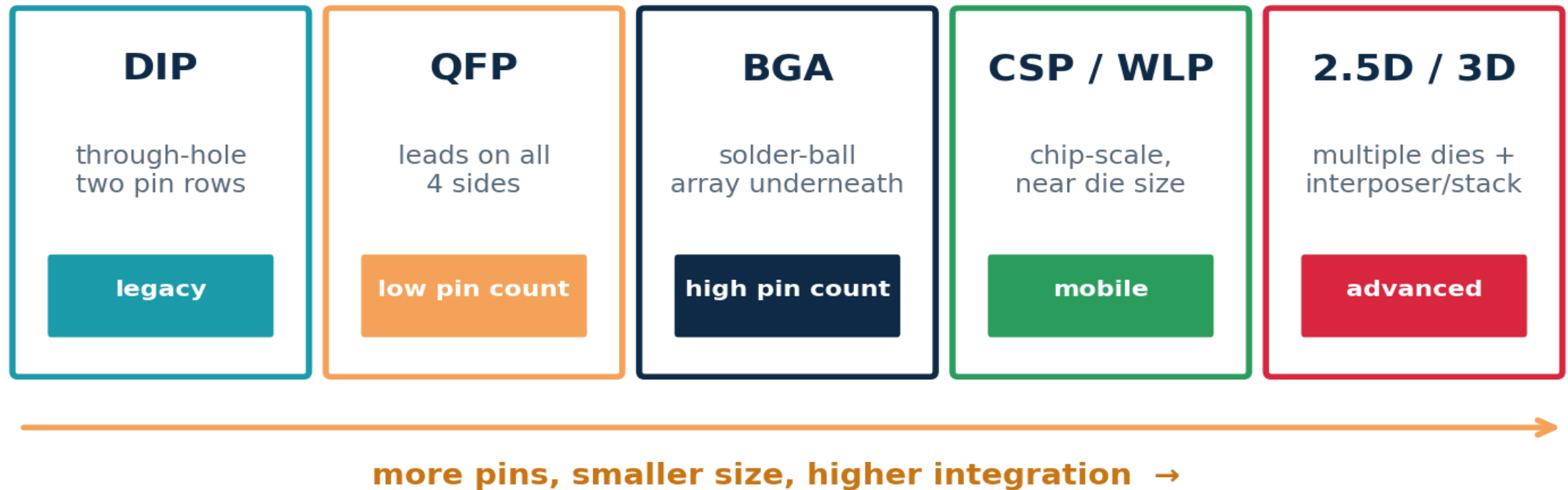
Wedge bonding (aluminium wire, no ball) is the other main method — slower, but used for power and aluminium pads.



Packaging Technologies (Families)

Packages have evolved from a handful of through-hole pins to dense ball arrays and stacked multi-die modules, tracking the rising pin counts and integration of the chips inside.

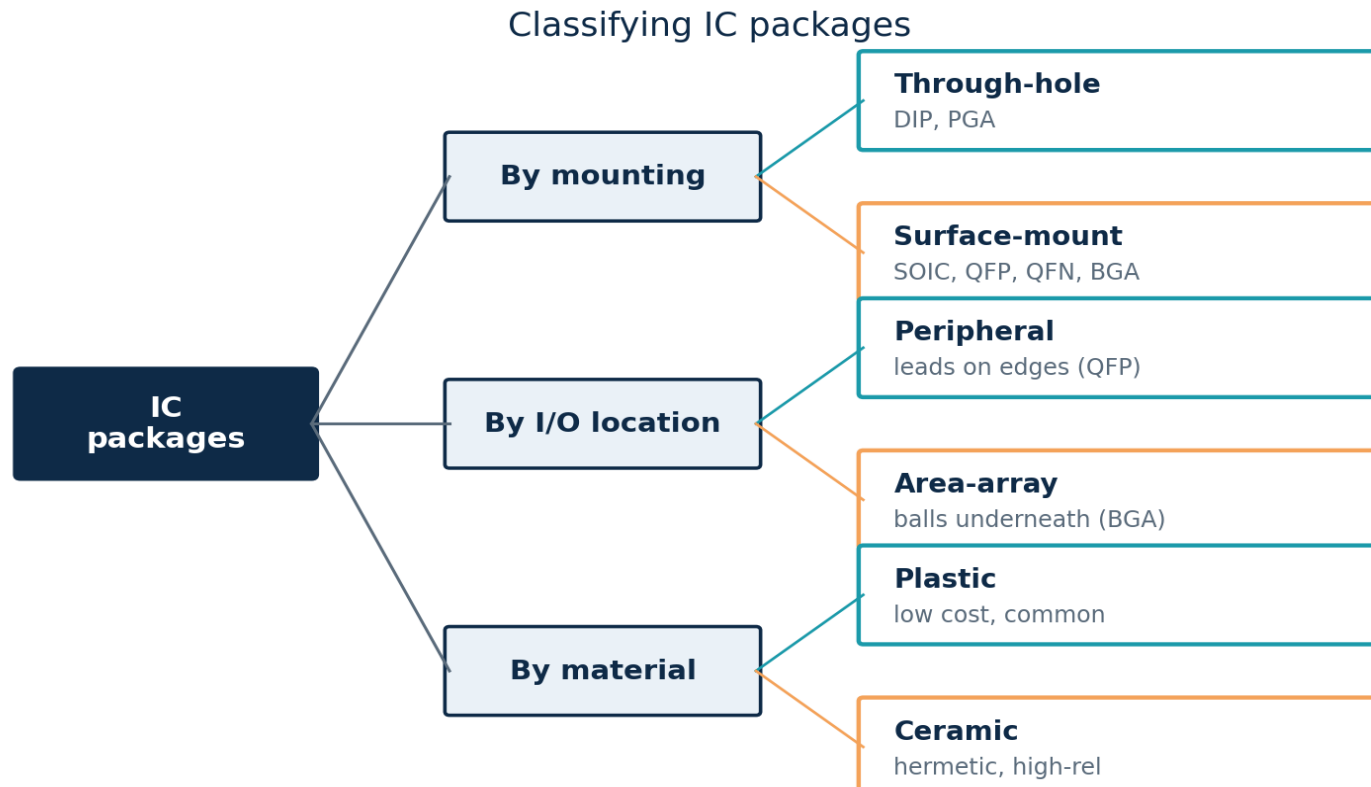
Package families: from DIP to advanced 2.5D / 3D





Classifying Packages

The same families can be sorted along three independent axes — a useful way to read any package's datasheet.



How to read it

Mounting: through-hole pins go through the board; surface-mount sit on top (denser, dominant today).

I/O location: peripheral leads sit on the edges; area-array balls cover the underside (far more I/O).

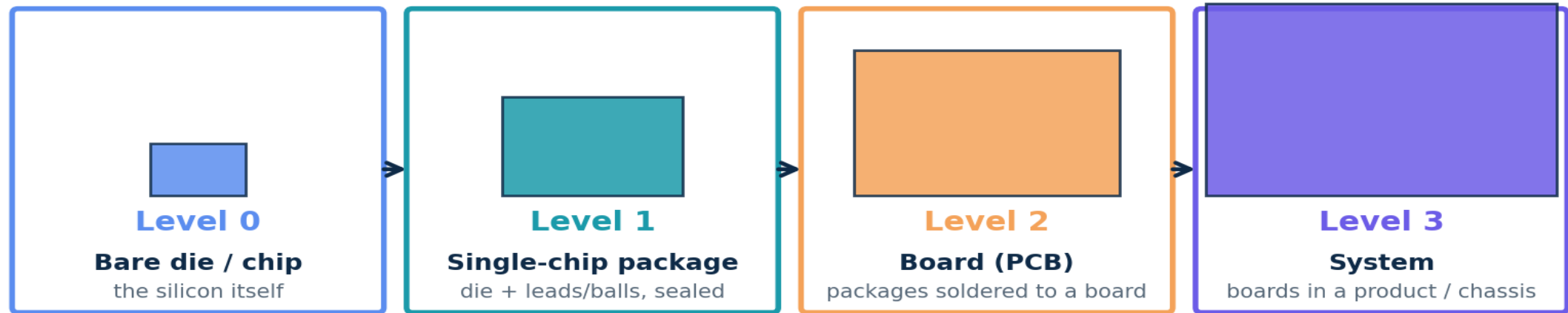
Material: plastic is cheap and common; ceramic is hermetic for harsh/high-reliability use.



Levels of Packaging

Packaging is a hierarchy. The single-chip package (Level 1) is just one rung between the bare die and the finished product.

Levels of electronic packaging



increasing scale: chip → package → board → system

Each level adds interconnect: bond wires/bumps (L1), solder joints to the PCB (L2), and connectors/cables (L3) — every hop adds delay, loss and cost.

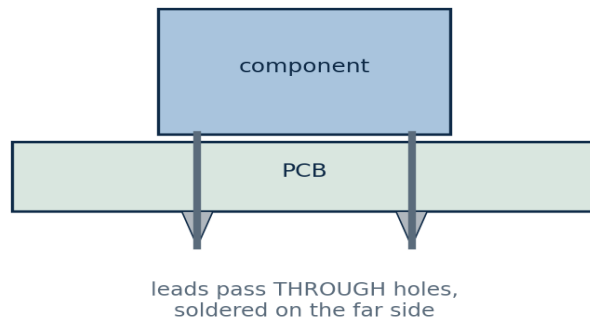


Through-Hole vs Surface-Mount

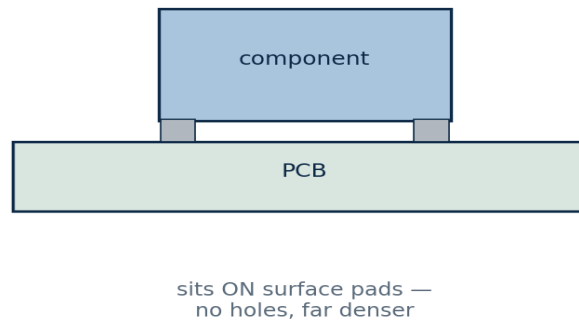
How a package attaches to the board is a basic technology split. Surface-mount has largely replaced through-hole because it is far denser and machine-assembled.

Two ways a package mounts to the board

Through-hole (THT)



Surface-mount (SMT)



Why SMT won

Both sides of the board can be populated.

No drilled holes → smaller pads, finer pitch.

Automated pick-and-place + reflow soldering.

Through-hole survives where mechanical strength matters (connectors, power).

Surface-mount enables the QFP, QFN and BGA families; through-hole is the DIP/PGA world.



Package Types at a Glance

A quick reference to the main package types, their typical pin counts and where each is used.

Package	Mounting	Pin count	Pitch	Typical use
DIP	Through-hole	8–64	2.54 mm	legacy, prototyping
SOIC	Surface-mount	8–32	1.27 mm	small jellybean ICs
QFP	Surface-mount	32–256	0.4–0.8 mm	MCUs, ASICs
QFN	Surface-mount	16–64	0.4–0.65 mm	compact, low-cost
PGA	Through-hole	100–1000+	2.54 mm	older CPUs
BGA	Surface-mount	100–2000+	0.4–1.0 mm	high pin count, SoCs
WLCSP	Surface-mount	4–200	0.35–0.5 mm	mobile, chip-scale

Pin count and pitch are indicative — exact ranges vary by vendor and generation.



Reading the trend

As you move down, mounting goes surface-mount, pitch shrinks and pin count climbs — exactly the DIP → BGA/WLCSP progression seen in the family chart.



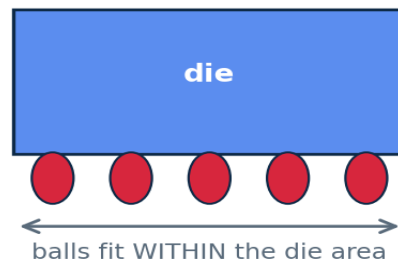
Wafer-Level & Advanced Assembly

Traditionally a wafer is diced first, then each die is packaged. Wafer-level packaging (WLP) builds the package on the whole wafer before dicing — giving the smallest possible packages.

Wafer-level packaging: building the package on the wafer itself

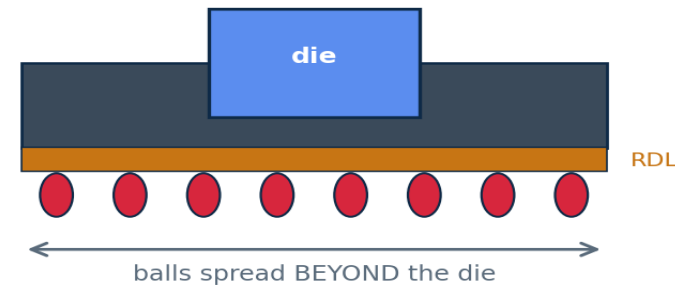
Fan-in (WLCSP)

package = die size; I/O limited to what fits under the die



Fan-out (FOWLP)

die embedded in mould; a redistribution layer fans I/O out → more balls



Why it matters

Smallest footprint and lowest profile.

Shorter connections → better electrical performance.

A path toward 2.5D/3D integration.

Fan-in (WLCSP): the package is the size of the die; solder balls fit within the die footprint. Best for low pin counts and tiny mobile parts.

Fan-out (FOWLP): the die is embedded in mould and a redistribution layer (RDL) spreads the I/O beyond the die, allowing more balls and finer integration.

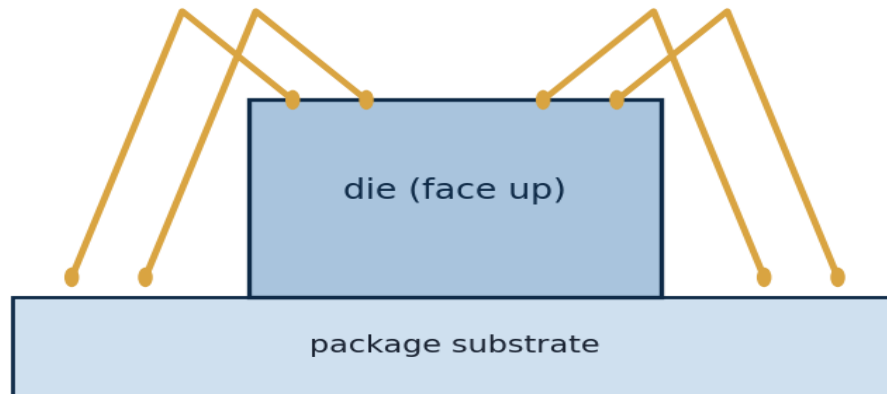


Connecting the Die: Wire-Bond vs Flip-Chip

Two ways to connect a die to its package

Wire bonding (die face-up)

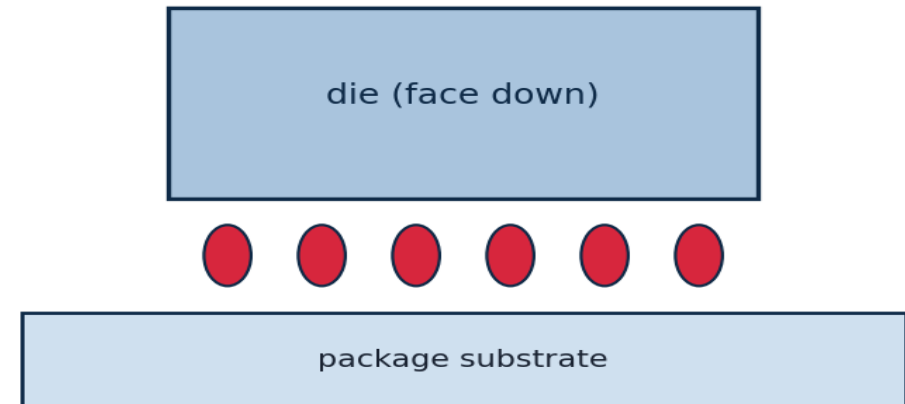
thin gold/copper wires



Wire bond: die face-up; thin gold/copper wires arc from pads to the substrate. Cheap, mature, but pads only on the die edge.

Flip-chip (bumps, die face-down)

solder bumps (area array)



Flip-chip: die face-down on solder bumps in an area array. More I/O, shorter connections, better electrical & thermal — used at high performance.



The Bonding Diagram & Its Considerations

A bonding diagram maps every die pad to its package pin and shows each bond wire — the contract between the chip designer and the assembly house.

Pad placement: distribute pads around the die edge for short, even wires.

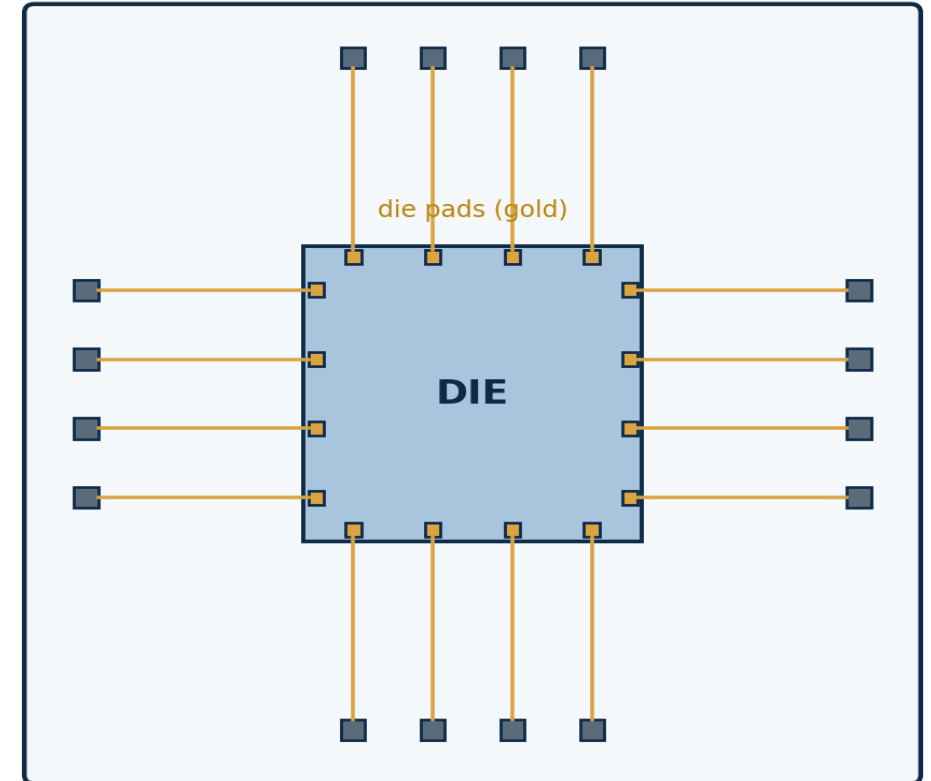
Wire length & angle: keep bonds short and avoid crossing or shorting wires.

Power & ground: use multiple bonds for V_{DD} /GND to lower inductance and IR drop.

Signal integrity: bond-wire inductance causes ground bounce — a real limit on fast I/O.

Pin assignment: match the package pinout to the board for clean routing.

Bonding diagram: die pad → package pin map



package pins (grey) — a bonding diagram maps every pad to a pin



Packaging Trends & Challenges

As single-die scaling slows, packaging has become a primary lever for performance — but it brings its own hard problems.

Trends

- ✓ **Heterogeneous integration:** combine CPU, memory, analog & chiplets in one package
- ✓ **2.5D (interposers):** dies side-by-side on a silicon interposer with dense links
- ✓ **3D stacking:** stack dies vertically (e.g. HBM memory) with through-silicon vias
- ✓ **System-in-Package (SiP):** a whole subsystem in one module

Challenges

Thermal: removing heat from dense, stacked dies.

Signal & power integrity: inductance, ground bounce, IR drop at high speed.

Mechanical stress: thermal-expansion mismatch and warpage.

Yield & test: known-good-die testing before costly stacking.

Cost: advanced packaging can rival the die in cost.

Delay, Power & Timing

How fast can the chip run? The next four sections build from first principles of delay to full sequential timing.

•

Foundations

Propagation delay, transition times and the RC / Elmore model

8

Gate delays & logical effort

Estimating and sizing gates for speed

9

Critical-path optimization

Finding and shortening the longest combinational path

10

Sequential-circuit timing

Flip-flops, setup/hold and pipelining





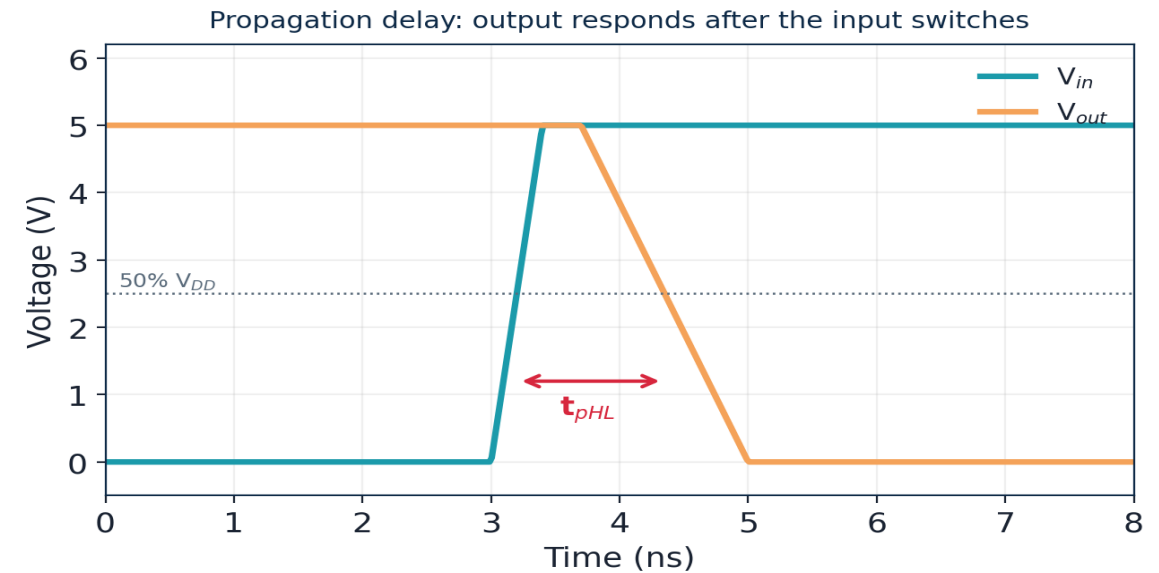
Delay & Transition Times

t_{pHL} / t_{pLH} : output high→low / low→high delay, measured at 50%.

$t_p = \frac{1}{2}(t_{pHL} + t_{pLH})$ — the gate's average delay.

Rise/fall t_r, t_f : 10%–90% transition; set by drive strength and load.

A balanced gate has $t_{pHL} \approx t_{pLH}$ (size PMOS $\sim 2\times$ NMOS to match).



Example — balancing

Electrons move $\sim 2\times$ faster than holes, so a same-size PMOS is $\sim 2\times$ weaker. Widening PMOS $\sim 2\times$ equalises rise and fall delays.



The RC Delay Model

- Model each driver as resistance R , each node as capacitance C .

Single stage: $t_p \approx 0.69 \cdot R \cdot C$.

RC trees: use **Elmore delay** — each node's delay = Σ (shared upstream $R \times$ downstream C).

- Series resistances add; parallel branches share upstream R .



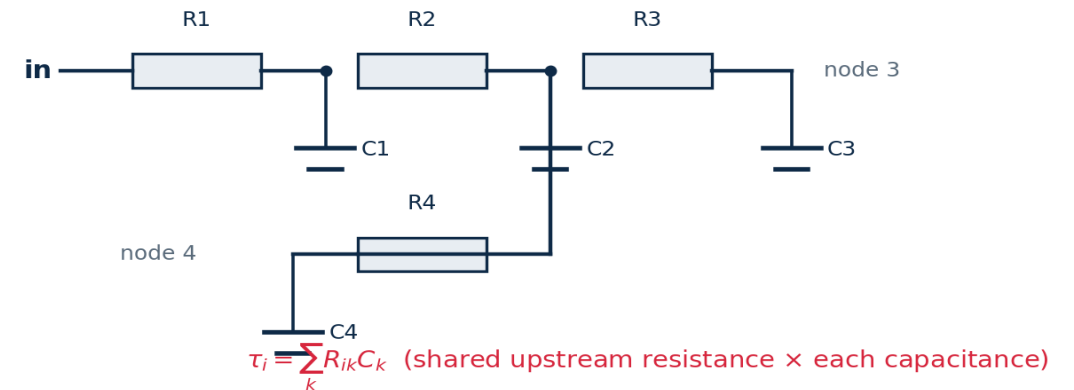
Worked example — two-stage path

Driver $R=6\text{ k}\Omega$ charges $C_1=15\text{ fF}$ (wire) then gate input $C_2=10\text{ fF}$.

Lumped $C = 25\text{ fF} \rightarrow t_p \approx 0.69 \times 6\text{ k} \times 25\text{ f} = \mathbf{104\text{ ps}}$.

To speed up: lower R (bigger driver) or split the wire with a buffer.

Elmore delay on an RC tree



Rule of thumb

Delay scales with **fan-out**: driving more gates adds load C and slows the stage.

Buffer insertion trades a little extra delay per stage for a big win on long nets.

Gate Delays & Logical Effort

A simple, powerful way to estimate gate delay by hand, size gates for speed, and balance speed against power.

a

Gate delay modeling & estimation

The linear delay model $d = g \cdot h + p$; effort, parasitic and electrical effort

b

Introduction to logical effort

Logical effort g of gates, path effort and sizing for minimum delay

c

Delay–power trade-off via P/N ratio

How $\beta = W_p/W_n$ trades speed against area and power





Modeling Gate Delay: $d = g \cdot h + p$

Logical effort models a gate's delay as a straight line. Delay is split into a fixed part and a part that grows with the load the gate must drive.

$d = g \cdot h + p$ (in units of an inverter's basic delay τ).

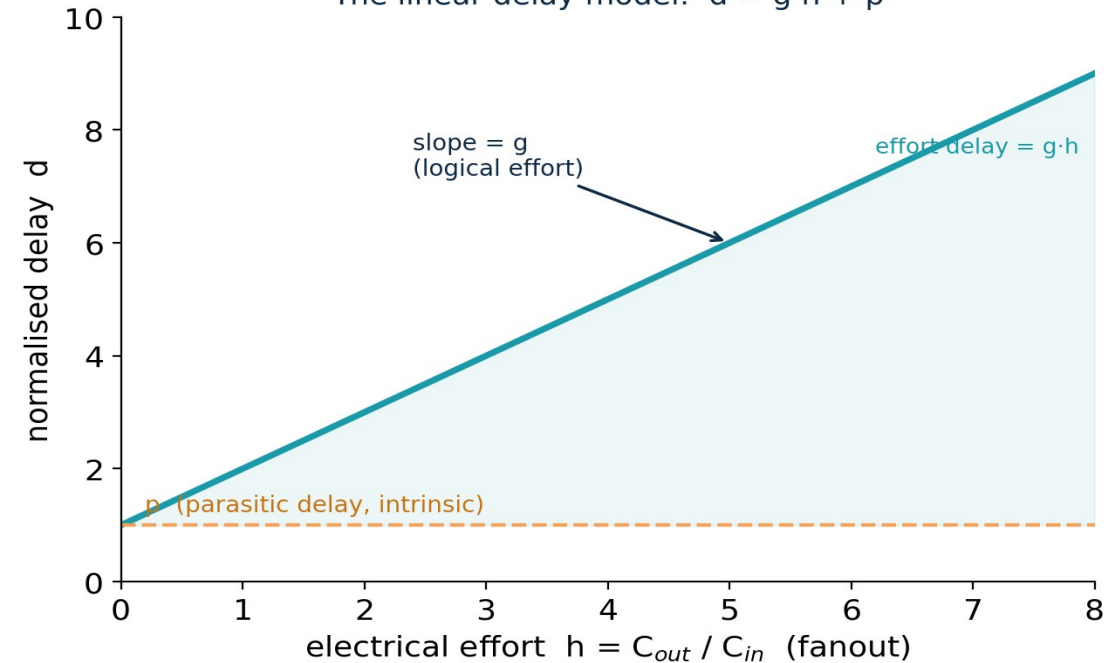
g — logical effort: how much worse than an inverter the gate is at producing output current (gate-type property).

h — electrical effort (fanout): $h = C_{out} / C_{in}$ — the load $\div$ the gate's own input cap.

p — parasitic delay: the intrinsic self-load delay, independent of fanout.

$g \cdot h$ is the **effort delay**; the line's slope is g and its intercept is p .

The linear delay model: $d = g \cdot h + p$



Real delay

Absolute delay = $d \times \tau$, where τ ($\approx$ a few ps) is the technology's basic inverter delay.



Estimating Delay: Fanout & Gate Type

Bigger load (higher h) → more delay, linearly.

A gate with larger g or p is slower than an inverter at the same fanout.

NAND beats NOR: series PMOS in the NOR make it weaker, so $g_{\text{NOR}} > g_{\text{NAND}}$ — prefer NAND-based logic.

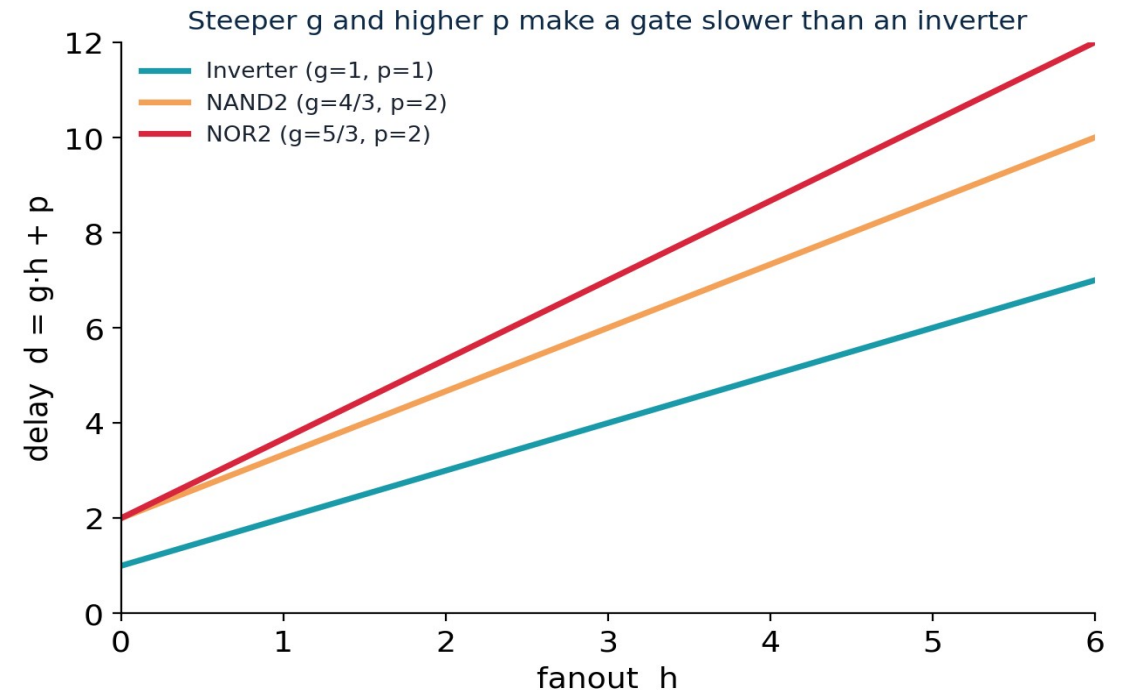


Worked estimate

A NAND2 ($g=4/3$, $p=2$) drives a load $3\times$ its input cap ($h=3$).

$$d = g \cdot h + p = (4/3)(3) + 2 = 4 + 2 = 6\tau.$$

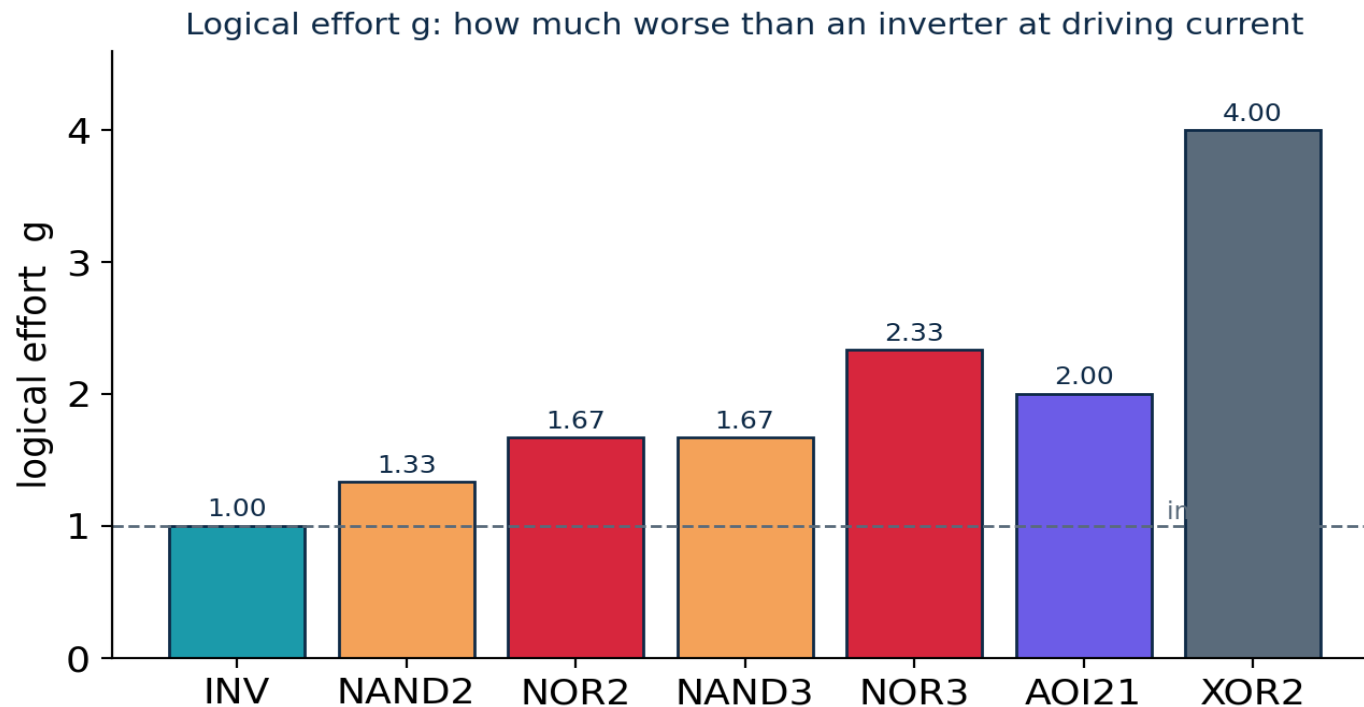
An inverter ($g=1$, $p=1$) at the same h : $d = 3 + 1 = 4\tau$ — clearly faster.





Logical Effort of Common Gates

Logical effort g captures how a gate's topology hurts its drive strength. An inverter is the reference ($g = 1$); every other gate is worse.



Where g comes from

Series transistors must be made wider to keep drive $\rightarrow$ larger input cap $\rightarrow$ larger g .

NAND stacks NMOS (strong); **NOR** stacks PMOS (weak), so NOR's g is higher.

n-input gates: g grows with n — wide gates are costly.

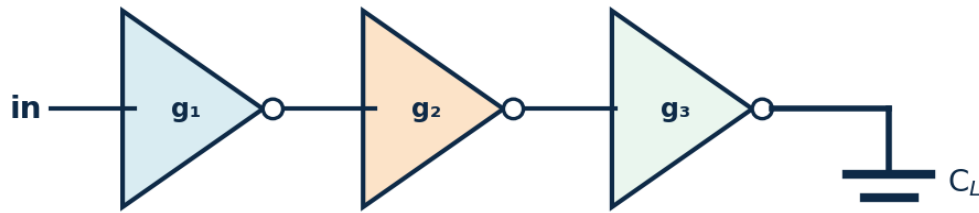
Rule: keep fan-in low and favour NAND.



Sizing a Path for Minimum Delay

Logical effort extends to a whole path of gates, telling us how to size each stage so the path is as fast as possible.

Path effort $F = G \cdot B \cdot H$ ($G = g_1 g_2 g_3$, $H = C_L / C_{in}$)



Minimum delay when every stage carries equal effort: $f = (F)^{1/N}$

Worked: $F = 64$

Best stage effort $\approx 4 \rightarrow N = \log_4 64 = 3$ stages.

- Each stage $f = 64^{(1/3)} = 4$; size $1\times \rightarrow 4\times \rightarrow 16\times$.

Path effort $F = G \cdot B \cdot H$ (G = product of g 's, B = branching, $H = C_L / C_{in}$).

Minimum delay when each of N stages carries equal effort **$f = F^{(1/N)}$** ; then size stages back-to-front from $C_{in} = g \cdot C_{out} / f$.



The P/N Ratio ($\beta = W_p / W_n$)

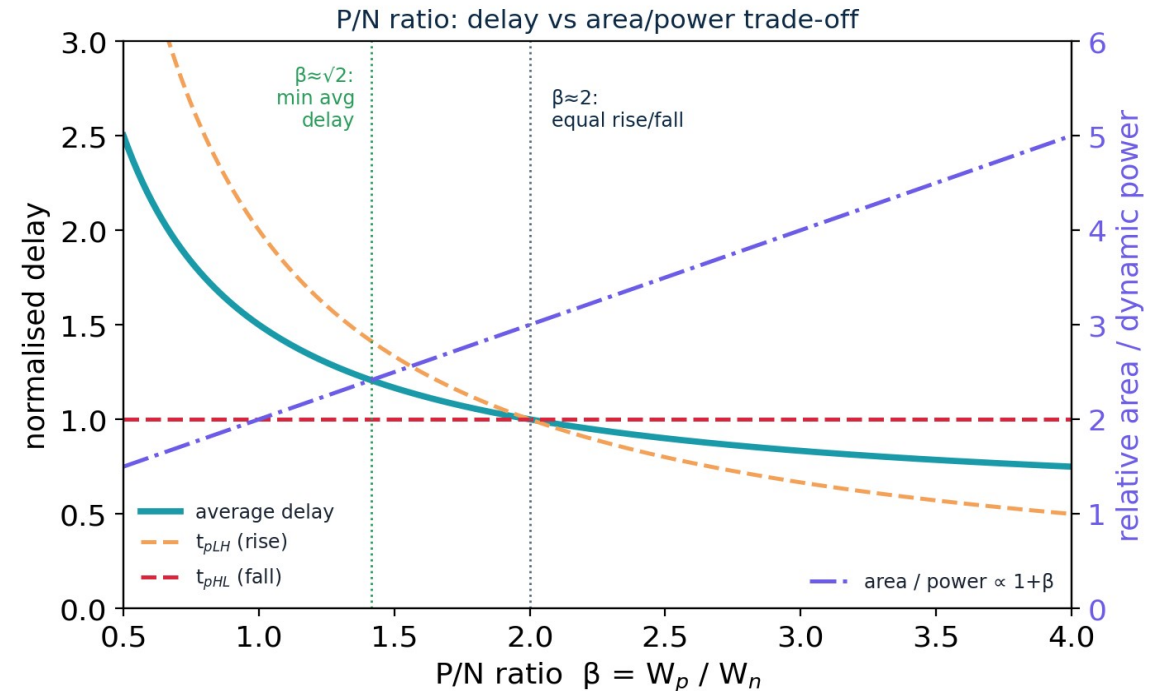
The P/N ratio sets how strong the PMOS pull-up is relative to the NMOS pull-down — the central knob for trading delay against area and power.

Holes are ~2× slower, so an equal-width PMOS is weaker; raising β strengthens the pull-up.

$\beta \approx 2$: equal rise and fall times (symmetric VTC, balanced margins).

$\beta \approx \sqrt{2}$ (~1.4): minimum average delay — the speed-optimal point.

$\beta = 1$: smallest PMOS → least area, input cap and dynamic power, but slower rise.



The trade-off

Larger β buys speed/symmetry; smaller β buys area & power.
Area/power $\propto (1 + \beta)$, so pushing β up costs more on every gate.



Choosing the Best P/N Ratio

There is no single 'right' β — it depends on what the design is optimising. These are the standard operating points.

P/N ratio β	What it optimises	Cost / trade-off
1	Minimum area, input cap & dynamic power	Asymmetric VTC; slower rise than fall
$\sqrt{2} \approx 1.4$	Minimum average gate delay (speed)	Rise/fall slightly unequal
2	Equal rise/fall, symmetric VTC, balanced noise margins	More area & power than $\beta=1$
>2	Faster rise / drives heavy capacitive loads	Diminishing returns; large area & power



Practical guidance

Low-power / dense logic: use $\beta \approx 1$ –1.5 to save area and switching power.

Balanced standard cells: use $\beta \approx 2$ so cells behave symmetrically in timing.

Speed-critical nets: size up (β toward 2 and beyond) only where the path demands it — the energy cost is paid on every switch.

Critical-Path Optimization

A chip is only as fast as its slowest combinational path. Finding that path — and shortening it — is the heart of timing closure.

a

Identifying critical paths

Arrival times, the longest path, slack and static timing analysis

b

Gate sizing & logic restructuring

Upsizing critical gates and rebuilding logic into shallow trees

c

Timing optimization for performance

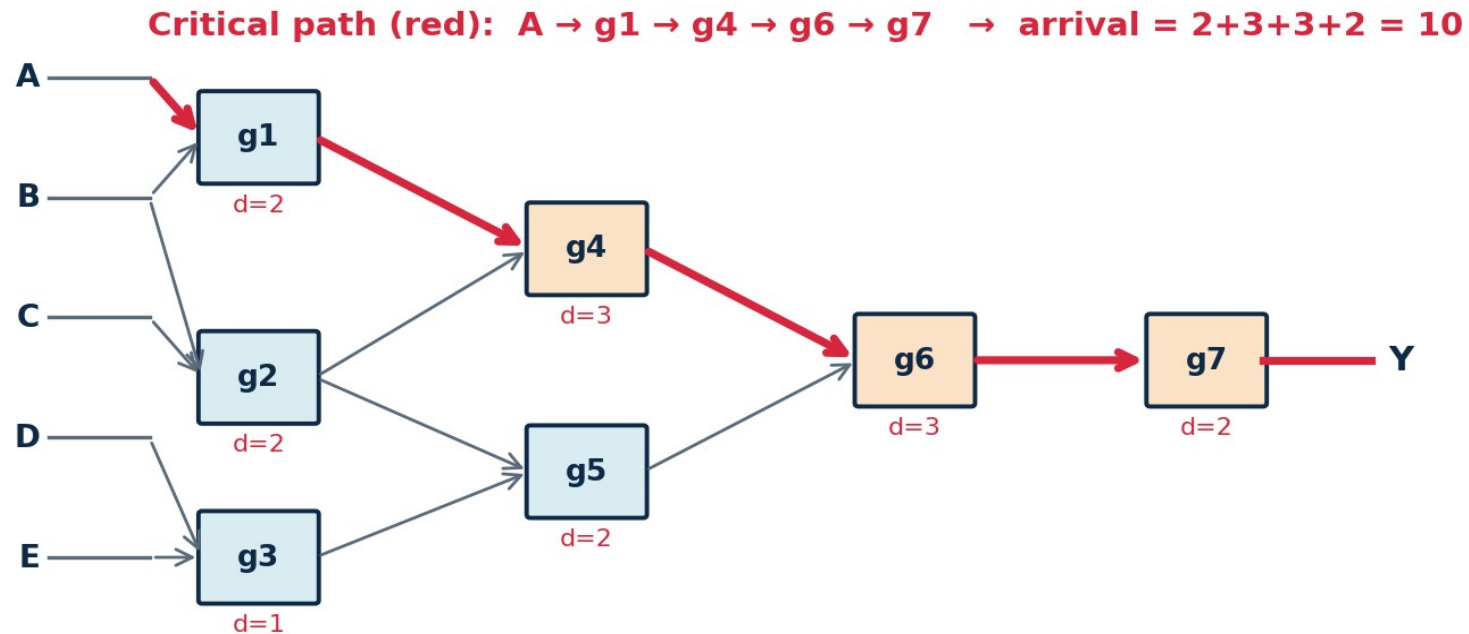
The optimisation loop and the full toolbox of speed-up techniques





What Is the Critical Path?

The critical path is the longest delay path from any input (or register) to any output (or register). Its delay sets the fastest clock the circuit can run at.



Every other path arrives earlier; the longest (critical) path sets the maximum clock speed

How to find it

Arrival time at each gate = max of input arrivals + that gate's delay.

Propagate forward from inputs to outputs.

The output with the **largest arrival** is the critical path.

Here: $A \rightarrow g1 \rightarrow g4 \rightarrow g6 \rightarrow g7 = 10$, the longest of all paths.



Static Timing Analysis & Slack

Real designs have millions of paths, so tools find the critical ones automatically with Static Timing Analysis (STA) — no simulation vectors needed.

Arrival time: when a signal actually reaches a node (forward pass).

Required time: when it must arrive to meet the clock (backward pass).

Slack = required – arrival. Negative slack = a timing violation.

The critical path is the one with the worst (most negative) slack.

STA also flags **false paths** (can't actually toggle) and **multicycle paths** so they aren't fixed needlessly.

The timing equation

$$T_{\text{clk}} \geq t_{\text{cq}} + t_{\text{logic}} + t_{\text{setup}}$$

t_{logic} is the combinational critical-path delay — the only term logic optimisation can shrink.

Lower t_{logic} → higher clock frequency.



Why only the critical path matters

Speeding up a non-critical path does nothing for the clock. Effort goes only where slack is negative — then the next-worst path becomes the new target.



Technique 1: Gate Sizing

The quickest fix: make the transistors in the critical gates wider so they drive their load faster. Logical effort tells us how much.

Gate sizing: widen gates on the critical path to cut their delay

Before: uniform 1× gates



path delay = 12

Upsize only the gates on the critical path — leave the rest small.

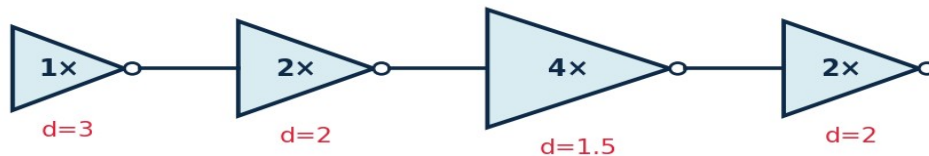
Larger gate → lower output resistance → less delay.

Cost: more area, input capacitance and power.

Diminishing returns: a bigger gate also loads the stage before it.

Best done with logical-effort sizing (Topic 8).

After: upsize the critical gates



path delay = 8.5

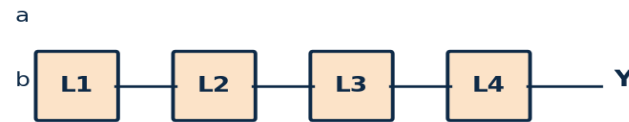


Technique 2: Logic Restructuring

Sizing only goes so far. Often the bigger win is changing the logic itself so the critical signal passes through fewer levels.

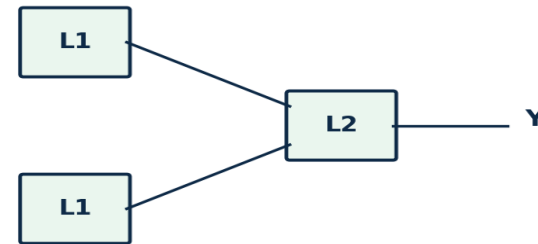
Logic restructuring: re-associate logic into a shallow, balanced tree

Unbalanced chain — 4 levels



depth = 4 gate delays

Balanced tree — 2 levels



depth = 2 gate delays (same function)

Other restructuring

Factoring to shorten the late-arriving input's path.

Buffer/repeater insertion on long, heavily-loaded nets.

Cloning/duplication to split a high-fanout driver.

Re-association / tree balancing: turn a deep chain of gates into a balanced tree — same function, fewer logic levels.



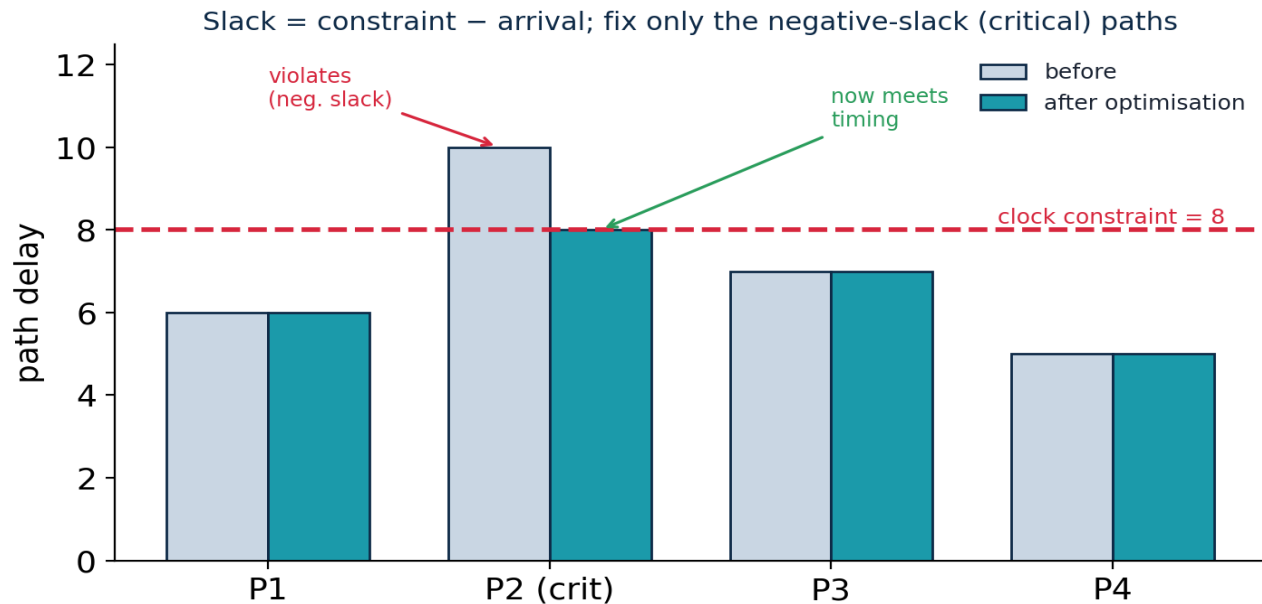
Late-arriving inputs

Restructure so the signal that arrives latest passes through the fewest remaining gates — a powerful, area-cheap speed-up.



Optimising for Performance

Timing optimisation is an iterative loop: find the worst path, fix it, re-analyse — until all slack is non-negative (timing closure).



The optimisation toolbox

Gate sizing — widen critical gates.

Restructuring — balance trees, factor, clone.

Buffer insertion — drive long/loaded nets.

Pipelining — split logic with a register to cut t_{logic} .

Retiming — move registers to balance stages.

$V_{\text{th}}/V_{\text{DD}}$ **tuning** — faster cells on critical paths.



Closure

Fixing one path can expose another; iterate until every path meets its constraint, balancing speed against area and power.

Timing in Sequential Circuits

Real chips have memory. Flip-flops store state on the clock edge — and the clock period must respect their setup and hold requirements.

a

Sequential circuits: flip-flops & registers

Latches vs edge-triggered flip-flops, and registers

b

Setup & hold time analysis

The sampling window and the max/min-delay timing equations

c

Timing considerations & optimization

Clock skew, jitter, metastability, pipelining and retiming

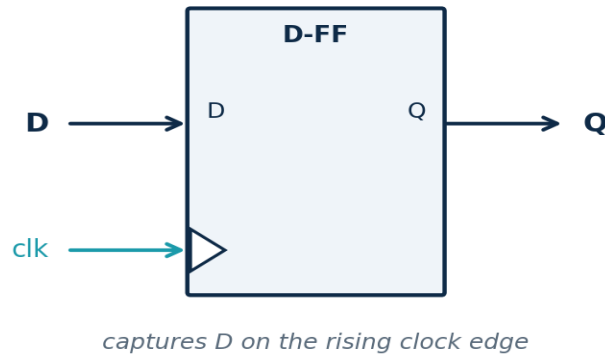




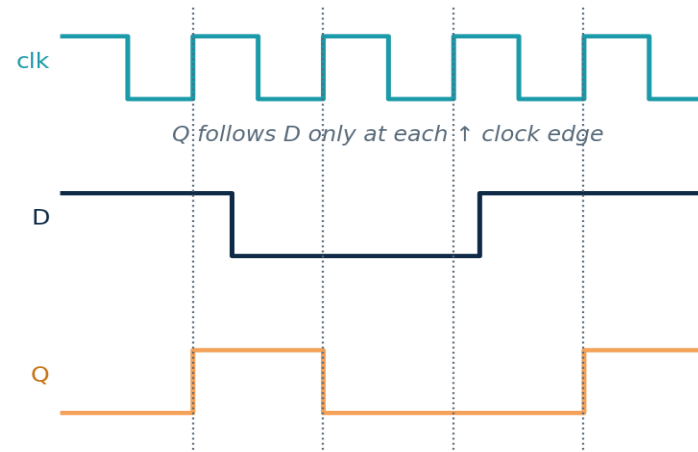
Sequential Circuits & the D Flip-Flop

Combinational logic depends only on its present inputs. Sequential logic also depends on stored state — held in flip-flops and clocked by a common clock.

Edge-triggered D flip-flop



Behaviour: sample-and-hold



A **D flip-flop** copies D to Q on the rising clock edge, then holds it.

Between edges the output is stable — a 1-bit memory.

A **register** is just N flip-flops sharing one clock, storing an N-bit word.

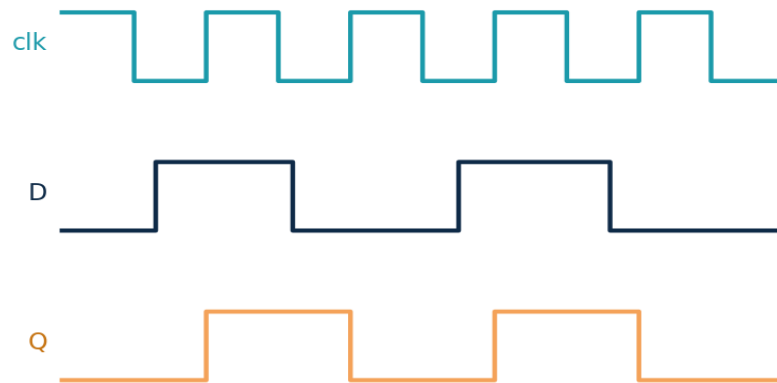
State + logic + clock = a synchronous system.



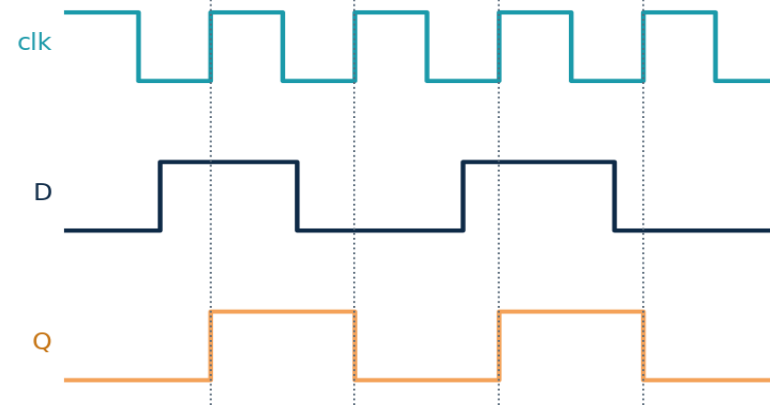
Latches vs Flip-Flops

Both store a bit, but they capture data differently — a distinction that matters for timing.

Level-sensitive latch (transparent when $\text{clk} = 1$)



Edge-triggered flip-flop (samples only at $\uparrow$ edge)



Build-up

2 latches (master + slave) on opposite clock phases = one edge-triggered flip-flop.

Latch (level-sensitive): transparent the whole time the clock is high — Q follows D — and holds when the clock is low.

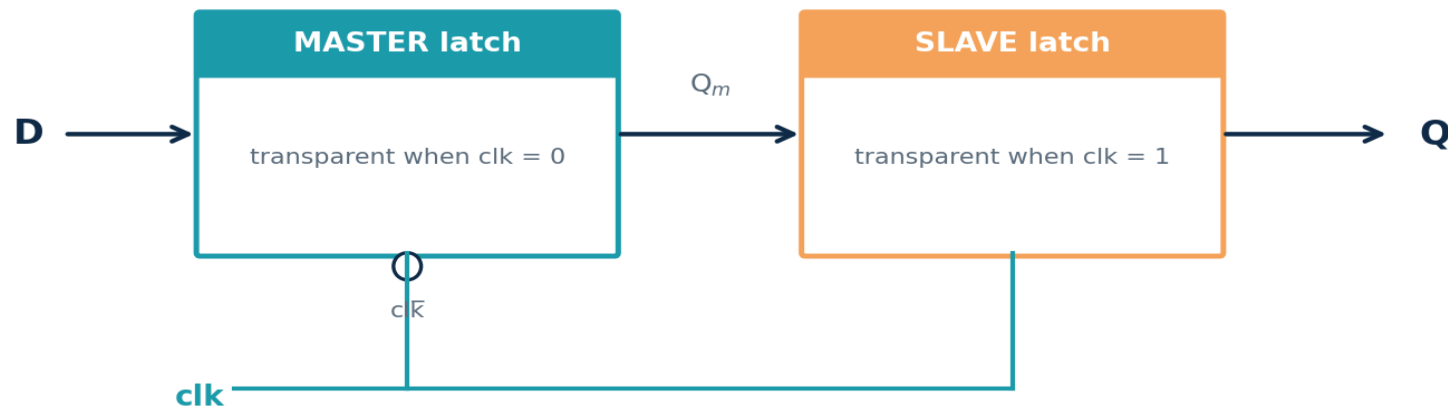
Flip-flop (edge-triggered): samples D only at the clock edge, so the output changes at most once per cycle — easier to reason about, and the default for synchronous design.



Inside the Flip-Flop: Master-Slave

An edge-triggered flip-flop is built from two level-sensitive latches in series, clocked on opposite phases — this is what makes it capture on the edge.

Inside a flip-flop: two latches in series (master-slave)



Master captures D while $\text{clk}=0$; on the rising edge the slave passes it to Q — net effect: edge-triggered

While $\text{clk} = 0$: the master is transparent and tracks D; the slave holds the old value.

On the rising edge: the master freezes its value and the slave becomes transparent, passing it to Q.

Net effect: Q updates once, exactly at the edge — true edge-triggered behaviour.

The master's last value before the edge is why setup time exists.



Flip-Flop Types

The D flip-flop is the workhorse of synchronous design, but the same edge-triggered storage comes in four classic flavours.

Type	Inputs	Next state Q^+	Typical use
D	D	$Q^+ = D$	data storage, registers (most common)
T	T	toggle if $T=1$, else hold	counters, frequency dividers
JK	J, K	set/reset; $J=K=1$ toggles	general-purpose, no invalid state
SR	S, R	$S=1$ set, $R=1$ reset; $S=R=1$ invalid	basic latch / control



Why D dominates

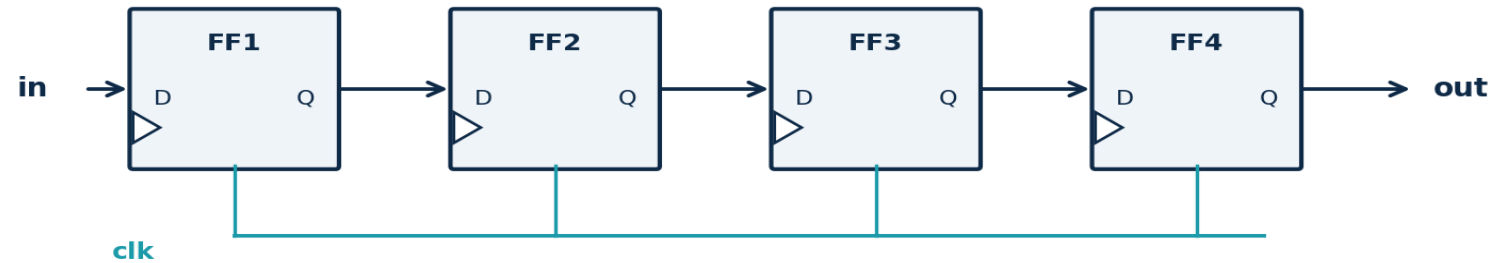
D flip-flops map directly onto how synthesis tools build registers, are easy to time, and avoid the invalid/forbidden states of SR. **T** and **JK** are mainly seen inside counters; **SR** is the underlying latch primitive.



Registers, Shift Registers & Counters

Group flip-flops together and they become the storage and data-movement building blocks of every datapath.

A 4-bit shift register (registers chained together)



On each clock edge every bit moves one stage to the right

Also built from FFs

Counters (toggle FFs that count clock edges).

State machines (a state register + next-state logic).

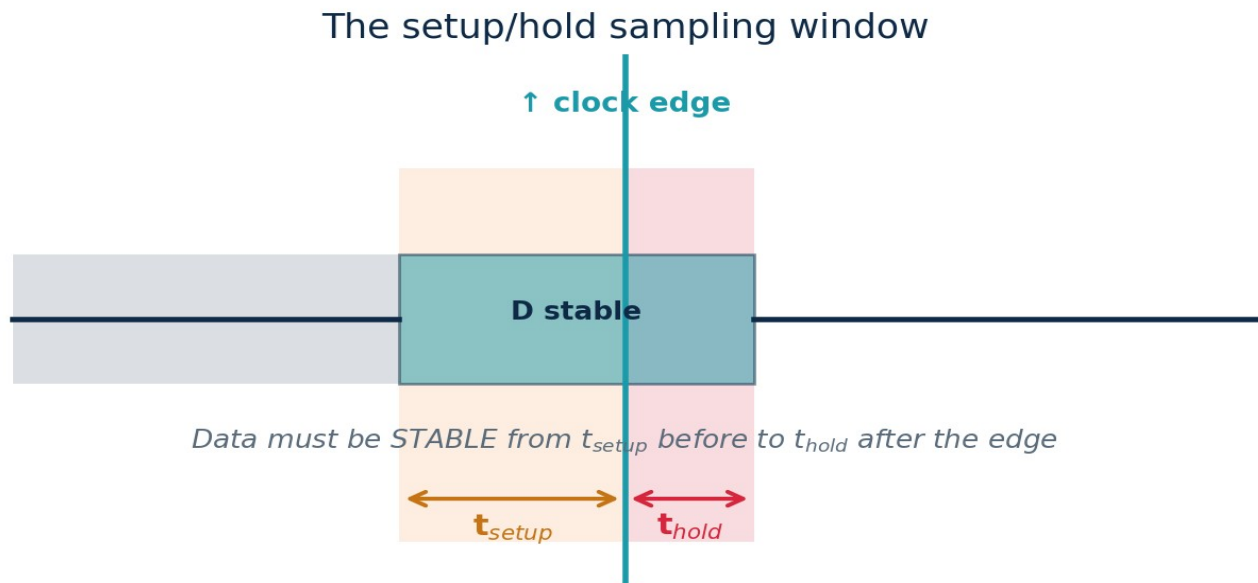
Register: N flip-flops on one clock holding an N-bit word — the basic parallel store.

Shift register: flip-flops chained $Q \rightarrow D$ so data moves one position per clock — used for serial $\leftrightarrow$ parallel conversion and delays.



Setup & Hold Time

For a flip-flop to capture data reliably, the data input must be stable in a window around the clock edge — or the output can go metastable.



Setup time t_{setup} : data must be steady before the edge.

Hold time t_{hold} : data must stay steady after the edge.

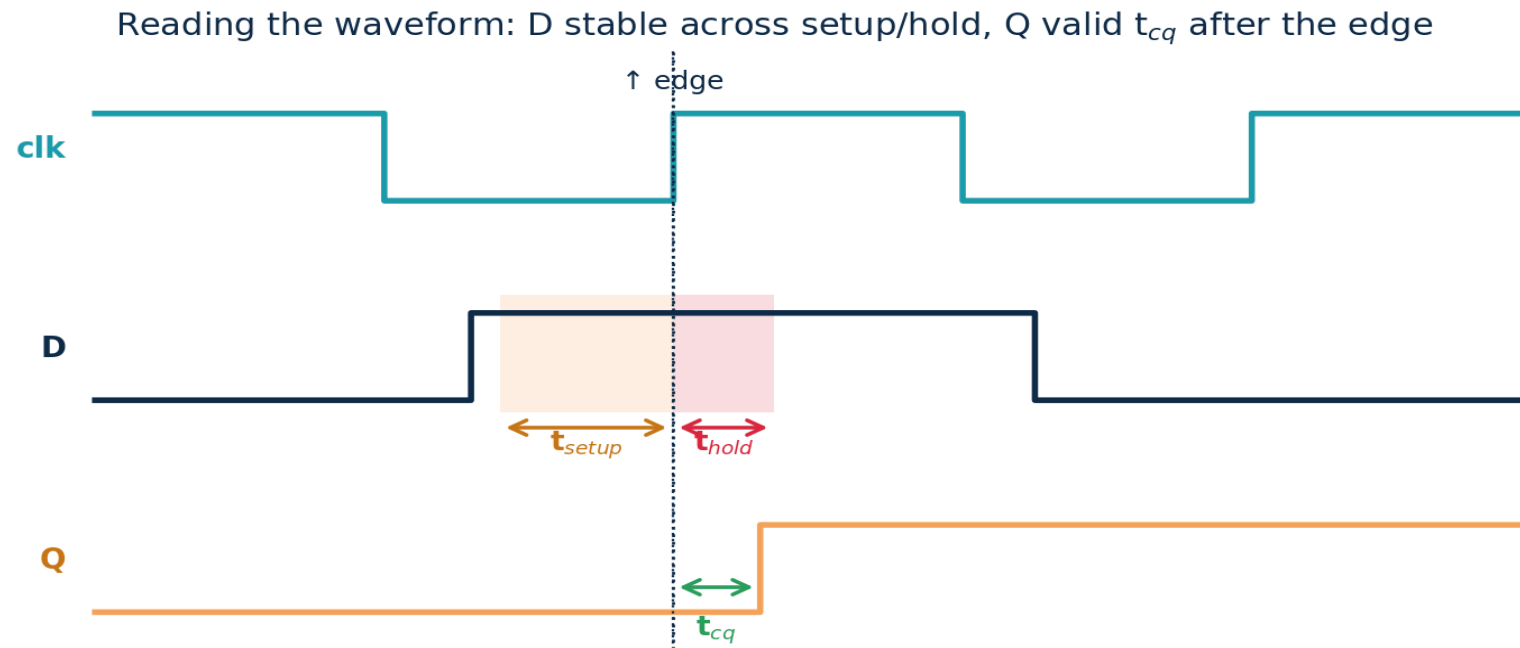
Violate either → **metastability**: the output hovers between 0 and 1 unpredictably.

These set the timing constraints on every path.



Reading a Timing Diagram

Putting clock, data and output on one timing diagram shows exactly where setup, hold and clock-to-Q live.



t_{cq} (clock-to-Q): delay from the edge until Q is valid.

t_{setup} : D must already be steady this long before the edge.

t_{hold} : D must stay steady this long after the edge.

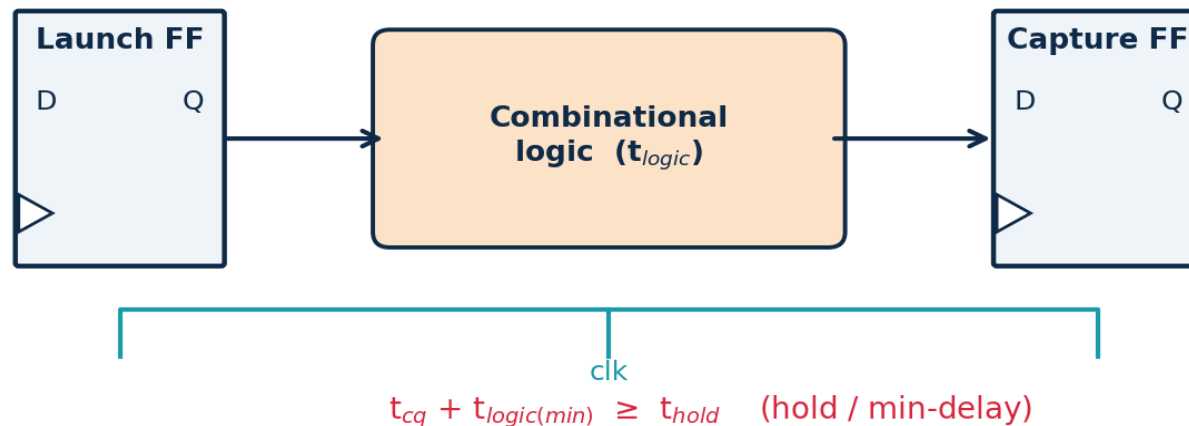
The shaded band is the keep-out window where D must not change.



Timing a Register-to-Register Path

Between two flip-flops sits combinational logic. Two inequalities must hold — one for the slowest path, one for the fastest.

$$T_{clk} \geq t_{cq} + t_{logic} + t_{setup} \quad (\text{setup / max-delay})$$



Worked: max clock

$$t_{cq} = 50\text{ps}, t_{logic} = 300\text{ps}, t_{setup} = 40\text{ps}.$$

$$T_{clk} \geq 390\text{ps} \rightarrow f_{max} \approx 2.56 \text{ GHz}.$$

Setup (max-delay): $T_{clk} \geq t_{cq} + t_{logic} + t_{setup}$ — limits the fastest clock.

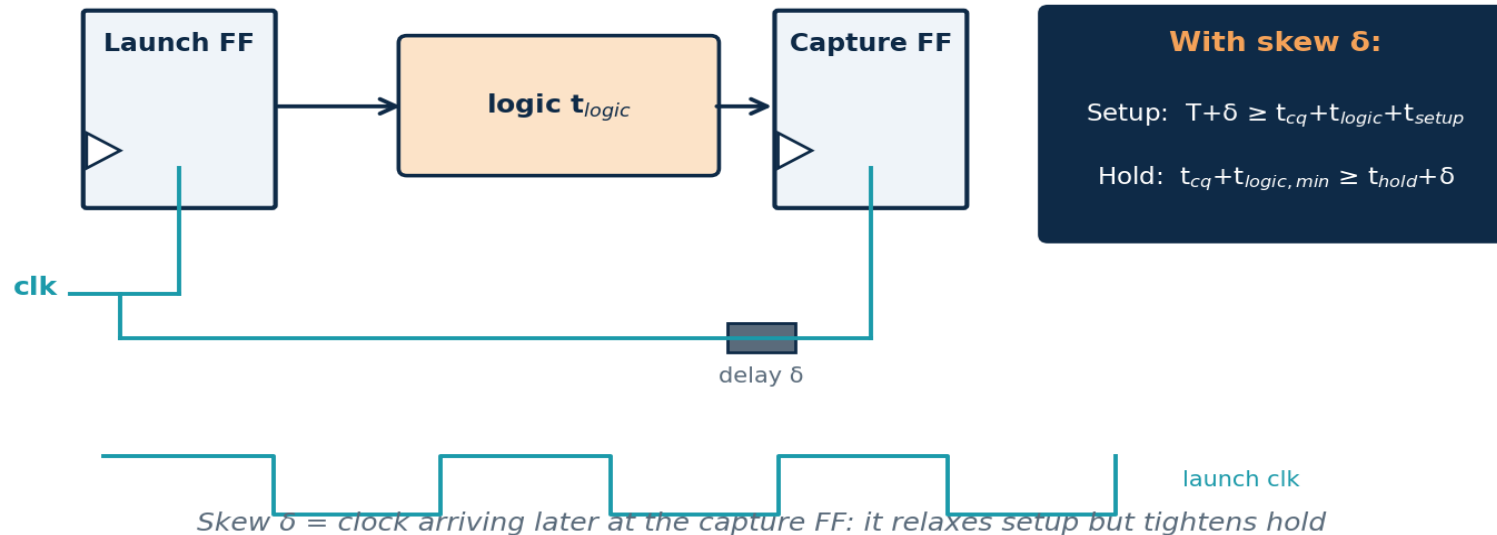
Hold (min-delay): $t_{cq} + t_{logic(min)} \geq t_{hold}$ — independent of clock period; fix with min-delay buffers.



Clock Skew & Timing Margins

The clock never reaches every flip-flop at the same instant. That difference — clock skew — shifts the timing budget, helping one check and hurting the other.

Clock skew shifts the timing budget



With skew δ :

Setup: $T + \delta \geq t_{cq} + t_{logic} + t_{setup}$

Hold: $t_{cq} + t_{logic, min} \geq t_{hold} + \delta$

Skew δ = clock arriving later at the capture flip-flop than the launch one.

Setup relaxes: $T_{clk} + \delta \geq t_{cq} + t_{logic} + t_{setup}$ — the data gets extra time.

Hold tightens: $t_{cq} + t_{logic(min)} \geq t_{hold} + \delta$ — skew eats hold margin.

So skew is a double-edged sword; clock-tree design works to minimise it.



Clock Skew, Jitter & Metastability

Beyond the basic equations, real clocks are imperfect — three effects every sequential design must manage.



Clock skew

the clock reaches different flip-flops at slightly different times; it eats into setup margin (or worsens hold) and must be budgeted.



Clock jitter

cycle-to-cycle variation in the clock edge; it shrinks the usable period, so designers keep a margin.



Metastability

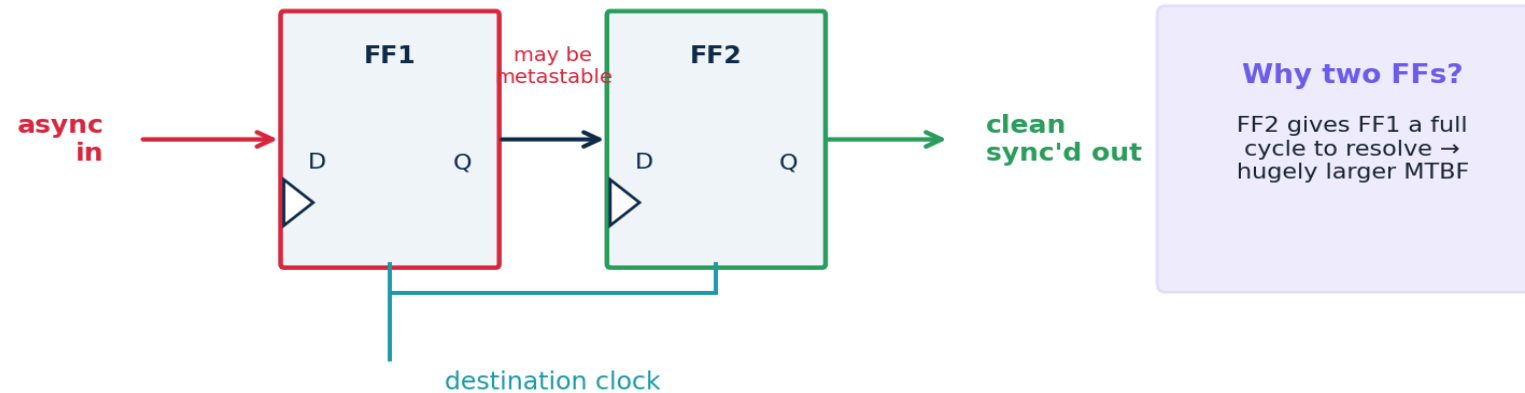
a setup/hold violation can leave a flip-flop's output undefined for a while; synchronizers (two FFs in series) reduce the risk for asynchronous inputs.



Metastability & Synchronizers

When an input is asynchronous (from another clock domain or the outside world), it can violate setup/hold and make a flip-flop metastable. A synchronizer manages the risk.

Two-flip-flop synchronizer (for asynchronous / cross-domain inputs)



Why two FFs?

FF2 gives FF1 a full cycle to resolve → hugely larger MTBF

Rules of thumb

Synchronise every signal crossing a clock domain.

Never fan out an un-synchronised async signal to many places.

Metastability: a flip-flop caught mid-transition can sit between 0 and 1 for an unpredictable time before settling.

Two-FF synchronizer: a second flip-flop gives the first a full clock cycle to resolve, so the chance of a metastable value reaching the logic becomes astronomically small (huge MTBF).



Pipelining & Retiming

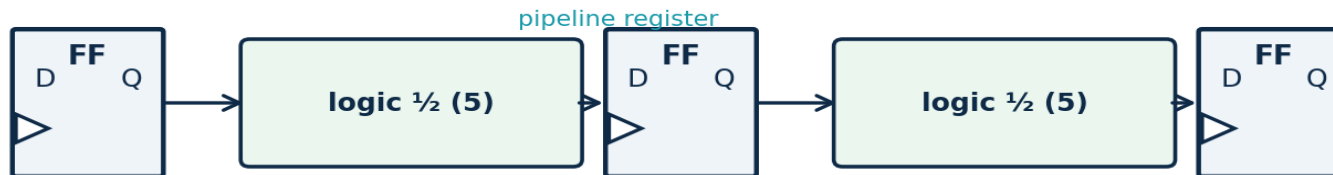
To run a sequential design faster, shorten the logic between registers. Two classic transformations do exactly that.

Before: one deep stage



$T_{clk} \geq 10 \rightarrow$ slow clock

After: pipelined into two shallower stages



each stage = 5 $\rightarrow T_{clk} \geq 5 \rightarrow \sim 2\times$ throughput

Pipelining: insert registers to split a deep logic block into shorter stages $\rightarrow$ smaller t_{logic} $\rightarrow$ faster clock and higher throughput (at the cost of latency).

Retiming: move existing registers across logic to balance stage delays — same function, better worst-case stage.

Also: parallelism, and the gate-level tricks from Topic 9 (sizing, restructuring).



Module 1 Formula Sheet

**Saturation current**

$$I_D = \frac{1}{2} \cdot k \cdot (V_{GS} - V_{th})^2$$

**Propagation delay**

$$t_p \approx 0.69 \cdot R \cdot C$$

**Dynamic power**

$$P_{dyn} = \alpha \cdot C \cdot V_{DD}^2 \cdot f$$

**Noise margins**

$$NM_H = V_{OH} - V_{IH}; \quad NM_L = V_{IL} - V_{OL}$$

**Setup constraint**

$$T \geq t_{cq} + t_{logic} + t_{setup}$$

**Triode current**

$$I_D = k \cdot [(V_{GS} - V_{th}) \cdot V_{DS} - V_{DS}^2 / 2]$$

**Elmore delay**

$$\tau_i = \sum_k R_{ik} \cdot C_k$$

**Energy per switch**

$$E = \frac{1}{2} \cdot C \cdot V_{DD}^2$$

**Logical effort delay**

$$d = g \cdot h + p$$

**Wire delay scaling**

$$t_{wire} \propto (\text{length})^2 \text{ (un-buffered)}$$



WRAP-UP

Module 1 — Key Takeaways



VLSI integrates billions of transistors

designed through a layered flow (spec → GDSII), trading off Power-Performance-Area.



CMOS is complementary

one network conducts at a time → full swing and near-zero static power.



Layout makes it physical

stick diagrams, λ design rules and DRC/LVS turn schematics into silicon.



Timing sets the speed

$t_p \approx 0.69 \cdot RC$; setup/hold and the critical path fix the clock frequency.



The MOSFET is a voltage-controlled switch

NMOS pulls down on a HIGH input; PMOS pulls up on a LOW input.



Every gate = PUN + PDN duals

series ↔ parallel; NAND, NOR and AOI all follow one 4-step recipe.



Wires are not free

RC interconnect adds delay; models and repeaters keep it in check.



Packaging bridges chip & system

bonding, assembly, thermal and advanced packaging finish the job.



Practice makes it permanent

Your companion workbook expands every topic above into guided tutorials and a large graded problem set — all with full worked solutions.

14 Guided Tutorials

Step-by-step: design flow, transistor analysis, gate design, delay, layout & wires.

80 Practice Problems

Conceptual, design and numerical — graded easy → hard, across nine sections.

Full Solutions

Every answer worked out so you can self-assess and learn.

Up next → Module 2: Verilog RTL Coding for Synthesis